Agent 开发面经

工程实答 + 八股文

2026-10-28 更新 · 642 题 · 11 家公司 · 23 轮

保留原 PDF 的公司、轮次、题号与全部问题。工程涉及的题根据当前源码回答；工程未涉及的题提供八股文答案，直接解释概念、实现流程、取舍、失败处理与常见追问。

当前项目：微信小程序 + FastAPI + LangGraph + SQLite。面向单用户自然语言健康记录、纠错、计划确认与复盘；不是通用企业 Agent 平台。

工程题重点：模型外发授权与预算、纯本地模式、四层记忆、血缘删除、受控输出、可靠调度、并发控制和分层观测；每题附当前源码依据。

八股文重点：RAG、混合检索与重排、多 Agent、ReAct、Skill/MCP、DSL、分布式任务、SSE、沙箱、Python/Java 和算法。不再以“工程没有实现”代替知识回答；通用设计也不写成个人项目经历。

验证口径：本次核对题目完整性、源码路径与 PDF 排版，未运行真实模型、未访问个人健康数据库、未重跑全量应用测试。历史报告只代表其记录的环境与时间。个人贡献、客户规模、薪酬与效果数字需本人据实补充。

两分钟项目介绍

我做的是“轻记”个人健康记录 Agent：用户在微信小程序用自然语言记录饮食、体重、睡眠、运动和生理周期，系统先理解输入，再由本地规则判断是否需要追问、能否保存和是否需要确认。LangGraph 管理暂停与恢复，SQLite 分别保存业务事实和工作流进度。

工程重点是把模型的不确定性限制在可验证范围内：模型只产生结构化计划或选择已批准的表达，业务计算、证据校验、幂等写入和目标确认由代码执行。模型只看本次用途允许的数据，纯本地模式可以禁止外发。记录纠错和删除会沿数据血缘处理相关派生结果。

可靠性方面有请求重放、运行预算、模型 I/O 期间释放锁、后台任务租约与 outbox，以及不采集健康原文的运行观测。目前仍是单用户、单 Engine 原型。个人负责的模块应按实际经历补充。

答题方法

工程题按“实际机制、一次执行路径、验证与局限”讲；八股文按“定义、核心步骤、取舍、失败与验证”讲。追问恢复时同时讲 checkpoint 与业务幂等；问权限时同时讲可见范围与执行端授权；问指标时说明分母、窗口和失败/降级口径。算法代码见附录，个人经历按模板补充真实信息。

一、字节跳动

一面

1. 为什么选择 LangGraph？与其他 Agent 框架相比，看中了哪些能力？

参考答案：轻记需要两类等待：记录缺字段时等用户补充，计划变化较大时等用户确认。因此选择 LangGraph 的显式状态图、interrupt 和 SQLite checkpoint。图负责编排，草稿完整性、营养来源、请求去重和计划版本检查由自己的业务代码实现。当前没有统一任务基准证明它优于其他框架；选择主要依据是与这两类暂停恢复需求匹配。

[Q001 · 工程现状] 依据：backend/app/workflows/engine.py; backend/tests/test_agent.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

2. ReAct 循环在什么情况下会迟迟不结束？

参考答案：ReAct 将决策、行动和观察交替执行，直到模型给出最终答案或命中终止条件。循环不结束常见于目标不明确、工具连续失败、重复查询、观察缺少关键信息以及上下文丢失目标。运行时应同时设轮次、总时长、Token/费用和重试预算，检测相同参数调用与连续无进展；超过限制进入明确失败、等待补充或人工处理。最终答案还需验证必需步骤和产物，不能只相信模型说“完成”。

[Q002 · 八股文 · 通用知识] 依据：参考资料：S1

3. 追问：如何限制循环次数、上下文规模和工具调用范围？

参考答案：轻记采用受控工作流而非开放式 ReAct。可选追问受轮次限制，必需字段不全不会强行保存。当前每个工作流最多预留 32 次模型调用、累计 400000 字节，其中规划最多 4 次、安全分类最多 8 次；发送前持久化计数，失败也占预算。单次外发限制 90000 字节。运行段默认 120 秒、并发槽 2，普通模型超时 45 秒，安全分类 8 秒。字节和次数预算不等于精确 Token 计费，也不代表已实现自动摘要压缩。

[Q003 · 工程现状与边界] 依据：backend/app/infrastructure/repositories/common.py; backend/app/modeling/boundary.py; backend/app/common/config.py; backend/app/common/execution_budget.py

4. Tool Calling 的协议怎么设计？模型看到的工具描述与实际执行实现如何对应？

参考答案：工具注册表维护稳定 tool_id、版本、用途说明、输入/输出 JSON Schema、执行适配器、权限、超时、重试和幂等策略。模型只看经身份与场景过滤后的描述，返回工具名、参数和调用 ID；服务端查注册表，先校验参数、资源归属与风险，再调用真实实现，返回关联原调用 ID 的结构化结果。未知工具、额外字段和越权参数必须拒绝；模型可见性筛选不能替代执行端鉴权。

[Q004 · 八股文 · 通用知识] 依据：参考资料：S3,S9

5. Agent 执行到一半进程崩溃，恢复后怎样找到正确的执行位置？

参考答案：Engine 启动时打开 checkpoints.db 并初始化 SqliteSaver。追问草稿、待补字段和节点进度以 thread_id 保存；恢复接口核对工作流和 interrupt_id，再用同一 thread_id 执行 Command(resume)。失败的初始请求可以用相同 request_id 重试，已有 checkpoint 时从图状态继续。健康事实以 health.db 为准，两库提交不是同一原子事务，所以仍需要业务幂等。

[Q005 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

6. 追问：如果外部工具已经执行过了，恢复时怎么避免重复调用、重复产生副作用？

参考答案：轻记把一次事件写入的操作键固定为 thread_id:event:事件下标，Repository 在事务内先查 operations，再写 events、revisions 和操作结果；重放时返回旧结果。HTTP 层另用 request_id 和请求指纹识别重试。它覆盖当前本地数据库副作用；若以后调用外部支付或推送服务，还必须由外部服务接受幂等键，checkpoint 本身不提供跨系统恰好一次。

[Q006 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

7. 面对理解需求、检索资料、生成报告等多步骤任务，会选择单 Agent 加工具，还是多 Agent？

参考答案：单 Agent 用一个决策上下文调用工具，适合目标统一、工具可控的开放任务；Workflow 用预设节点和条件边控制流程，适合步骤稳定、合规或一致性要求高的业务；多 Agent 适合独立子目标可并行、上下文或权限需隔离的场景。选择时比较完成率、延迟、成本、可调试性和协调开销。任务复杂并不必然需要多个 Agent；常见组合是外层 Workflow 控制阶段，局部 Agent 自主探索。

[Q007 · 八股文 · 通用知识] 依据：参考资料：S1,S6

8. 追问：Manager-Worker 这样的架构适合什么情况？与单 Agent 的取舍是什么？

参考答案：Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果，Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema；结果返回状态、证据及产物引用。Worker 可预声明，也可受控动态创建，动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并，副作用用幂等键；检测循环委派并传播取消，避免所有 Worker 同时修改同一业务对象。

[Q008 · 八股文 · 通用知识] 依据：参考资料：S1,S6

9. Agent 运行时记录了哪些 Metrics、Trace 和 Log？

参考答案：已有运行元数据白名单、HTTP 关联 ID、节点/模型/数据库/锁/任务分层耗时，以及 Prometheus Counter、Histogram、Gauge。模型传输成功、Schema 成功和业务完成分别计数。业务质量贡献在同一业务事务写入，按保留窗口聚合并参与删除。日志和 span 不采集健康原文、完整 prompt 或模型响应；业务 process_trace 不能直接复制到遥测。指标导出默认关闭，尚无完整集中观测平台或分角色运维权限。

[Q009 · 工程现状与边界] 依据：backend/app/observability/metrics.py; backend/app/observability/spans.py; backend/app/observability/contracts.py; backend/app/api/observability.py; backend/tests/test_observability.py

10. RAG 使用什么向量存储？

参考答案：向量存储按数据量、过滤能力、更新频率、一致性、延迟和运维条件选择。已有 PostgreSQL 且规模合适时，可评估向量扩展；搜索与关键词过滤需求强时，可评估搜索引擎的向量能力；需要专用向量服务时，再比较其索引、分片和混合检索能力。选型要用同一数据与查询集测试 Recall@K、P95、写入/删除传播和资源成本。保存 doc_id、chunk_id、权限、版本和来源；向量索引通常可重建，不能替代原文真值库。

[Q010 · 八股文 · 通用知识] 依据：参考资料：S2

11. 检索返回了很多候选片段，但上下文空间有限，如何排序、压缩和组装?

参考答案：上下文按任务目的构造。首轮规划只收到本次文字、时间和是否选中记录，不再默认携带最近 12 条历史。本地审核计划后，从固定视图取必要数据；纠错绑定单条记录，补充只发送当前草稿和待补字段，营养估算只发送本次食物与分量。建议使用白名单结构化投影，不发送历史 raw_text。目的、资源和实际 payload 都经授权核对，最终返回用途与数据类别说明。

[Q011 · 工程现状与边界] 依据：backend/app/modeling/payloads.py; backend/app/modeling/projections.py; backend/app/modeling/boundary.py; backend/tests/test_model.py

12. Agent 如何部署？逻辑上的 Agent 与进程、Worker 是什么关系?

参考答案：当前入口是 Taro/React/TypeScript 微信小程序，连接独立 FastAPI JSON API；旧 Web 前端已废弃，根路径不再托管页面。Engine 复用聊天、计划、分析图，health.db 保存业务事实，checkpoints.db 保存图进度。开发启动用 start.ps1；仓库另有 Docker 加固部署配置。后台任务与 Web 仍共享唯一 Engine，数据目录所有者锁阻止第二实例，不是独立 Worker 集群。正式对外服务仍需 HTTPS、鉴权及相应部署验收。

[Q012 · 工程现状与边界] 依据：miniprogram/package.json; backend/app/main.py; backend/app/infrastructure/data_owner.py; README.md; compose.hardened.yaml; start.ps1

13. 追问：每个 Agent 都需要一个独立运行实例吗?

参考答案：当前入口是 Taro/React/TypeScript 微信小程序，连接独立 FastAPI JSON API；旧 Web 前端已废弃，根路径不再托管页面。Engine 复用聊天、计划、分析图，health.db 保存业务事实，checkpoints.db 保存图进度。开发启动用 start.ps1；仓库另有 Docker 加固部署配置。后台任务与 Web 仍共享唯一 Engine，数据目录所有者锁阻止第二实例，不是独立 Worker 集群。正式对外服务仍需 HTTPS、鉴权及相应部署验收。

[Q013 · 工程现状与边界] 依据：miniprogram/package.json; backend/app/main.py; backend/app/infrastructure/data_owner.py; README.md; compose.hardened.yaml; start.ps1

14. 后端平台与 AI 平台如何协作，哪些能力可以复用?

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q014 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

15. 如何统一接入不同的模型 Provider?

参考答案：模型通过兼容 Chat Completions 的传输层接入，供应商地址、模型名和凭据由运行配置显式提供，不能把示例模型名当作运行时事实。源码不再硬编码密钥；支持仓库外密钥文件，开发模式也可用环境变量。配置全空进入演示模式，部分缺失会失败；模型失败不会悄悄切换演示营养。当前没有多供应商路由网关，换模型需回归结构化输出、错误映射和数据边界。

[Q015 · 工程现状与边界] 依据：backend/app/common/config.py; backend/app/modeling/parser.py; backend/app/modeling/transport.py; backend/tests/test_config.py

16. 追问：不同 Provider 的流式协议、限流、降级和成本统计怎么处理？

参考答案：在模型适配层统一非流式结果、增量事件、结束原因、错误码和 usage，保留供应商特有能力而不强求虚假一致。按模型与租户设置并发、RPM/TPM 及费用预算，429/瞬时网络错误有限退避并尊重 Retry-After。降级前检查备用模型的工具/结构化能力与数据许可，防止切换破坏契约。成本按实际输入、输出和缓存用量配合价格版本计算，区分一次传输、一次模型调用和整个任务；流中断的重试还需处理重复内容。

[Q016 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

17. 【算法题】 Two Sum 类问题，如何通过一次遍历找到目标组合？题面信息不完整，不还原额外约束。

参考答案：一次遍历，用哈希表保存已访问数值到下标。遍历 x 时先查 target-x 是否在表中，命中则返回两个下标，否则记录 x。先查询后插入可避免同一元素被重复使用，也能处理 [3,3]、target=6。平均时间 O(n)，空间 O(n)；若要求返回所有组合或输入已排序，需按题意调整。题面未明确约束时先确认返回下标还是数值、是否保证有解、能否修改输入。代码见附录 A。

[Q017 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

二面

1. 为什么考虑换工作？目前的工作地点和职级情况如何？

参考答案：先用正向原因说明换工作的目标，再如实给出当前工作城市、可接受地点、现职级和职责范围。不同公司的职级不可直接对齐，可以用负责模块、决策范围和团队影响补充。不要回避地点限制，也不要把薪资作为唯一原因。

[Q018 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. AI 应用平台最初为什么立项，面向什么用户？

参考答案：轻记面向希望用自然语言记录饮食和身体状态的个人用户。核心问题是表单记录成本高、补记和纠错容易出错、后续建议缺少可追溯依据。当前交付是单用户微信小程序原型，具备五类记录、关键追问、历史查询、目标确认、复盘与站内提醒；没有企业租户、客户应用发布市场或通用低代码平台。

[Q019 · 工程现状] 依据：backend/app/main.py; miniprogram/src/app.tsx; README.md

3. 追问：现在有哪些真实使用场景，用户实际怎么使用？

参考答案：真实的产品使用路径是：建档后输入“昨天中午吃了一碗牛肉面”，补充分量，保存后在历史页纠错；计划页填写或采用默认目标并确认；饮食记录完整后标记完整，再生成复盘。代码证明这些流程存在，但仓库没有可核验的活跃用户量、使用频率或真实减重效果，不能补造运营数据。

[Q020 · 工程现状] 依据：miniprogram/src/app.tsx; README.md

4. 用户在平台上创建的应用如何发布、嵌入或集成到已有系统？

参考答案：应用先经过版本校验、权限检查、测试和审批，再发布为固定版本。外部系统可通过 API、SDK、网页组件或事件订阅接入。运行时要传递用户身份和租户信息，并支持灰度、回滚、限流和调用审计。

[Q021 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

5. 如果从平台研发转向客户现场交付，可以沉淀哪些方法和可复用资产？

参考答案：这个工程可以沉淀的资产是五类事件 Schema、草稿到严格记录的校验流程、营养来源追踪、request_id/操作键幂等模板、计划差异确认和回归用例。它们比某个用户的饮食原文更适合复用。若交付给其他健康场景，应把目标计算、提醒和统计模板作为领域策略替换，同时保留恢复与验证接口。

[Q022 · 工程现状] 依据：backend/app/api/schemas.py; backend/tests/

6. 通用运行内核、业务扩展与服务入口各负责什么？

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q023 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

7. 业务扩展由谁开发？客户最终买到的是产品、服务还是一套定制应用？

参考答案：业务扩展可由平台团队、交付团队或客户研发按统一接口开发，平台团队负责规范、审核和运行保障。客户购买的形态必须说清，可以是云服务、私有部署产品或产品加扩展包。专有代码的归属、升级和运维责任也要明确。

[Q024 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

8. 云端部署与私有化部署分别怎么做？

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

[Q025 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

9. 追问：客户的数据不能出内网时，模型部署和调用链路如何处理？

参考答案：本地存储不等于模型模式下不外发。当前可由服务端或单次请求启用 local_only，统一禁止规划、纠错、估算与表达等外部调用，该约束随暂停流程保存，恢复不能放宽。模型模式按用途授权与最小字段投影发送，并披露实际调用类别。加固部署提供受限模型出口和密钥文件配置；若要求完全内网运行，需使用纯本地模式或经过验证的内网模型，并检查部署出口，不能只依赖提示词。

[Q026 · 工程现状与边界] 依据：backend/app/modeling/boundary.py; backend/app/modeling/payloads.py; backend/app/common/config.py; deploy/network-policy.md

10. 工具在平台中怎样抽象？能否讲清一次工具调用的链路？

参考答案：工具注册表维护稳定 tool_id、版本、用途说明、输入/输出 JSON Schema、执行适配器、权限、超时、重试和幂等策略。模型只看经身份与场景过滤后的描述，返回工具名、参数和调用 ID；服务端查注册表，先校验参数、资源归属与风险，再调用真实实现，返回关联原调用 ID 的结构化结果。未知工具、额外字段和越权参数必须拒绝；模型可见性筛选不能替代执行端鉴权。

[Q027 · 八股文 · 通用知识] 依据：参考资料：S3,S9

11. Workflow 的工具节点和 ReAct 的 Tool Calling 能否复用同一套执行能力?

参考答案：可以复用同一套工具注册、参数校验、鉴权、超时、幂等和执行适配器。区别只在调用来源：Workflow 由固定节点发起，ReAct 由模型决策发起。两者都必须经过相同的服务端安全检查，并返回统一工具结果。

[Q028 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

12. 运行时对 LangGraph 的依赖有多深?

参考答案：当前 LangGraph 承担 StateGraph、条件路由、interrupt 和 checkpoint；Engine 另外管理请求指纹、回执、锁和 API 结果。营养准备、历史检索、计划版本、统计、归档都在普通 Python 模块中。边界仍有耦合：Engine 把图和业务函数连在一起，还不是可插拔通用 Runtime，但更换编排层不必重写所有健康算法。

[Q029 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

13. 追问：Python 性能是否会成为瓶颈?

参考答案：同步模型 I/O 仍在线程内执行，但已不再在整个网络等待期间持有 Engine 全局锁：model_io 释放本地锁，返回后重取并检查执行代次及读依赖。相同工作流/会话通过键控锁协调，SQLite 写事务仍需互斥；RuntimeLimiter 默认允许两个运行段。数据目录仅允许一个 Engine，不能直接增加 Uvicorn workers 扩容。现有局部基准不代表生产 QPS，应分别测锁等待、模型、数据库和端到端负载。

[Q030 · 工程现状与边界] 依据：backend/app/workflows/concurrency.py; backend/app/common/execution_budget.py; backend/app/infrastructure/data_owner.py; backend/tests/test_workflow_concurrency.py

14. 用户身份和角色如何传递到运行时？工具和 Skill 的可见范围、调用权限怎样控制?

参考答案：身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q031 · 八股文 · 通用知识] 依据：参考资料：S9,S10

15. 日志、Trace 和指标存在哪里？不同人员如何获得查看和排障权限?

参考答案：已有运行元数据白名单、HTTP 关联 ID、节点/模型/数据库/锁/任务分层耗时，以及 Prometheus Counter、Histogram、Gauge。模型传输成功、Schema 成功和业务完成分别计数。业务质量贡献在同一业务事务写入，按保留窗口聚合并参与删除。日志和 span 不采集健康原文、完整 prompt 或模型响应；业务 process_trace 不能直接复制到遥测。指标导出默认关闭，尚无完整集中观测平台或分角色运维权限。

[Q032 · 工程现状与边界] 依据：backend/app/observability/metrics.py; backend/app/observability/spans.py; backend/app/observability/contracts.py; backend/app/api/observability.py; backend/tests/test_observability.py

16. 排障效率的提升具体怎么测量?

参考答案：先统一故障开始、开始处理、完成定位和恢复服务的时间定义。然后统计定位耗时、恢复耗时、一次定位成功率 、人工操作次数、人员交接次数和问题再次打开率。耗时既要看中位数，也要看 P95，避免少量极慢故障被平均值掩盖。只有故障等级、系统范围和统计周期一致，改动前后的数据才可以比较。

[Q033 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

17. 追问：比较基线和统计范围是什么，如何证明是这次改动带来的提升?

参考答案：基线应选择改动前的原有排障流程，或者同期未使用新方案的对照组。统计范围要固定系统、故障类型、严重等级、值班方式和时间窗口。归因时优先使用同一批故障回放、分批上线或对照实验，并排除流量、人员经验和故障难度的影响。如果只有简单的改动前后对比，只能说明两者相关，不能完全证明提升由本次改动造成。

[Q034 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

18. 是否考虑过建设统一控制面？当前为什么做或不做?

参考答案：当前控制配置包括 Settings、档案、偏好、目标来源和规则版本，运行执行在 Engine 三张图里，均由同一个 FastAPI 服务承载。单用户和少量固定图还不需要独立控制面。以后有多模型、多用户、动态发布版本时，再把配置管理、发布、权限和评测服务拆出，运行任务只引用不可变版本。

[Q035 · 工程现状] 依据：backend/app/common/config.py

19. 你在项目中具体负责哪些部分?

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

[Q036 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

20. 行业 Agent 实际解决了什么问题，交付物是什么?

参考答案：轻记解决的是个人健康记录从“随口说”到“可纠错、可统计”的转换问题。交付包括微信小程序、五类事件数据、可恢复工作流、计划与复盘。工程价值可用字段正确率、重试零重复、追问完成率、过时计划拦截率和用户记录完整度衡量；当前没有生产付费客户和医学效果数据，职责也应以本人实际贡献为准。

[Q037 · 工程现状] 依据：backend/tests/test_agent.py; README.md

21. 托管式 Agent Runtime 是否可行，客户会因为什么选择它?

参考答案：托管服务是可行的，前提是能提供稳定运行、弹性资源、模型与工具接入、状态恢复、权限、观测和成本治理。 客户选择它的原因通常是减少基础设施建设和运维，但数据边界、私有化和供应商锁定必须明确。

[Q038 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

22. 【算法题】 将两个有序数组原地合并到具有预留空间的第一个数组中。

参考答案：设第一个数组有效长度 m、第二个 n，第一数组总容量至少 m+n。用 i=m-1、j=n-1、k=m+n-1，从后向前把较大值写到 k，避免覆盖尚未读取的有效元素。循环直到第二数组耗尽；第一数组剩余元素已在正确位置。时间 O(m+n)，额外空间 O(1)。注意 m=0、n=0、相等元素及预留区不属于有效输入。代码见附录 B。

[Q039 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

三面

1. AI 应用平台的整体架构怎么设计，采用了哪些技术?

参考答案：当前入口是 Taro/React/TypeScript 微信小程序，连接独立 FastAPI JSON API；旧 Web 前端已废弃，根路径不再托管页面。Engine 复用聊天、计划、分析图，health.db 保存业务事实，checkpoints.db 保存图进度。开发启动用 start.ps1；仓库另有 Docker 加固部署配置。后台任务与 Web 仍共享唯一 Engine，数据目录所有者锁阻止第二实例，不是独立 Worker 集群。正式对外服务仍需 HTTPS、鉴权及相应部署验收。

[Q040 · 工程现状与边界] 依据：miniprogram/package.json; backend/app/main.py; backend/app/infrastructure/data_owner.py; README.md; compose.hardened.yaml; start.ps1

2. 为什么选择 LangGraph?

参考答案：轻记需要两类等待：记录缺字段时等用户补充，计划变化较大时等用户确认。因此选择 LangGraph 的显式状态图、interrupt 和 SQLite checkpoint。图负责编排，草稿完整性、营养来源、请求去重和计划版本检查由自己的业务代码实现。当前没有统一任务基准证明它优于其他框架；选择主要依据是与这两类暂停恢复需求匹配。

[Q041 · 工程现状] 依据：backend/app/workflows/engine.py; backend/tests/test_agent.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

3. DSL 如何定义，又如何转换为执行图?

参考答案：DSL 应声明版本、节点 ID/类型、参数、类型化输入输出、边、条件、变量引用、超时与错误处理。编译流程是解析 Schema、检查节点和端口、校验引用及类型、分析可达性与依赖、验证循环边界，再生成执行图。自然语言先生成受限中间计划，再编译为 DSL，避免模型直接拼接任意代码。执行前做静态检查与沙箱测试，运行时仍检查权限和写入条件；静态合法不等于业务正确。

[Q042 · 八股文 · 通用知识] 依据：参考资料：S6,S9

4. LangGraph 使用过程中遇到过什么问题?

参考答案：可讲一个有测试证据的困难场景：用户说“昨天吃得有点咸”，其中没有食物名称。如果直接要求严格餐食 Schema，会校验失败；若猜一个食物又会污染数据。现在先保留宽松 ExtractionEvent，由本地 check_drafts 决定是否追问；补充只更新待补字段，保留首次日期。测试覆盖重启和跨午夜补充，不能据此虚构某次线上事故。

[Q043 · 工程现状] 依据：backend/app/chat/drafts.py; backend/tests/test_draft_clarification.py

5. 追问：是否做过性能测试，系统容量如何?

参考答案：同步模型 I/O 仍在线程内执行，但已不再在整个网络等待期间持有 Engine 全局锁：model_io 释放本地锁，返回后重取并检查执行代次及读依赖。相同工作流/会话通过键控锁协调，SQLite 写事务仍需互斥；RuntimeLimiter 默认允许两个运行段。数据目录仅允许一个 Engine，不能直接增加 Uvicorn workers 扩容。现有局部基准不代表生产 QPS，应分别测锁等待、模型、数据库和端到端负载。

[Q044 · 工程现状与边界] 依据：backend/app/workflows/concurrency.py; backend/app/common/execution_budget.py; backend/app/infrastructure/data_owner.py; backend/tests/test_workflow_concurrency.py

6. 平台质量怎么衡量，主要看哪些指标?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q045 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

7. 平台后续如何演进？业务 Agent 是否直接建立在原平台之上?

参考答案：演进重点应放在稳定接口和可替换能力：模型、框架、工具协议和记忆实现都通过适配层接入。内核保持任务状态、权限、恢复、观测和评测的稳定，新的模型或框架先经过兼容性和回归评测，再逐步替换。

[Q046 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

8. LangGraph 之外，团队自行实现了哪些能力?

参考答案：LangGraph 之外，工程已经实现请求幂等、恢复回执、模型外发授权与预算、结构化事实冲突、数据血缘删除、受控输出、安全路由、可靠后台任务和分层观测。下一步优先补真实营养来源、真实模型评测及目标部署验收，再按需要做鉴权、多用户和进程拆分。没有文档 RAG、MCP、动态 Skill、任意代码执行或多 Agent 集群，不把这些当作既有成果。

[Q047 · 工程现状与边界] 依据：backend/app/workflows/runtime.py; backend/app/modeling/boundary.py; backend/app/privacy/deletion.py; backend/app/jobs/runner.py; backend/app/observability/service.py

9. 运行时怎样划分职责，通用执行层与业务扩展如何分工?

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q048 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

10. 【工程编码题】 写出 ReAct Loop 伪代码，并说明如何通过 Hook 接入 Skill、Memory 等能力。

参考答案：最小 ReAct Loop 的步骤是：构建受预算约束的上下文，调用模型，解析 final 或 tool_calls；若为工具调用，先做 Schema、权限及幂等检查，执行后把 Observation 反馈给下一轮，并保存检查点。循环检查取消、截止时间、最大轮数、重复调用和完成条件。Hook 可插在构建上下文前、模型前后、工具前后和任务结束时，用来接入 Memory、Skill、日志及策略。示例伪代码见附录 C。

[Q049 · 八股文 · 通用知识] 依据：参考资料：S1

11. Memory 在什么时候检索、更新和保存?

参考答案：记忆写入由业务边界控制：完整事件经校验、营养准备和事务入账；明确长期信息走结构化事实服务，带证据、有效期和修订。助手建议、模型推理或一次偏咸反馈不会自动升级为长期事实。读取使用与任务目的匹配的投影，纠错后让相关派生结果过期。管理接口支持新增、修改、撤回、冲突选择、历史与隐私删除；它仍是领域服务，不是动态 Memory 插件协议。

[Q050 · 工程现状与边界] 依据：backend/app/health/facts.py; backend/app/api/profile.py; backend/app/infrastructure/repositories/facts.py; backend/app/modeling/projections.py

12. 工具调用超时怎么处理?

参考答案：当前模型接口非流式，普通调用默认 45 秒超时，安全分类默认 8 秒，并受运行段截止时间、响应大小和持久化调用预算约束。解析失败返回安全错误，营养失败保留未知，建议选择失败回退本地已批准表达；它们不是同一降级语义。后台可重试任务有独立退避与失败分类，不等于所有模型请求自动重试。尚无供应商级 Token 费用统计或自动备用模型切换。

[Q051 · 工程现状与边界] 依据：backend/app/modeling/transport.py; backend/app/modeling/errors.py; backend/app/common/config.py; backend/app/jobs/runner.py

13. 页面关闭后，后台任务如何继续执行?

参考答案：后台规划器按时间生成持久任务，执行器负责领取、执行、心跳与重试，当前仍同进程共享唯一 Engine。关闭小程序后，只要服务存活就能生成站内消息；服务停机期间不会实时通知，重启按限定窗口补跑。失败区分可重试与终止，达到上限进入可处理的失败状态。没有设备推送、SSE 字流续传或独立分布式队列；交互请求的重试仍使用原 request_id。

[Q052 · 工程现状与边界] 依据：backend/app/jobs/scheduler.py; backend/app/jobs/planner.py; backend/app/jobs/runner.py; backend/app/jobs/manage.py; backend/tests/test_scheduler_operations.py

14. 多个 Worker 怎么避免执行冲突?

参考答案：当前后台任务已有原子领取、lease_until、owner_token、递增 fencing_token 和 job generation。心跳续租；提交与失败更新必须匹配有效执行权，旧执行者不能覆盖新结果。业务效果用稳定幂等键，报告/通知等通过事务和 outbox 协调。该协议提高单 Engine 内的恢复可靠性，并不表示已支持分布式 Worker；独立运行进程和共享集群属于后续阶段，外部服务副作用还需自己的幂等/对账协议。

[Q053 · 工程现状与边界] 依据：backend/app/infrastructure/repositories/jobs.py; backend/app/infrastructure/repositories/outbox.py; backend/app/jobs/context.py; backend/tests/test_scheduler_reliability.py

15. 如何看待 Agent 与 Harness 的后续演进?

参考答案：Agent 会继续提高规划、工具使用和多模态能力，Harness 则会更重视状态持久化、工作空间、权限、评测、成本和跨模型适配。模型能力增强不会消除 Harness，反而要求运行环境提供更稳定的边界和验证。架构上应让模型与框架可替换。

[Q054 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

16. 目前的职级是什么?

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

[Q055 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

二、千问

一面

1. 介绍一下当前工作和比较擅长的方向。

参考答案：按时间顺序说明当前岗位、主要工作和转向 Agent 的阶段，再说明最熟悉的技术方向。重点写清实际负责的模块 、投入时间和已经交付的结果，不要把学习、验证和生产经验混在一起。

[Q056 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 对工作地点是否有要求，能否接受异地机会?

参考答案：按真实情况给明确边界：首选【城市】，可接受【城市/出差频率/迁移时间】。若有家庭或通勤约束，直接说明可协商范围和最早到岗时间。

[Q057 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

3. AI 应用平台与业务 Agent 项目是什么关系?

参考答案：轻记是独立的垂直业务应用，代码中没有依赖某个原有 AI 平台或企业 SDK。它复用了 LangGraph、FastAPI 和兼容模型 API，自己补健康数据模型与可靠性规则。如果以后挂到已有平台，应让平台复用身份、模型接入和运维能力，轻记保留事件完整性、营养来源、计划版本这些领域约束，不能虚构已有平台共建经历。

[Q058 · 工程现状] 依据：backend/requirements.txt; backend/app/main.py

4. 你在其中具体承担哪些工作?

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

[Q059 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

5. 为什么需要单独建设 Harness，它要解决什么问题?

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、 工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

[Q060 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

6. 企业 Agent 除了 ReAct、Tool Calling 和上下文管理，还需要补哪些能力?

参考答案：还需要身份与权限、工具治理、长任务状态、取消与恢复、上下文预算、长期记忆、沙箱、多租户、观测、评测 、成本控制和发布回滚。ReAct 只解决模型与工具怎样循环，不能独立保证生产可靠性。

[Q061 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

7. 通用运行内核、业务扩展与服务入口如何划分?

参考答案: 接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q062 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

8. 业务扩展模块具体包含什么?

参考答案: 当前业务模块包括 drafts 的缺字段检查和增量补丁、nutrition 的营养准备与来源、targets 的默认目标公式、advice/replies 的上下文与表达校验、services 的计划统计归档。运行控制由 Engine 管理，数据由 Repository 保存。它们是普通 Python 模块，并没有通过动态插件清单加载。

[Q063 · 工程现状] 依据：backend/app/chat/drafts.py

9. 这些模块如何注册、管理版本和升级?

参考答案: Tool 是可执行动作，契约是参数、结果与副作用；Skill 是任务方法及能力组合，包含适用条件、步骤和产物；Artifact 是持久化结果，需类型、版本、位置和访问权；Hook 在生命周期边界扩展观测、策略或记忆。清单应声明 ID、版本、入口、依赖、输入输出、权限、资源预算和兼容范围。描述用于发现，代码由受控执行器运行；版本需锁定到任务，升级不能静默改变运行中契约。

[Q064 · 八股文 · 通用知识] 依据：参考资料：S3

10. 运行内核如何与具体业务实现解耦?

参考答案: 当前通过普通函数和 Repository 分离业务计算与图路由：营养和目标计算不需要直接操作图 API。新增饮食规则应放入业务模块，新增通用请求或恢复语义才改 Engine。这个边界尚未成为正式接口协议；若继续通用化，应抽出服务端口、节点结果契约和状态版本迁移，而不是把所有规则堆进图节点。

[Q065 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

11. Memory 策略应该由业务决定，还是固定在运行内核中?

参考答案: 在轻记里，记什么由健康业务决定：过敏与明确偏好长期保留，主观偏咸只属于当前反馈，未知营养保留空值。Engine 负责把 State 存进 checkpoint，Repository 提供存取；不由运行框架自行提炼用户画像。以后复用到别的行业，应替换写入、检索和遗忘策略，保留存储与生命周期接口。

[Q066 · 工程现状] 依据：backend/app/chat/feedback.py

12. 能力声明文件需要描述哪些信息?

参考答案: Tool 是可执行动作，契约是参数、结果与副作用；Skill 是任务方法及能力组合，包含适用条件、步骤和产物；Artifact 是持久化结果，需类型、版本、位置和访问权；Hook 在生命周期边界扩展观测、策略或记忆。清单应声明 ID、版本、入口、依赖、输入输出、权限、资源预算和兼容范围。描述用于发现，代码由受控执行器运行；版本需锁定到任务，升级不能静默改变运行中契约。

[Q067 · 八股文 · 通用知识] 依据：参考资料：S3

13. Tool、Skill、Artifact 和 Hook 如何组织?

参考答案：Tool 是可执行动作，契约是参数、结果与副作用；Skill 是任务方法及能力组合，包含适用条件、步骤和产物；Artifact 是持久化结果，需类型、版本、位置和访问权；Hook 在生命周期边界扩展观测、策略或记忆。清单应声明 ID、版本、入口、依赖、输入输出、权限、资源预算和兼容范围。描述用于发现，代码由受控执行器运行；版本需锁定到任务，升级不能静默改变运行中契约。

[Q068 · 八股文 · 通用知识] 依据：参考资料：S3

14. Workflow 与 ReAct 如何配合?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q069 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

15. 业务扩展到什么程度应当修改内核，边界在哪里?

参考答案：新增食物规则、默认目标公式或统计模板应改轻记业务模块；只有新增通用恢复状态、事件协议、取消或统一执行契约时才考虑改 Engine。当前图仍直接导入领域函数，边界不是完全插件化。我的判断标准是新需求是否对所有业务都成立，若只服务一类健康规则，就不把它硬编码成运行时通用行为。

[Q070 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

16. 目前有多少类业务扩展，是否经过不同场景验证?

参考答案：工程覆盖一个健康领域中的五类记录：饮食、体重、睡眠、运动和生理周期，配套计划与分析三类工作流。它们证明了同领域不同事件的复用，没有证明跨金融、客服或工业场景的通用性。跨领域复用要用新的数据模型和真实任务验证，而不能把五种事件说成五个行业。

[Q071 · 工程现状] 依据：backend/app/api/schemas.py; README.md

17. 多人同时开发不同业务能力时，怎样协作和集成?

参考答案：按稳定接口和业务域拆分模块，使用独立版本、测试数据和运行命名空间。合并前执行接口兼容检查、自动测试 、代码评审和端到端回归。共享内核只接受通过扩展接口的能力，禁止业务模块直接修改全局状态。

[Q072 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

18. 不同业务独立部署，还是共享运行时?

参考答案：当前入口是 Taro/React/TypeScript 微信小程序，连接独立 FastAPI JSON API；旧 Web 前端已废弃，根路径不再托管页面。Engine 复用聊天、计划、分析图，health.db 保存业务事实，checkpoints.db 保存图进度。开发启动用 start.ps1；仓库另有 Docker 加固部署配置。后台任务与 Web 仍共享唯一 Engine，数据目录所有者锁阻止第二实例，不是独立 Worker 集群。正式对外服务仍需 HTTPS、鉴权及相应部署验收。

[Q073 · 工程现状与边界] 依据：miniprogram/package.json; backend/app/main.py; backend/app/infrastructure/data_owner.py; README.md; compose.hardened.yaml; start.ps1

19. 追问：共享时怎么做隔离?

参考答案：身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q074 · 八股文 · 通用知识] 依据：参考资料：S9,S10

20. Skill 如何路由和选择?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q075 · 八股文 · 通用知识] 依据：参考资料：S3,S9

21. 用户怎样发现和使用已有 Skill?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q076 · 八股文 · 通用知识] 依据：参考资料：S3,S9

22. Skill 是代码实现还是配置定义?

参考答案：Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q077 · 八股文 · 通用知识] 依据：参考资料：S3,S6

23. 是否提供沙箱，主要解决什么问题?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q078 · 八股文 · 通用知识] 依据：参考资料：S9,S10

24. 沙箱带来的延迟与安全收益怎么权衡?

参考答案：按信任和风险分级隔离：低风险只读调用可共享服务，高风险代码使用独立受限环境。Serverless 通常便于弹性与托管，代价是冷启动、时限及运行环境限制；按任务创建 Pod 更便于镜像、资源和网络定制，但启动与运维成本更高。可用预热池和镜像缓存减延迟，同时严格清除上一任务状态。评估需比较冷/热启动 P95、资源单价、并发、隔离边界和清理可靠性，不能仅看平均启动时间。

[Q079 · 八股文 · 通用知识] 依据：参考资料：S10

25. Serverless 与临时创建 Kubernetes Pod 的方式有什么差别?

参考答案：按信任和风险分级隔离：低风险只读调用可共享服务，高风险代码使用独立受限环境。Serverless 通常便于弹性与托管，代价是冷启动、时限及运行环境限制；按任务创建 Pod 更便于镜像、资源和网络定制，但启动与运维成本更高。可用预热池和镜像缓存减延迟，同时严格清除上一任务状态。评估需比较冷/热启动 P95、资源单价、并发、隔离边界和清理可靠性，不能仅看平均启动时间。

[Q080 · 八股文 · 通用知识] 依据：参考资料：S10

26. 目前的用户量和并发规模如何?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q081 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

27. 实时交互与后台长任务如何安排 Worker?

参考答案：实时任务使用低延迟队列和短时 Worker，优先保证首响应。后台长任务进入独立队列，由可释放和恢复的 Worker 执行。两类任务使用不同并发、超时和资源配额，避免长任务占满实时资源。状态统一外置，任务可跨 Worker 继续。

[Q082 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

28. 你在平台项目中负责哪些模块?

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

[Q083 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

29. 平台的实际使用规模怎样，是否已有线上用户?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q084 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

30. 应用如何发布和集成到外部系统?

参考答案: 应用先经过版本校验、权限检查、测试和审批，再发布为固定版本。外部系统可通过 API、SDK、网页组件或事件订阅接入。运行时要传递用户身份和租户信息，并支持灰度、回滚、限流和调用审计。

[Q085 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

31. 底层执行引擎是什么?

参考答案: 执行引擎基于 LangGraph，并非从零手写调度器。Engine 启动时构建 chat、plan、analysis 三张 StateGraph，再用同一个 SqliteSaver 按 thread_id 保存运行状态。FastAPI 接受请求，Engine 选择图，普通 Python 模块实现健康业务。自建部分是可靠性和领域规则，不是重新实现 LangGraph。

[Q086 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/main.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

32. 画布 DSL 如何转换为执行图?

参考答案: DSL 应声明版本、节点 ID/类型、参数、类型化输入输出、边、条件、变量引用、超时与错误处理。编译流程是解析 Schema、检查节点和端口、校验引用及类型、分析可达性与依赖、验证循环边界，再生成执行图。自然语言先生成受限中间计划，再编译为 DSL，避免模型直接拼接任意代码。执行前做静态检查与沙箱测试，运行时仍检查权限和写入条件；静态合法不等于业务正确。

[Q087 · 八股文 · 通用知识] 依据：参考资料：S6,S9

33. 是否比较过 AgentScope、pi-agent 等框架，各自有什么特点?

参考答案: 不要只列功能。用同一组任务比较状态模型、持久化恢复、多 Agent 协议、工具生态、可观测性、部署复杂度、扩展边界和社区成熟度，再做最小概念验证（PoC）。最终选择取决于团队语言栈、故障模型和交付形态，并保留适配层降低替换成本。

[Q088 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

34. ToB 与 ToC Agent 的主要区别是什么?

参考答案: ToB 更重租户隔离、审批审计、私有化、领域流程和 SLA。ToC 更重低延迟、个性化 Memory、海量弹性和单位成本。内核可复用，但身份、权限、记忆授权、资源调度和产品交互策略应分别设计，不能简单复制。

[Q089 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

35. 面向 ToC 时，Memory 应该怎样设计?

参考答案: 记忆分四层：工作记忆保存当前 State、草稿、等待点和恢复凭据；事件账本保存健康事件及修订；结构化长期事实保存偏好、过敏、专业限制等及其来源和有效期；派生层保存规律、复盘、计划与归档摘要。派生结果记录窗口、方法版本和来源版本，不能当作原始事实。工作记忆可到期清理，已保存事件按独立保留策略管理；当前不依赖向量记忆或自动对话摘要。

[Q090 · 工程现状与边界] 依据：backend/app/workflows/state.py; backend/app/health/facts.py; backend/app/health/derived.py

36. 工具越来越多，会带来哪些上下文和选择噪声，如何处理?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q091 · 八股文 · 通用知识] 依据：参考资料：S3,S9

37. Skill 规则与 Memory 冲突时怎么办?

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

[Q092 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

38. Python 和 Java 分别使用到什么程度?

参考答案：轻记后端实际使用 Python，主要是 FastAPI、Pydantic、LangGraph 和 SQLite；前端使用 TypeScript/React。当前工程没有 Java 后端代码，因此只能把 Java 单例、线程池等作为知识题说明，不能声称在本项目做过 Java 微服务。其他语言经验需结合本人的真实履历。

[Q093 · 工程现状] 依据：backend/app/; miniprogram/src/; miniprogram/package.json

39. 如果团队使用另一种语言，准备怎样适应?

参考答案：先承认语言切换需要学习成本，再说明会优先掌握团队的工程规范、构建工具、并发模型、依赖管理和测试方式 。通过小模块逐步进入主链路，并借助代码评审降低早期风险。

[Q094 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

二面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q095 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 最近在读哪些 Agent 框架?

参考答案：如实说明阅读过的项目、跟踪的调用链和完成的验证。重点应覆盖会话创建、上下文组装、工具执行、退出条件 、状态保存、恢复和流式事件，并说明自己确认了哪些设计限制。

[Q096 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

3. 是否看过 Coding Agent 的实现，重点研究了哪些部分?

参考答案：如实说明阅读过的项目、跟踪的调用链和完成的验证。重点应覆盖会话创建、上下文组装、工具执行、退出条件 、状态保存、恢复和流式事件，并说明自己确认了哪些设计限制。

[Q097 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

4. 怎样判断一个 Harness 好不好?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q098 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

5. 自己的运行时实现了什么，能力边界在哪里?

参考答案：运行时接收任务、身份、配置、上下文和资源限制，负责模型与工具循环、状态保存、恢复、预算和事件输出。 输出包括最终答案或产物、结构化状态、执行轨迹和用量指标。它不负责具体业务规则、业务数据定义和模型训练。

[Q099 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

6. 通用运行内核、业务扩展和服务入口怎样分层?

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q100 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

7. Tool、Skill 和 Memory 的接口分别如何设计?

参考答案：Tool 是可执行动作，契约是参数、结果与副作用；Skill 是任务方法及能力组合，包含适用条件、步骤和产物；Artifact 是持久化结果，需类型、版本、位置和访问权；Hook 在生命周期边界扩展观测、策略或记忆。清单应声明 ID、版本、入口、依赖、输入输出、权限、资源预算和兼容范围。描述用于发现，代码由受控执行器运行；版本需锁定到任务，升级不能静默改变运行中契约。

[Q101 · 八股文 · 通用知识] 依据：参考资料：S3

8. 运行时的输入和输出是什么?

参考答案：当前不是模型直接调用任意 Python 工具，而是先返回 ProcessingPlan 等固定 Schema，再由本地审核、取数、路由和执行。模型任务包括规划、安全分类、单条纠错、草稿补丁、营养估算和受限建议选择。每项用途有固定请求字段与输出类型，外发还要消费一次性授权；模型不能提交 SQL、任意字段或记录 ID 来取得数据。业务写入继续经过证据、完整性、幂等和确认检查。

[Q102 · 工程现状与边界] 依据：backend/app/modeling/payloads.py; backend/app/modeling/boundary.py; backend/app/contracts/workflows.py; backend/app/workflows/planning.py

9. 多个业务并行扩展时，怎样避免互相影响?

参考答案：身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q103 · 八股文 · 通用知识] 依据：参考资料：S9,S10

10. 如何评估自研系统与开源方案的差异?

参考答案：不要只列功能。用同一组任务比较状态模型、持久化恢复、多 Agent 协议、工具生态、可观测性、部署复杂度、扩展边界和社区成熟度，再做最小概念验证（PoC）。最终选择取决于团队语言栈、故障模型和交付形态，并保留适配层降低替换成本。

[Q104 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

11. 追问：是否真正用相同任务对比过通用 Agent?

参考答案：应使用相同模型、工具、上下文、任务和预算做对照，只改变 Harness 或运行策略。比较任务完成率、轨迹合规 、延迟、成本和人工干预次数。若没有做过严格对照，应直接说明目前只有局部测试，并列出仍需验证的假设。

[Q105 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

12. 测试用例、Ground Truth 和 A/B 对比具体怎么做?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q106 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

13. 当前使用哪些模型?

参考答案：模型通过兼容 Chat Completions 的传输层接入，供应商地址、模型名和凭据由运行配置显式提供，不能把示例模型名当作运行时事实。源码不再硬编码密钥；支持仓库外密钥文件，开发模式也可用环境变量。配置全空进入演示模式，部分缺失会失败；模型失败不会悄悄切换演示营养。当前没有多供应商路由网关，换模型需回归结构化输出、错误映射和数据边界。

[Q107 · 工程现状与边界] 依据：backend/app/common/config.py; backend/app/modeling/parser.py; backend/app/modeling/transport.py; backend/tests/test_config.py

14. 是否做过模型后训练?

参考答案：如实区分应用调用、推理部署和训练经验。多数企业 Agent 先用 Prompt、工具、RAG 与评测解决问题。只有错误模式稳定、数据量足且收益可验证时才考虑 SFT/DPO，并建立数据授权、离线评测和灰度回滚。

[Q108 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

15. 框架给开发者提供什么接口，开发者需要写哪些代码?

参考答案：在轻记里新增能力需要写 Pydantic 事件或请求 Schema、领域服务函数、图节点及路由，再补对应 API 和前端类型。开发者不是只上传提示词即可发布应用。已有清晰的接点包括 Parser 的结构化方法、Repository 的数据接口以及 Engine 的三张 build_* 图，但没有公开插件 SDK。

[Q109 · 工程现状] 依据：backend/app/api/schemas.py; miniprogram/src/types.ts

16. 与 AgentScope 相比，有什么不同?

参考答案：不要只列功能。用同一组任务比较状态模型、持久化恢复、多 Agent 协议、工具生态、可观测性、部署复杂度、扩展边界和社区成熟度，再做最小概念验证（PoC）。最终选择取决于团队语言栈、故障模型和交付形态，并保留适配层降低替换成本。

[Q110 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

17. 是否了解托管式 Agent 服务?

参考答案：托管服务是可行的，前提是能提供稳定运行、弹性资源、模型与工具接入、状态恢复、权限、观测和成本治理。 客户选择它的原因通常是减少基础设施建设和运维，但数据边界、私有化和供应商锁定必须明确。

[Q111 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

18. 目前有多少真实用户，实际使用情况如何?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q112 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

19. 开发中遇到的最大挑战是什么?

参考答案：可讲一个有测试证据的困难场景：用户说“昨天吃得有点咸”，其中没有食物名称。如果直接要求严格餐食 Schema，会校验失败；若猜一个食物又会污染数据。现在先保留宽松 ExtractionEvent，由本地 check_drafts 决定是否追问；补充只更新待补字段，保留首次日期。测试覆盖重启和跨午夜补充，不能据此虚构某次线上事故。

[Q113 · 工程现状] 依据：backend/app/chat/drafts.py; backend/tests/test_draft_clarification.py

20. 较重的 Workflow 如何与 ReAct 主循环结合?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q114 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

21. Sub-agent 如何定义和启动?

参考答案：Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果，Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema；结果返回状态、证据及产物引用。Worker 可预声明，也可受控动态创建，动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并，副作用用幂等键；检测循环委派并传播取消，避免所有 Worker 同时修改同一业务对象。

[Q115 · 八股文 · 通用知识] 依据：参考资料：S1,S6

22. 追问：子 Agent 是提前声明的，还是运行过程中动态创建的?

参考答案：两种方式都可。预声明子 Agent 固定职责、工具、模型和输入输出，便于评测与权限审核；动态创建则根据当前任务生成受限配置，灵活但更难控制数量、成本和行为。生产中可采用预声明能力池加动态任务参数，先避免任意生成执行角色。动态方案也必须从已批准能力中选工具，限制深度、并发和总预算，记录父子关系并传播取消，禁止自行扩权。

[Q116 · 八股文 · 通用知识] 依据：参考资料：S1,S6

23. 为什么考虑离开当前公司?

参考答案：用正向原因回答：当前工作已经形成了哪些积累，下一阶段希望扩大哪类责任或技术深度，以及目标岗位为什么更匹配。不要评价原公司的人和管理，也不要只把薪资作为原因。

[Q117 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

24. 为什么选择这个方向和这家公司?

参考答案：用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

[Q118 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

25. 如何看待当前公司的优势与不足?

参考答案：优势应从业务、团队或工程能力中选择自己真正了解的部分。不足要描述为客观限制，并说明它对工作产生的影响以及自己采取过的改进。避免情绪化评价或涉及保密信息。

[Q119 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

26. 后续职业规划是什么?

参考答案: 用正向动机回答:现岗位让我积累了【能力】,下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司,也不要只说薪资。补充未来 2-3 年希望承担的结果。

[Q120 · 个人经历答题模板] 依据:按本人真实经历补充;不代填职级、规模、贡献或效果数字

三面

1. 请做一下自我介绍。

参考答案: 回答顺序:先说明【X 年】后端或 AI 应用研发经验,再说明近【X 年】负责的 Agent 工作、核心职责和量化结果,最后说明这些能力与岗位要求的关系。全程控制在两分钟内,只保留与岗位相关的信息。

[Q121 · 个人经历答题模板] 依据:按本人真实经历补充;不代填职级、规模、贡献或效果数字

2. 入职当前公司时是什么级别,现在是什么级别?

参考答案: 如实说明入职级别、当前级别和对应时间。重点补充职责范围怎样变化、是否承担关键设计或团队协作责任,以及变化由哪些可核验结果支持。不同公司的职级不能直接横向等同。

[Q122 · 个人经历答题模板] 依据:按本人真实经历补充;不代填职级、规模、贡献或效果数字

3. 是否有晋升经历?如有,最近一次是什么时候?

参考答案: 如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明,绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较,可补充带队规模、决策权和交付复杂度。

[Q123 · 个人经历答题模板] 依据:按本人真实经历补充;不代填职级、规模、贡献或效果数字

4. 追问:如果有晋升经历,主要凭借哪些工作和贡献?

参考答案: 如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明,绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较,可补充带队规模、决策权和交付复杂度。

[Q124 · 个人经历答题模板] 依据:按本人真实经历补充;不代填职级、规模、贡献或效果数字

5. 如果有晋升和转向 AI 研发的经历,两者的时间先后关系是什么?

参考答案: 如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明,绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较,可补充带队规模、决策权和交付复杂度。

[Q125 · 个人经历答题模板] 依据:按本人真实经历补充;不代填职级、规模、贡献或效果数字

6. 介绍一下当前业务 Agent 项目。

参考答案：我选轻记，因为它能完整讲清一条 Agent 业务闭环：自然语言输入拆成健康草稿，缺字段时追问，营养准备后写库，支持历史纠错，再用完整记录做计划和复盘。最有代表性的设计是 checkpoint 恢复配合幂等写入、未知营养不算零，以及计划确认时核对数据版本。它仍是单用户本地原型。

[Q126 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py; README.md

7. FDE 在实际工作中意味着什么?

参考答案：FDE 通常指深入客户场景、同时承担需求澄清、方案设计、集成实施和结果验证的工程角色。重点不是临时定制 ，而是把现场问题转化为可复用的产品能力，并对最终业务结果负责。

[Q127 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

8. 项目团队有哪些角色，怎样分工?

参考答案：按真实团队说明产品、业务、算法、平台、后端、前端、测试、交付和运维的职责。重点说明需求、架构、代码 、评测、发布和线上问题分别由谁负责，以及自己处于哪个协作位置。

[Q128 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

9. 是否所有研发人员都参与运行内核开发?

参考答案：通常不需要。少量平台人员维护运行内核和稳定接口，业务研发通过 Skill、Workflow、工具和数据适配扩展能力。这样能减少内核变更和回归风险，同时让业务团队独立交付。

[Q129 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

10. 业务 Agent 是否建立在原有应用平台之上?

参考答案：轻记是独立的垂直业务应用，代码中没有依赖某个原有 AI 平台或企业 SDK。它复用了 LangGraph、FastAPI 和兼容模型 API，自己补健康数据模型与可靠性规则。如果以后挂到已有平台，应让平台复用身份、模型接入和运维能力，轻记保留事件完整性、营养来源、计划版本这些领域约束，不能虚构已有平台共建经历。

[Q130 · 工程现状] 依据：backend/requirements.txt; backend/app/main.py

11. 你具体负责哪部分?

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

[Q131 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

12. 架构方案由谁决定，你参与了哪些设计决策?

参考答案：先说明最终决策人和评审机制，再列出自己负责评估和推动的设计项。回答应包含候选方案、主要约束、取舍依据和最终责任边界，避免把团队决策全部说成个人决定。

[Q132 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

13. 为什么采用运行内核、业务扩展和服务入口的分层方式?

参考答案: 接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q133 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

14. 运行内核内部包含哪些模块?

参考答案: 当前业务模块包括 drafts 的缺字段检查和增量补丁、nutrition 的营养准备与来源、targets 的默认目标公式、advice/replies 的上下文与表达校验、services 的计划统计归档。运行控制由 Engine 管理，数据由 Repository 保存。它们是普通 Python 模块，并没有通过动态插件清单加载。

[Q134 · 工程现状] 依据：backend/app/chat/drafts.py

15. Workflow 和 ReAct 如何结合?

参考答案: 规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q135 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

16. 除此之外，还有哪些比较有特点的设计?

参考答案: LangGraph 之外，工程已经实现请求幂等、恢复回执、模型外发授权与预算、结构化事实冲突、数据血缘删除、受控输出、安全路由、可靠后台任务和分层观测。下一步优先补真实营养来源、真实模型评测及目标部署验收，再按需要做鉴权、多用户和进程拆分。没有文档 RAG、MCP、动态 Skill、任意代码执行或多 Agent 集群，不把这些当作既有成果。

[Q136 · 工程现状与边界] 依据：backend/app/workflows/runtime.py; backend/app/modeling/boundary.py; backend/app/privacy/deletion.py; backend/app/jobs/runner.py; backend/app/observability/service.py

17. 为什么把 Memory 实现放在运行内核之外?

参考答案: 在轻记里，记什么由健康业务决定：过敏与明确偏好长期保留，主观偏咸只属于当前反馈，未知营养保留空值。Engine 负责把 State 存进 checkpoint，Repository 提供存取；不由运行框架自行提炼用户画像。以后复用到别的行业，应替换写入、检索和遗忘策略，保留存储与生命周期接口。

[Q137 · 工程现状] 依据：backend/app/chat/feedback.py

18. 记忆接口如何定义?

参考答案: 记忆写入由业务边界控制：完整事件经校验、营养准备和事务入账；明确长期信息走结构化事实服务，带证据、有效期和修订。助手建议、模型推理或一次偏咸反馈不会自动升级为长期事实。读取使用与任务目的匹配的投影，纠错后让相关派生结果过期。管理接口支持新增、修改、撤回、冲突选择、历史与隐私删除；它仍是领域服务，不是动态 Memory 插件协议。

[Q138 · 工程现状与边界] 依据：backend/app/health/facts.py; backend/app/api/profile.py; backend/app/infrastructure/repositories/facts.py; backend/app/modeling/projections.py

19. 追问：具体实现由谁提供，又怎样注入运行时?

参考答案：记忆写入由业务边界控制：完整事件经校验、营养准备和事务入账；明确长期信息走结构化事实服务，带证据、有效期和修订。助手建议、模型推理或一次偏咸反馈不会自动升级为长期事实。读取使用与任务目的匹配的投影，纠错后让相关派生结果过期。管理接口支持新增、修改、撤回、冲突选择、历史与隐私删除；它仍是领域服务，不是动态 Memory 插件协议。

[Q139 · 工程现状与边界] 依据：backend/app/health/facts.py; backend/app/api/profile.py; backend/app/infrastructure/repositories/facts.py; backend/app/modeling/projections.py

20. 上下文工程做了哪些工作?

参考答案：上下文按任务目的构造。首轮规划只收到本次文字、时间和是否选中记录，不再默认携带最近 12 条历史。本地审核计划后，从固定视图取必要数据；纠错绑定单条记录，补充只发送当前草稿和待补字段，营养估算只发送本次食物与分量。建议使用白名单结构化投影，不发送历史 raw_text。目的、资源和实际 payload 都经授权核对，最终返回用途与数据类别说明。

[Q140 · 工程现状与边界] 依据：backend/app/modeling/payloads.py; backend/app/modeling/projections.py; backend/app/modeling/boundary.py; backend/tests/test_model.py

21. Token 预算如何分配，什么时候触发压缩?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q141 · 八股文 · 通用知识] 依据：参考资料：S1,S6

22. 工具结果和历史消息分别怎么压缩?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q142 · 八股文 · 通用知识] 依据：参考资料：S1,S6

23. 这些设计中，哪些已经做过实际验证?

参考答案：可核验测试已覆盖追问和提交恢复、运行限额、模型传输、证据与输出约束、结构化事实、血缘删除、调度租约和观测隐私等。当前还有微信小程序协议、恢复和交互测试，旧版的 124 个后端与 10 个 Web 测试数字已不适用。本次只做面试资料更新及文档校验，完整测试数和结果应引用对应日期的运行记录；合成或 mock 用例通过不代表真实模型、生产容量或医学效果。

[Q143 · 工程现状与边界] 依据：backend/tests; docs/evidence; README.md

24. 平时怎样追踪 Agent 领域的变化?

参考答案: 固定关注官方文档、版本说明、核心仓库、技术论文和可靠的工程实践。对重要变化先阅读接口与测试，再用小型任务验证行为、性能和兼容性，最后判断是否影响现有系统。只说明真实阅读和验证过的内容。

[Q144 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

25. 最近观察到哪些值得关注的架构趋势?

参考答案: 可从长任务持久化、统一工具协议、渐进式能力加载、工作空间与 Artifact、上下文压缩、计算机操作和多模型路由说明。回答重点是这些变化会影响现有架构的哪一层，以及是否经过实际验证。

[Q145 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

26. 下一阶段的职业发展考虑是什么?

参考答案: 用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

[Q146 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

27. 工作地点上有哪些考虑?

参考答案: 按真实情况给明确边界：首选【城市】，可接受【城市/出差频率/迁移时间】。若有家庭或通勤约束，直接说明可协商范围和最早到岗时间。

[Q147 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

三、淘天

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q148 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 单 Agent、多 Agent 和 Workflow 分别适合什么场景?

参考答案：单 Agent 用一个决策上下文调用工具，适合目标统一、工具可控的开放任务；Workflow 用预设节点和条件边控制流程，适合步骤稳定、合规或一致性要求高的业务；多 Agent 适合独立子目标可并行、上下文或权限需隔离的场景。选择时比较完成率、延迟、成本、可调试性和协调开销。任务复杂并不必然需要多个 Agent；常见组合是外层 Workflow 控制阶段，局部 Agent 自主探索。

[Q149 · 八股文 · 通用知识] 依据：参考资料：S1,S6

3. 它们的核心区别是什么?

参考答案：单 Agent 用一个决策上下文调用工具，适合目标统一、工具可控的开放任务；Workflow 用预设节点和条件边控制流程，适合步骤稳定、合规或一致性要求高的业务；多 Agent 适合独立子目标可并行、上下文或权限需隔离的场景。选择时比较完成率、延迟、成本、可调试性和协调开销。任务复杂并不必然需要多个 Agent；常见组合是外层 Workflow 控制阶段，局部 Agent 自主探索。

[Q150 · 八股文 · 通用知识] 依据：参考资料：S1,S6

4. 任务复杂，就一定应该用多 Agent 吗?

参考答案：不一定。复杂性可能来自业务规则、状态恢复或一致性，这些往往更适合一个 Agent 加工具或受控 Workflow。多 Agent 只有在子目标可独立验证、上下文或权限需要隔离、并行收益明显时才值得引入。先用单 Agent 建基线，再比较拆分后的完成率、成本、延迟、通信次数和结果合并错误；角色数量增加本身不是架构改进。

[Q151 · 八股文 · 通用知识] 依据：参考资料：S1

5. 追问：什么条件下才值得引入多 Agent?

参考答案：至少需要清晰的职责边界、独立输入输出、可验证结果，以及可量化的并行或隔离收益。例如多个独立资料源调查可并行，最后由主流程核对证据；若角色都要反复共享全文并修改同一状态，协调成本可能更高。还需具备任务契约、预算、取消、失败恢复和合并策略。判断依据是同任务基准优于较简单方案，而不是任务描述看起来复杂。

[Q152 · 八股文 · 通用知识] 依据：参考资料：S1

6. 为什么将 Workflow 封装为 Skill?

参考答案：Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q153 · 八股文 · 通用知识] 依据：参考资料：S3,S6

7. 一次用户请求从进入到结束，生命周期是什么样的?

参考答案：请求进入 POST /api/chat 后先校验参数和 request_id 指纹，创建或恢复 thread_id。图依次解析、检查草稿、准备营养，完整独立事件可先保存；缺字段则 interrupt。恢复后仅合并待补字段，最终保存、组装卡片与证据，将状态、耗时和消息落库。模型失败、取消和等待用户分别有不同结果，HTTP 返回并不必然代表新事件已保存。

[Q154 · 工程现状] 依据：backend/app/main.py

8. 运行内核、业务扩展和服务入口分别负责什么?

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q155 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

9. 一次完整任务与其中一轮模型推理、工具调用是什么关系?

参考答案：一条轻记任务可能包含多轮用户补充和多个模型请求。首次解析、缺失字段 patch、营养粗估、建议表达是不同调用，整个工作流由同一 thread_id 关联；事件写入又有独立操作键。普通记录不一定调用表达模型，查看历史也不会重新营养估算，因此不能把一次模型成功当作整个任务完成。

[Q156 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

10. Skill 如何被选中，并进入上下文?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q157 · 八股文 · 通用知识] 依据：参考资料：S3,S9

11. 工具调用什么时候并行，什么时候串行?

参考答案：无依赖、无共享写入冲突的工具调用可并行，例如读取两个独立城市天气；后一步依赖前一步结果、同一资源需顺序修改或调用间有配额限制时应串行。运行时用 DAG 表示依赖，对就绪节点限并发，并设置单次/总超时。并行结果按稳定调用 ID 关联，明确部分失败是取消、继续还是降级；共享状态通过显式合并和版本检查处理。并发可降低总耗时，但会提高峰值资源与失败处理复杂度。

[Q158 · 八股文 · 通用知识] 依据：参考资料：S4,S8

12. 为什么选择 LangGraph?

参考答案：轻记需要两类等待：记录缺字段时等用户补充，计划变化较大时等用户确认。因此选择 LangGraph 的显式状态图、interrupt 和 SQLite checkpoint。图负责编排，草稿完整性、营养来源、请求去重和计划版本检查由自己的业务代码实现。当前没有统一任务基准证明它优于其他框架；选择主要依据是与这两类暂停恢复需求匹配。

[Q159 · 工程现状] 依据：backend/app/workflows/engine.py; backend/tests/test_agent.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

13. Agent 效果从哪些维度评估?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q160 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

14. 测试用例和参考执行轨迹怎么构建?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q161 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

15. LLM-as-a-Judge 具体评价哪些内容?

参考答案：评价任务是否完成、答案是否正确完整、是否忠于证据、格式是否符合要求，以及轨迹是否满足规则。评分标准要拆成独立维度，并要求给出依据。延迟、成本和安全事件应由系统指标评价，不交给 Judge 猜测。

[Q162 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

16. 怎样把评测接入发布和回归流程?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q163 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

17. 如果上线后效果下降，怎么定位和回滚?

参考答案：按数据→预处理→Embedding/Reranker 版本→索引构建/增量同步→查询改写→过滤条件→服务缓存的顺序排查 。先复现同一批查询，核对文档数、向量维度、索引时间和配置哈希，再做组件旁路对照，快速定位从哪一阶段开始偏离。

[Q164 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

18. 如何防范 Prompt Injection?

参考答案：把用户文本和模型输出视为待验证数据。系统不授予任意 SQL 或命令执行权，模型计划只能触发本地批准的固定视图与业务动作，外发按用途、资源、Schema 和一次性授权检查。输出策略只接受已批准的 point_id/variant_id，数字、日期、保存状态和追问由本地生成；未知标识或自由正文失败回退本地表达。输入安全路由与写入校验也在恢复路径执行。提示词只是辅助，执行端约束才是边界。

[Q165 · 工程现状与边界] 依据：backend/app/modeling/boundary.py; backend/app/modeling/payloads.py; backend/app/policy/output.py; backend/app/workflows/safety.py; backend/tests/test_output_policy.py

19. 沙箱承担哪些安全职责?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q166 · 八股文 · 通用知识] 依据：参考资料：S9,S10

20. Agent 需要访问敏感数据时，怎样控制权限?

参考答案：把用户文本和模型输出视为待验证数据。系统不授予任意 SQL 或命令执行权，模型计划只能触发本地批准的固定视图与业务动作，外发按用途、资源、Schema 和一次性授权检查。输出策略只接受已批准的 point_id/variant_id，数字、日期、保存状态和追问由本地生成；未知标识或自由正文失败回退本地表达。输入安全路由与写入校验也在恢复路径执行。提示词只是辅助，执行端约束才是边界。

[Q167 · 工程现状与边界] 依据：backend/app/modeling/boundary.py; backend/app/modeling/payloads.py; backend/app/policy/output.py; backend/app/workflows/safety.py; backend/tests/test_output_policy.py

21. 上下文接近模型窗口上限时怎么办?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q168 · 八股文 · 通用知识] 依据：参考资料：S1,S6

22. 平时如何保持上下文干净，避免无关信息不断积累?

参考答案：上下文按任务目的构造。首轮规划只收到本次文字、时间和是否选中记录，不再默认携带最近 12 条历史。本地审核计划后，从固定视图取必要数据；纠错绑定单条记录，补充只发送当前草稿和待补字段，营养估算只发送本次食物与分量。建议使用白名单结构化投影，不发送历史 raw_text。目的、资源和实际 payload 都经授权核对，最终返回用途与数据类别说明。

[Q169 · 工程现状与边界] 依据：backend/app/modeling/payloads.py; backend/app/modeling/projections.py; backend/app/modeling/boundary.py; backend/tests/test_model.py

23. 工具返回过长，或连续调用工具产生大量记录时，怎么处理?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q170 · 八股文 · 通用知识] 依据：参考资料：S1,S6

24. 追问：压缩时哪些信息必须保留?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q171 · 八股文 · 通用知识] 依据：参考资料：S1,S6

25. 业务场景中的长期记忆如何组织?

参考答案：记忆分四层：工作记忆保存当前 State、草稿、等待点和恢复凭据；事件账本保存健康事件及修订；结构化长期事实保存偏好、过敏、专业限制等及其来源和有效期；派生层保存规律、复盘、计划与归档摘要。派生结果记录窗口、方法版本和来源版本，不能当作原始事实。工作记忆可到期清理，已保存事件按独立保留策略管理；当前不依赖向量记忆或自动对话摘要。

[Q172 · 工程现状与边界] 依据：backend/app/workflows/state.py; backend/app/health/facts.py; backend/app/health/derived.py

26. 新旧记忆冲突时怎么处理?

参考答案：事件纠错保留事件 ID、递增版本并保存修订；长期事实另有结构化主题、值、极性、来源、有效期和变更记录。明确改口的普通偏好可替代旧值，临时偏好不静默覆盖长期信息；冲突可让用户选择。过敏和专业限制到期进入待核对，继续保留保护，不因低使用频率遗忘。用户确认只是明确陈述，转述专业建议不等于独立核验；旧自由文本不自动猜成可靠结构。

[Q173 · 工程现状与边界] 依据：backend/app/health/fact_resolution.py; backend/app/health/facts.py; backend/app/contracts/facts.py; backend/tests/test_structured_facts.py

27. 【工程编码题】 实现一个简化的 Plan-and-Execute Agent：查询两个城市的天气，并计算温度差。

参考答案：Planner 将目标拆成有 ID、依赖、能力名、参数和输出 Schema 的步骤；Executor 按依赖拓扑执行就绪节点，对无数据或副作用冲突的步骤限并发；Summarizer 基于已验证结果汇总，不补造未完成结果。后续参数通过结构化引用绑定前序输出，禁止执行任意表达式。两城市天气可并行查询，温差步骤依赖两者，先统一单位再计算。计划有版本，失败按错误类型重试或重规划，并保存完成步骤。附录 D 给出参考实现。

[Q174 · 八股文 · 通用知识] 依据：参考资料：S1,S6

28. Planner、Executor 和 Summarizer 各负责什么?

参考答案：Planner 将目标拆成有 ID、依赖、能力名、参数和输出 Schema 的步骤；Executor 按依赖拓扑执行就绪节点，对无数据或副作用冲突的步骤限并发；Summarizer 基于已验证结果汇总，不补造未完成结果。后续参数通过结构化引用绑定前序输出，禁止执行任意表达式。两城市天气可并行查询，温差步骤依赖两者，先统一单位再计算。计划有版本，失败按错误类型重试或重规划，并保存完成步骤。附录 D 给出参考实现。

[Q175 · 八股文 · 通用知识] 依据：参考资料：S1,S6

29. 步骤怎样表示，后续步骤如何引用前序结果?

参考答案：Planner 将目标拆成有 ID、依赖、能力名、参数和输出 Schema 的步骤；Executor 按依赖拓扑执行就绪节点，对无数据或副作用冲突的步骤限并发；Summarizer 基于已验证结果汇总，不补造未完成结果。后续参数通过结构化引用绑定前序输出，禁止执行任意表达式。两城市天气可并行查询，温差步骤依赖两者，先统一单位再计算。计划有版本，失败按错误类型重试或重规划，并保存完成步骤。附录 D 给出参考实现。

[Q176 · 八股文 · 通用知识] 依据：参考资料：S1,S6

30. 怎样区分调用工具的步骤和调用模型的步骤?

参考答案：能用明确接口和确定规则完成的动作应定义为工具步骤，输入输出必须结构化。需要理解自然语言、生成内容、 判断意图或制定计划的步骤才交给模型。关键业务计算、权限判断和数据写入不应依赖模型自由生成。

[Q177 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

31. 工具注册表如何设计?

参考答案: 工具注册表维护稳定 tool_id、版本、用途说明、输入/输出 JSON Schema、执行适配器、权限、超时、重试和幂等策略。模型只看经身份与场景过滤后的描述，返回工具名、参数和调用 ID；服务端查注册表，先校验参数、资源归属与风险，再调用真实实现，返回关联原调用 ID 的结构化结果。未知工具、额外字段和越权参数必须拒绝；模型可见性筛选不能替代执行端鉴权。

[Q178 · 八股文 · 通用知识] 依据：参考资料：S3,S9

32. 目前其他求职流程进展如何?

参考答案: 如实说明当前流程所处阶段和预计决策时间，同时表达对本岗位的兴趣。不要虚构 Offer，也不要用无法核实的信息施压。

[Q179 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

33. 对工作地点有什么考虑?

参考答案: 按真实情况给明确边界：首选【城市】，可接受【城市/出差频率/迁移时间】。若有家庭或通勤约束，直接说明可协商范围和最早到岗时间。

[Q180 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

34. 为什么考虑换工作?

参考答案: 用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

[Q181 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

二面

1. 请做一下自我介绍。

参考答案: 回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q182 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. AI 应用平台最初要解决什么问题，为什么建设这个平台?

参考答案: 轻记面向希望用自然语言记录饮食和身体状态的个人用户。核心问题是表单记录成本高、补记和纠错容易出错、后续建议缺少可追溯依据。当前交付是单用户微信小程序原型，具备五类记录、关键追问、历史查询、目标确认、复盘与站内提醒；没有企业租户、客户应用发布市场或通用低代码平台。

[Q183 · 工程现状] 依据：backend/app/main.py; miniprogram/src/app.tsx; README.md

3. 与 Dify、Coze、Palantir 等方案相比，借鉴了什么，又有什么不同?

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

[Q184 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

4. 是否阅读过相关开源项目的源码?

参考答案：固定关注官方文档、版本说明、核心仓库、技术论文和可靠的工程实践。对重要变化先阅读接口与测试，再用小型任务验证行为、性能和兼容性，最后判断是否影响现有系统。只说明真实阅读和验证过的内容。

[Q185 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

5. Workflow 节点非常多时，怎样处理复杂度?

参考答案：先按业务阶段拆成子 Workflow，并明确每个子流程的输入输出和错误边界。公共步骤抽成可复用节点，复杂分支使用子图和命名空间。发布前做静态检查、可达性分析和端到端测试，避免一个巨型流程难以维护。

[Q186 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

6. 是否支持通过自然语言生成 Workflow?

参考答案：DSL 应声明版本、节点 ID/类型、参数、类型化输入输出、边、条件、变量引用、超时与错误处理。编译流程是解析 Schema、检查节点和端口、校验引用及类型、分析可达性与依赖、验证循环边界，再生成执行图。自然语言先生成受限中间计划，再编译为 DSL，避免模型直接拼接任意代码。执行前做静态检查与沙箱测试，运行时仍检查权限和写入条件；静态合法不等于业务正确。

[Q187 · 八股文 · 通用知识] 依据：参考资料：S6,S9

7. 为什么不让模型直接生成最终 DSL?

参考答案：DSL 应声明版本、节点 ID/类型、参数、类型化输入输出、边、条件、变量引用、超时与错误处理。编译流程是解析 Schema、检查节点和端口、校验引用及类型、分析可达性与依赖、验证循环边界，再生成执行图。自然语言先生成受限中间计划，再编译为 DSL，避免模型直接拼接任意代码。执行前做静态检查与沙箱测试，运行时仍检查权限和写入条件；静态合法不等于业务正确。

[Q188 · 八股文 · 通用知识] 依据：参考资料：S6,S9

8. 生成后怎样校验节点、参数和连线?

参考答案：DSL 应声明版本、节点 ID/类型、参数、类型化输入输出、边、条件、变量引用、超时与错误处理。编译流程是解析 Schema、检查节点和端口、校验引用及类型、分析可达性与依赖、验证循环边界，再生成执行图。自然语言先生成受限中间计划，再编译为 DSL，避免模型直接拼接任意代码。执行前做静态检查与沙箱测试，运行时仍检查权限和写入条件；静态合法不等于业务正确。

[Q189 · 八股文 · 通用知识] 依据：参考资料：S6,S9

9. 能否自动执行测试，并根据错误修复生成结果?

参考答案：可以，但修复必须受限。系统先在沙箱运行固定测试，根据结构化错误只修改相关节点或参数，并限制修复次数 。每次修改都重新执行完整校验和测试。超过次数或涉及权限与业务语义时转人工处理。

[Q190 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

10. 工具数量很多时，如何组织和选择?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q191 · 八股文 · 通用知识] 依据：参考资料：S3,S9

11. 如果是面向 ToC 的 Agent，工具信息如何进入上下文?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q192 · 八股文 · 通用知识] 依据：参考资料：S3,S9

12. 如何通过作用域和渐进式披露限制当前可见能力?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q193 · 八股文 · 通用知识] 依据：参考资料：S3,S9

13. 工具描述和历史消息过长时如何压缩?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q194 · 八股文 · 通用知识] 依据：参考资料：S1,S6

14. 完整的 RAG 链路怎么设计?

参考答案：RAG 分离线入库和在线问答。入库先解析与清洗文档，保留结构、页码、来源和权限，再切分、生成向量并写入检索索引；文档更新与删除需同步处理版本。在线先理解问题并按需改写，执行权限过滤、关键词/向量召回、融合、重排与去重，再按 Token 预算组装证据，生成带引用的答案。证据不足时澄清或拒答。评测要把解析、召回、排序、引用和答案分别统计，便于定位故障。

[Q195 · 八股文 · 通用知识] 依据：参考资料：S2,S11

15. 不同文件类型分别如何处理?

参考答案：解析先保留文档结构与来源，再做统一索引。PDF/扫描件需要版面分析、OCR 和阅读顺序；Word 按标题、段落及表格；PPT 保留页、标题和图文关系；Excel 保留工作表、表头、单位及行列坐标；图片可用 OCR 和视觉描述，必要时保留可回看的图像引用。表格与图片描述应绑定原页，避免单独文本失去条件。解析质量会限制召回上限，应检查缺页、乱码、表格错列及来源定位。

[Q196 · 八股文 · 通用知识] 依据：参考资料：S2

16. PPL 切分是怎么做的?

参考答案：PPL 是语言模型对文本预测难度的度量，通常为平均负对数似然的指数。基于 PPL 的切分利用相邻句或局部窗口的预测难度变化，寻找可能的主题边界；不同实现的上下文、打分与边界规则并不统一。可对候选句边界计算条件负对数似然，平滑后取局部峰值，同时施加最小/最大块长及结构规则。分数高也可能来自术语、代码或 OCR 错误，不能直接等同语义断点。需与结构切分对照评测。

[Q197 · 八股文 · 通用知识] 依据：参考资料：S2

17. BM25、向量检索、RRF、MMR 和 Rerank 如何组合?

参考答案：BM25 擅长实体、术语和精确词匹配，稠密向量检索擅长语义近邻，二者互补。分别召回后按文档/片段 ID 去重，用 RRF 等方法融合；RRF 的分数是各路 1/(k+rank) 之和，避免直接混加不同量纲的原始分数。随后用 Cross-Encoder 等重排器精排少量候选。结果高度重复时可用 MMR 在相关性与多样性间取舍。每增加一层都要验证 Recall、答案质量、延迟和成本，不能默认链路越长越好。

[Q198 · 八股文 · 通用知识] 依据：参考资料：S2,S11

18. 如何评估检索和回答效果?

参考答案：先建立带相关证据标注的固定问题集，并按实体、同义表达、多跳、无答案和权限场景分层。Recall@K 衡量相关证据覆盖；MRR 是首个相关结果倒数排名的均值；NDCG 结合相关性等级与位置衡量排序。回答另测忠实性、完整性、引用正确率和拒答。对照实验固定语料版本、模型和预算，一次改变主要因素，报告样本、差值和不确定性；最终质量提升不应只由单个检索分数推断。

[Q199 · 八股文 · 通用知识] 依据：参考资料：S2

19. 如果生成答案和评价答案都依赖同类模型，怎样看待评测偏差?

参考答案：同类模型可能偏好相似表达和推理方式，也可能共同忽略同一类错误。应使用明确评分标准、不同模型交叉评审 、人工盲测和一致性统计，并保留无法判定的样本。Judge 只能辅助，不能替代人工校准。

[Q200 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

20. 模型生成的评测集可信吗，怎样验证?

参考答案：只能作为候选数据。生成后要检查任务是否真实、答案是否正确、难度是否覆盖、是否泄漏提示词，并由领域人员抽检或复核。最终评测集还要包含真实用户任务和线上失败记录。

[Q201 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

21. 在已有平台之外，为什么还需要独立的业务 Harness?

参考答案：轻记是独立的垂直业务应用，代码中没有依赖某个原有 AI 平台或企业 SDK。它复用了 LangGraph、FastAPI 和兼容模型 API，自己补健康数据模型与可靠性规则。如果以后挂到已有平台，应让平台复用身份、模型接入和运维能力，轻记保留事件完整性、营养来源、计划版本这些领域约束，不能虚构已有平台共建经历。

[Q202 · 工程现状] 依据：backend/requirements.txt; backend/app/main.py

22. 运行内核、业务扩展与服务入口怎样分工?

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q203 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

23. 这套运行时能否迁移到其他行业?

参考答案：当前通过普通函数和 Repository 分离业务计算与图路由：营养和目标计算不需要直接操作图 API。新增饮食规则应放入业务模块，新增通用请求或恢复语义才改 Engine。这个边界尚未成为正式接口协议；若继续通用化，应抽出服务端口、节点结果契约和状态版本迁移，而不是把所有规则堆进图节点。

[Q204 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

24. 不同行业的业务指标不同，通用能力和业务评价怎么分开?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q205 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

25. 页面关闭或服务重启后，任务如何恢复?

参考答案：后台规划器按时间生成持久任务，执行器负责领取、执行、心跳与重试，当前仍同进程共享唯一 Engine。关闭小程序后，只要服务存活就能生成站内消息；服务停机期间不会实时通知，重启按限定窗口补跑。失败区分可重试与终止，达到上限进入可处理的失败状态。没有设备推送、SSE 字流续传或独立分布式队列；交互请求的重试仍使用原 request_id。

[Q206 · 工程现状与边界] 依据：backend/app/jobs/scheduler.py; backend/app/jobs/planner.py; backend/app/jobs/runner.py; backend/app/jobs/manage.py; backend/tests/test_scheduler_operations.py

26. 如何认领任务，避免重复执行?

参考答案：轻记把一次事件写入的操作键固定为 thread_id:event:事件下标，Repository 在事务内先查 operations，再写 events、revisions 和操作结果；重放时返回旧结果。HTTP 层另用 request_id 和请求指纹识别重试。它覆盖当前本地数据库副作用；若以后调用外部支付或推送服务，还必须由外部服务接受幂等键，checkpoint 本身不提供跨系统恰好一次。

[Q207 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

27. 追问：如果工具会产生外部副作用，怎样处理重复调用?

参考答案：当前健康写入在本地事务中，操作键可查已提交结果。若以后外部设备或通知调用中断，连接失败不能证明对方未执行，应先用稳定业务键查询状态；无法确定就保留 unknown 并等待对账或确认。不能一边靠 checkpoint 重放，一边对外部副作用盲目重试。当前没有这样的外部状态对账机制。

[Q208 · 现状 + 扩展设计] 依据：backend/app/infrastructure/repositories/repository.py

28. 从业务模板演进到通用运行时，需要做哪些调整?

参考答案：服务入口处理认证、协议、会话、限流和 SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

[Q209 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

29. 这套设计迁移到 ToC 场景，会遇到什么问题?

参考答案：ToB 更重租户隔离、审批审计、私有化、领域流程和 SLA。ToC 更重低延迟、个性化 Memory、海量弹性和单位成本。内核可复用，但身份、权限、记忆授权、资源调度和产品交互策略应分别设计，不能简单复制。

[Q210 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

30. 为什么考虑离开当前公司?

参考答案：用正向原因回答：当前工作已经形成了哪些积累，下一阶段希望扩大哪类责任或技术深度，以及目标岗位为什么更匹配。不要评价原公司的人和管理，也不要只把薪资作为原因。

[Q211 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

31. 其他求职流程进展如何?

参考答案：如实说明当前流程所处阶段和预计决策时间，同时表达对本岗位的兴趣。不要虚构 Offer，也不要用无法核实的信息施压。

[Q212 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

四、小红书

方向1一面

1. 平台的定位是什么，核心能力有哪些?

参考答案：轻记面向希望用自然语言记录饮食和身体状态的个人用户。核心问题是表单记录成本高、补记和纠错容易出错、后续建议缺少可追溯依据。当前交付是单用户微信小程序原型，具备五类记录、关键追问、历史查询、目标确认、复盘与站内提醒；没有企业租户、客户应用发布市场或通用低代码平台。

[Q213 · 工程现状] 依据：backend/app/main.py; miniprogram/src/app.tsx; README.md

2. 客户实际怎样使用这个平台?

参考答案：真实的产品使用路径是：建档后输入“昨天中午吃了一碗牛肉面”，补充分量，保存后在历史页纠错；计划页填写或采用默认目标并确认；饮食记录完整后标记完整，再生成复盘。代码证明这些流程存在，但仓库没有可核验的活跃用户量、使用频率或真实减重效果，不能补造运营数据。

[Q214 · 工程现状] 依据：miniprogram/src/app.tsx; README.md

3. 同时支持 Workflow 和自主 Agent 吗？应用如何发布?

参考答案：可以共用模型、工具、权限、状态和观测能力。Workflow 负责固定、可审计的流程，Agent 负责需要动态决策的局部。应用发布时固定配置与版本，经过测试和审批后再通过 API、SDK 或页面入口提供服务。

[Q215 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

4. 平台与现有产品相比，有什么差异化?

参考答案：轻记的差异化集中在健康业务约束：宽松草稿追问、逐字段营养来源、未知值传播、累计目标变化确认、数据版本防旧计划和删除上下文副本。它不追求覆盖通用平台的画布、插件和多租户功能。这些设计与场景匹配，但是否比其他方案更好要看同任务对照，不能说优势绝对。

[Q216 · 工程现状] 依据：backend/app/chat/drafts.py

5. 为什么用 LangGraph，而不是沿用原有后端语言自行实现?

参考答案：轻记需要两类等待：记录缺字段时等用户补充，计划变化较大时等用户确认。因此选择 LangGraph 的显式状态图、interrupt 和 SQLite checkpoint。图负责编排，草稿完整性、营养来源、请求去重和计划版本检查由自己的业务代码实现。当前没有统一任务基准证明它优于其他框架；选择主要依据是与这两类暂停恢复需求匹配。

[Q217 · 工程现状] 依据：backend/app/workflows/engine.py; backend/tests/test_agent.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

6. 多 Agent 采用什么协作模式?

参考答案：Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果，Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema；结果返回状态、证据及产物引用。Worker 可预声明，也可受控动态创建，动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并，副作用用幂等键；检测循环委派并传播取消，避免所有 Worker 同时修改同一业务对象。

[Q218 · 八股文 · 通用知识] 依据：参考资料：S1,S6

7. Planner 如何选择和路由到具体 Agent?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q219 · 八股文 · 通用知识] 依据：参考资料：S3,S9

8. 多个 Agent 能力相似时，会不会选错?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q220 · 八股文 · 通用知识] 依据：参考资料：S3,S9

9. 追问：路由是否正确，怎样评估?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q221 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

10. Agent 之间怎样传递上下文?

参考答案：Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果，Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema；结果返回状态、证据及产物引用。Worker 可预声明，也可受控动态创建，动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并，副作用用幂等键；检测循环委派并传播取消，避免所有 Worker 同时修改同一业务对象。

[Q222 · 八股文 · 通用知识] 依据：参考资料：S1,S6

11. 如果并行执行并写入共享状态，怎么处理冲突?

参考答案：当前后台任务已有原子领取、lease_until、owner_token、递增 fencing_token 和 job generation。心跳续租；提交与失败更新必须匹配有效执行权，旧执行者不能覆盖新结果。业务效果用稳定幂等键，报告/通知等通过事务和 outbox 协调。该协议提高单 Engine 内的恢复可靠性，并不表示已支持分布式 Worker；独立运行进程和共享集群属于后续阶段，外部服务副作用还需自己的幂等/对账协议。

[Q223 · 工程现状与边界] 依据：backend/app/infrastructure/repositories/jobs.py; backend/app/infrastructure/repositories/outbox.py; backend/app/jobs/context.py; backend/tests/test_scheduler_reliability.py

12. 检索效果用什么指标衡量?

参考答案：先建立带相关证据标注的固定问题集，并按实体、同义表达、多跳、无答案和权限场景分层。Recall@K 衡量相关证据覆盖；MRR 是首个相关结果倒数排名的均值；NDCG 结合相关性等级与位置衡量排序。回答另测忠实性、完整性、引用正确率和拒答。对照实验固定语料版本、模型和预算，一次改变主要因素，报告样本、差值和不确定性；最终质量提升不应只由单个检索分数推断。

[Q224 · 八股文 · 通用知识] 依据：参考资料：S2

13. 评测数据集怎样构建，预期召回结果由谁确定?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q225 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

14. 为什么从单一检索演进到混合检索，再加入重排?

参考答案：BM25 擅长实体、术语和精确词匹配，稠密向量检索擅长语义近邻，二者互补。分别召回后按文档/片段 ID 去重，用 RRF 等方法融合；RRF 的分数是各路 1/(k+rank) 之和，避免直接混加不同量纲的原始分数。随后用 Cross-Encoder 等重排器精排少量候选。结果高度重复时可用 MMR 在相关性与多样性间取舍。每增加一层都要验证 Recall、答案质量、延迟和成本，不能默认链路越长越好。

[Q226 · 八股文 · 通用知识] 依据：参考资料：S2,S11

15. Query Rewrite 具体做了什么?

参考答案：Query Rewrite 把口语、缩写、指代和多轮省略转换成适合检索的独立查询，也可拆成多个子问题或生成多查询。必须保留实体、时间、否定和业务约束，不能凭空补事实；通常同时保留原始查询，结合原查询与改写查询召回，避免单路漂移。对明确精确词可跳过改写。用意图保持率、证据覆盖和最终答案质量做对照，并检查改写带来的额外延迟与成本。

[Q227 · 八股文 · 通用知识] 依据：参考资料：S2

16. 追问：提升怎么测出来的，有没有失败案例？

参考答案：先建立带相关证据标注的固定问题集，并按实体、同义表达、多跳、无答案和权限场景分层。Recall@K 衡量相关证据覆盖；MRR 是首个相关结果倒数排名的均值；NDCG 结合相关性等级与位置衡量排序。回答另测忠实性、完整性、引用正确率和拒答。对照实验固定语料版本、模型和预算，一次改变主要因素，报告样本、差值和不确定性；最终质量提升不应只由单个检索分数推断。

[Q228 · 八股文 · 通用知识] 依据：参考资料：S2

17. 多模态文档如何解析和切分，对检索有什么影响？

参考答案：解析先保留文档结构与来源，再做统一索引。PDF/扫描件需要版面分析、OCR 和阅读顺序；Word 按标题、段落及表格；PPT 保留页、标题和图文关系；Excel 保留工作表、表头、单位及行列坐标；图片可用 OCR 和视觉描述，必要时保留可回看的图像引用。表格与图片描述应绑定原页，避免单独文本失去条件。解析质量会限制召回上限，应检查缺页、乱码、表格错列及来源定位。

[Q229 · 八股文 · 通用知识] 依据：参考资料：S2

18. PPL 切分的原理是什么？

参考答案：PPL 是语言模型对文本预测难度的度量，通常为平均负对数似然的指数。基于 PPL 的切分利用相邻句或局部窗口的预测难度变化，寻找可能的主题边界；不同实现的上下文、打分与边界规则并不统一。可对候选句边界计算条件负对数似然，平滑后取局部峰值，同时施加最小/最大块长及结构规则。分数高也可能来自术语、代码或 OCR 错误，不能直接等同语义断点。需与结构切分对照评测。

[Q230 · 八股文 · 通用知识] 依据：参考资料：S2

19. 用什么模型计算，成本如何？

参考答案：计算 PPL 需要能取得 Token 级对数概率的语言模型，按滑动窗口或句级条件上下文求负对数似然。模型大小、窗口重叠和重复前缀决定计算成本，可批处理、缓存和复用前缀，并先用小模型比较收益。阈值可按文档类型和分数分布调节，配合块长约束；固定阈值跨语料通常不稳。上线前同时比较检索质量、处理吞吐和入库成本，避免切分成本超过收益。

[Q231 · 八股文 · 通用知识] 依据：参考资料：S2

20. 切分阈值是固定的还是动态的？

参考答案：不宜把一个固定阈值直接用于所有文档。可以在每篇或每类文档中对句级负对数似然做归一化，用分位数、局部峰值或滑动均值差形成自适应候选，再约束最小/最大块长。标题、列表、表格和代码边界应优先遵循结构规则。阈值越敏感通常产生更多小块与索引成本，过于迟钝又混合主题；在标注问题集上调参，并用独立测试集验证，防止只拟合入库样本。

[Q232 · 八股文 · 通用知识] 依据：参考资料：S2

21. 长对话中出现偏离当前任务的消息，怎样处理边界？

参考答案：轻记把每条新消息作为新的 thread_id 工作流，待追问流程单独保存，用户可以另开记录。恢复时只把答案用于 pending_fields，当前反馈不会自动继承到新流程。若用户转为咨询、取消或说不知道，代码按必需/可选信息分别处理。它没有通用语义任务栈，但已经避免把不同餐食的关注点混在一起。

[Q233 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/chat/feedback.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

22. 短期上下文与长期记忆如何区分?

参考答案: 记忆分四层:工作记忆保存当前 State、草稿、等待点和恢复凭据;事件账本保存健康事件及修订;结构化长期事实保存偏好、过敏、专业限制等及其来源和有效期;派生层保存规律、复盘、计划与归档摘要。派生结果记录窗口、方法版本和来源版本,不能当作原始事实。工作记忆可到期清理,已保存事件按独立保留策略管理;当前不依赖向量记忆或自动对话摘要。

[Q234 · 工程现状与边界] 依据:backend/app/workflows/state.py; backend/app/health/facts.py; backend/app/health/derived.py

23. 长期记忆怎样更新?

参考答案: 事件纠错保留事件 ID、递增版本并保存修订;长期事实另有结构化主题、值、极性、来源、有效期和变更记录。明确改口的普通偏好可替代旧值,临时偏好不静默覆盖长期信息;冲突可让用户选择。过敏和专业限制到期进入待核对,继续保留保护,不因低使用频率遗忘。用户确认只是明确陈述,转述专业建议不等于独立核验;旧自由文本不自动猜成可靠结构。

[Q235 · 工程现状与边界] 依据:backend/app/health/fact_resolution.py; backend/app/health/facts.py; backend/app/contracts/facts.py; backend/tests/test_structured_facts.py

24. Memory 与固定 Workflow 的关系是什么?

参考答案: 在轻记里,记什么由健康业务决定:过敏与明确偏好长期保留,主观偏咸只属于当前反馈,未知营养保留空值。Engine 负责把 State 存进 checkpoint,Repository 提供存取;不由运行框架自行提炼用户画像。以后复用到别的行业,应替换写入、检索和遗忘策略,保留存储与生命周期接口。

[Q236 · 工程现状] 依据:backend/app/chat/feedback.py

25. 是否支持 Skill?Memory 与 Skill 的规则冲突时怎么办?

参考答案: 记忆保存事实、偏好、来源、时间、置信度和版本,不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理:保留新记录,并把旧记录标记为已被替代,同时保留审计。错误内容支持删除和更正,低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

[Q237 · 八股文 · 通用知识] 依据:通用原理与答题方法;不表示项目已实现

方向1二面

1. 请做一下自我介绍。

参考答案: 回答顺序:先说明【X 年】后端或 AI 应用研发经验,再说明近【X 年】负责的 Agent 工作、核心职责和量化结果,最后说明这些能力与岗位要求的关系。全程控制在两分钟内,只保留与岗位相关的信息。

[Q238 · 个人经历答题模板] 依据:按本人真实经历补充;不代填职级、规模、贡献或效果数字

2. 平台的整体架构和核心模块是什么?

参考答案：当前入口是 Taro/React/TypeScript 微信小程序，连接独立 FastAPI JSON API；旧 Web 前端已废弃，根路径不再托管页面。Engine 复用聊天、计划、分析图，health.db 保存业务事实，checkpoints.db 保存图进度。开发启动用 start.ps1；仓库另有 Docker 加固部署配置。后台任务与 Web 仍共享唯一 Engine，数据目录所有者锁阻止第二实例，不是独立 Worker 集群。正式对外服务仍需 HTTPS、鉴权及相应部署验收。

[Q239 · 工程现状与边界] 依据：miniprogram/package.json; backend/app/main.py; backend/app/infrastructure/data_owner.py; README.md; compose.hardened.yaml; start.ps1

3. 画布 DSL 如何转换成 LangGraph 图?

参考答案：DSL 应声明版本、节点 ID/类型、参数、类型化输入输出、边、条件、变量引用、超时与错误处理。编译流程是解析 Schema、检查节点和端口、校验引用及类型、分析可达性与依赖、验证循环边界，再生成执行图。自然语言先生成受限中间计划，再编译为 DSL，避免模型直接拼接任意代码。执行前做静态检查与沙箱测试，运行时仍检查权限和写入条件；静态合法不等于业务正确。

[Q240 · 八股文 · 通用知识] 依据：参考资料：S6,S9

4. Workflow 与 ReAct 如何复用底层能力?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q241 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

5. 与 OpenClaw 这类 Agent 相比，有什么区别?

参考答案：比较时应分开看模型能力和 Harness 能力。重点比较工具协议、上下文管理、计划与执行方式、代码与终端权限 、检查点、人工交互、扩展机制和安全边界。不能只根据一次回答效果判断产品差异。

[Q242 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

6. 长期记忆和个人工作空间怎样设计?

参考答案：记忆分四层：工作记忆保存当前 State、草稿、等待点和恢复凭据；事件账本保存健康事件及修订；结构化长期事实保存偏好、过敏、专业限制等及其来源和有效期；派生层保存规律、复盘、计划与归档摘要。派生结果记录窗口、方法版本和来源版本，不能当作原始事实。工作记忆可到期清理，已保存事件按独立保留策略管理；当前不依赖向量记忆或自动对话摘要。

[Q243 · 工程现状与边界] 依据：backend/app/workflows/state.py; backend/app/health/facts.py; backend/app/health/derived.py

7. 业务数据平台中的数据、逻辑和动作如何接入 Agent?

参考答案：数据通过受权限控制的查询接口或 RAG 接入，业务逻辑通过服务或 Workflow 接入，写操作通过有明确参数和审计的工具接入。所有调用携带用户和租户身份，读取与写入分别授权，模型不能直接连接底层数据库。

[Q244 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

8. 业务本体由平台统一建设，还是由具体项目提供?

参考答案：平台负责本体规范、标识、版本、查询接口和治理工具，具体项目负责领域实体、关系和业务含义。共性部分可以沉淀到平台，但领域定义必须由业务方确认，避免平台替业务作错误解释。

[Q245 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

9. 知识图谱怎样查询，查询结果如何用于任务?

参考答案：知识图谱用实体、关系与属性表示结构化知识。查询先做实体链接和关系识别，再生成受限图查询或调用参数化图工具，执行权限与复杂度限制后返回相关子图。它适合多跳关系、路径解释和实体约束；向量检索适合非结构化语义，两者可结合。给模型的结果应包含节点/边标识、来源、时间和置信信息，避免无界遍历。图谱维护与实体消歧成本高，需用关系类问题验证收益。

[Q246 · 八股文 · 通用知识] 依据：参考资料：S2

10. RAG 是固定执行链路，还是作为 Tool 让模型决定是否调用?

参考答案：固定 RAG 在回答前按固定流程检索，路径可控、延迟容易预算，适合多数知识问答。Agentic RAG 把检索封成工具，由模型决定是否检索、如何拆问题及是否再次检索，适合多跳和跨源探索，但调用次数、错误传播与评测更复杂。可采用混合设计：必需权限与证据检查由 Workflow 执行，局部检索策略由 Agent 决策，并设置轮次和预算。无论哪种方式，都要校验引用与证据不足场景。

[Q247 · 八股文 · 通用知识] 依据：参考资料：S1,S2

11. 工具返回内容很长时，如何裁剪?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q248 · 八股文 · 通用知识] 依据：参考资料：S1,S6

12. 使用了哪些文档切分策略，各自适合什么情况?

参考答案：常见切分有固定长度加重叠、递归分隔符、标题层级、语义切分和父子块。固定切分简单稳定；结构切分适合章节明确文档；语义切分尝试保留主题完整性但成本较高。表格保留表头与单位，代码按函数/类及依赖处理。检索可用小块定位，再扩展相邻块或父章节，处理跨块证据。每块保存来源、位置、层级、版本和权限，重叠过多会增加重复与索引成本。

[Q249 · 八股文 · 通用知识] 依据：参考资料：S2

13. PDF、PPT、Excel 分别如何处理?

参考答案：解析先保留文档结构与来源，再做统一索引。PDF/扫描件需要版面分析、OCR 和阅读顺序；Word 按标题、段落及表格；PPT 保留页、标题和图文关系；Excel 保留工作表、表头、单位及行列坐标；图片可用 OCR 和视觉描述，必要时保留可回看的图像引用。表格与图片描述应绑定原页，避免单独文本失去条件。解析质量会限制召回上限，应检查缺页、乱码、表格错列及来源定位。

[Q250 · 八股文 · 通用知识] 依据：参考资料：S2

14. 追问：图片和表格怎样解析?

参考答案：解析先保留文档结构与来源，再做统一索引。PDF/扫描件需要版面分析、OCR 和阅读顺序；Word 按标题、段落及表格；PPT 保留页、标题和图文关系；Excel 保留工作表、表头、单位及行列坐标；图片可用 OCR 和视觉描述，必要时保留可回看的图像引用。表格与图片描述应绑定原页，避免单独文本失去条件。解析质量会限制召回上限，应检查缺页、乱码、表格错列及来源定位。

[Q251 · 八股文 · 通用知识] 依据：参考资料：S2

15. 切分粒度与召回效果之间是什么关系?

参考答案：块过小容易丢失定义、条件和指代，块过大则混入无关信息、降低向量区分度并占用上下文。可先按文档结构确定边界，再用 Token 上限控制长度，适度重叠；常见做法是小块召回、父块或相邻块扩展。没有适用于所有语料的固定最佳长度。实验中固定模型、问题集和检索预算，对比必要证据覆盖、Recall@K、引用准确率、延迟及 Token 成本。

[Q252 · 八股文 · 通用知识] 依据：参考资料：S2

16. 混合检索和重排具体怎么组合?

参考答案：BM25 擅长实体、术语和精确词匹配，稠密向量检索擅长语义近邻，二者互补。分别召回后按文档/片段 ID 去重，用 RRF 等方法融合；RRF 的分数是各路 1/(k+rank) 之和，避免直接混加不同量纲的原始分数。随后用 Cross-Encoder 等重排器精排少量候选。结果高度重复时可用 MMR 在相关性与多样性间取舍。每增加一层都要验证 Recall、答案质量、延迟和成本，不能默认链路越长越好。

[Q253 · 八股文 · 通用知识] 依据：参考资料：S2,S11

17. 为什么没有采用 MMR?

参考答案：MMR 每轮选择“与查询相关、又与已选结果不太相似”的候选，常写为 λ×相关性 − (1−λ)×与已选结果的最大相似度。λ 越大越偏相关，越小越偏多样性。适用于重复段落很多、问题需要多方面证据的情况；对只需精确答案的任务可能稀释最相关证据。它不能代替召回和权限过滤，也不等同于重排模型。是否使用应通过证据覆盖率、重复率和最终回答质量验证。

[Q254 · 八股文 · 通用知识] 依据：参考资料：S2

18. Workflow 作为 Skill，究竟是什么意思?

参考答案：Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q255 · 八股文 · 通用知识] 依据：参考资料：S3,S6

19. Skill、Tool、Workflow 与通用运行内核怎样解耦?

参考答案：当前通过普通函数和 Repository 分离业务计算与图路由：营养和目标计算不需要直接操作图 API。新增饮食规则应放入业务模块，新增通用请求或恢复语义才改 Engine。这个边界尚未成为正式接口协议；若继续通用化，应抽出服务端口、节点结果契约和状态版本迁移，而不是把所有规则堆进图节点。

[Q256 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

20. 当前职级和过去的绩效情况如何?

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

[Q257 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

方向1三面

1. 平台主要解决什么问题，核心指标是什么?

参考答案：轻记面向希望用自然语言记录饮食和身体状态的个人用户。核心问题是表单记录成本高、补记和纠错容易出错、后续建议缺少可追溯依据。当前交付是单用户微信小程序原型，具备五类记录、关键追问、历史查询、目标确认、复盘与站内提醒；没有企业租户、客户应用发布市场或通用低代码平台。

[Q258 · 工程现状] 依据：backend/app/main.py; miniprogram/src/app.tsx; README.md

2. 主要部署在公有云还是客户私有环境?

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

[Q259 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

3. 当前客户、用户以及 Agent / Workflow 的使用规模如何?

参考答案：分别报告客户数、活跃用户数、已发布 Agent 和 Workflow 数、日任务量、峰值并发和统计周期。还要区分概念验证、灰度和生产，避免把创建数量当成真实使用量。

[Q260 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

4. 整体架构怎么设计?

参考答案：当前入口是 Taro/React/TypeScript 微信小程序，连接独立 FastAPI JSON API；旧 Web 前端已废弃，根路径不再托管页面。Engine 复用聊天、计划、分析图，health.db 保存业务事实，checkpoints.db 保存图进度。开发启动用 start.ps1；仓库另有 Docker 加固部署配置。后台任务与 Web 仍共享唯一 Engine，数据目录所有者锁阻止第二实例，不是独立 Worker 集群。正式对外服务仍需 HTTPS、鉴权及相应部署验收。

[Q261 · 工程现状与边界] 依据：miniprogram/package.json; backend/app/main.py; backend/app/infrastructure/data_owner.py; README.md; compose.hardened.yaml; start.ps1

5. 为什么选 LangGraph，与其他框架比较过吗?

参考答案：轻记需要两类等待：记录缺字段时等用户补充，计划变化较大时等用户确认。因此选择 LangGraph 的显式状态图、interrupt 和 SQLite checkpoint。图负责编排，草稿完整性、营养来源、请求去重和计划版本检查由自己的业务代码实现。当前没有统一任务基准证明它优于其他框架；选择主要依据是与这两类暂停恢复需求匹配。

[Q262 · 工程现状] 依据：backend/app/workflows/engine.py; backend/tests/test_agent.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

6. DSL 从加载、编译到执行，完整过程是什么?

参考答案：DSL 应声明版本、节点 ID/类型、参数、类型化输入输出、边、条件、变量引用、超时与错误处理。编译流程是解析 Schema、检查节点和端口、校验引用及类型、分析可达性与依赖、验证循环边界，再生成执行图。自然语言先生成受限中间计划，再编译为 DSL，避免模型直接拼接任意代码。执行前做静态检查与沙箱测试，运行时仍检查权限和写入条件；静态合法不等于业务正确。

[Q263 · 八股文 · 通用知识] 依据：参考资料：S6,S9

7. 节点类型如何组织，不同能力怎么接入?

参考答案：节点是接收 State、返回字段更新的 Python 函数。Chat 图包含 parse、validate_drafts、prepare_nutrition、save_ready、prepare_clarification、clarify、persist、fact、respond；计划图是 prepare、approve、activate；分析图是 analyze、review。节点可以调模型、查库、计算或 interrupt，但没有通用用户代码节点和动态节点市场。

[Q264 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

8. 与 Dify、Coze 相比有什么不同?

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

[Q265 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

9. Workflow 与 ReAct 怎样共用执行能力?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q266 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

10. Agent 能否作为一个 Workflow 节点?

参考答案：可以。该节点应有明确输入输出、工具和权限范围、预算、超时、检查点和完成条件。Workflow 负责节点前后的确定性控制，Agent 只在节点内部自主决策。失败后按节点策略重试、降级或转人工。

[Q267 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

11. Worker、Token 和模型资源如何管理?

参考答案：Worker 按任务类型和资源需求分池，使用队列、并发配额和自动扩缩容。模型 Token 按租户、任务和步骤设置预算，并记录实际用量。模型并发通过网关限流和路由，避免单一租户或长任务占满资源。

[Q268 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

12. 项目最大的挑战是什么，你具体贡献了什么?

参考答案：可讲一个有测试证据的困难场景：用户说“昨天吃得有点咸”，其中没有食物名称。如果直接要求严格餐食 Schema，会校验失败；若猜一个食物又会污染数据。现在先保留宽松 ExtractionEvent，由本地 check_drafts 决定是否追问；补充只更新待补字段，保留首次日期。测试覆盖重启和跨午夜补充，不能据此虚构某次线上事故。

[Q269 · 工程现状] 依据：backend/app/chat/drafts.py; backend/tests/test_draft_clarification.py

13. 接入 API、MCP、知识库和业务数据分别需要多少开发工作?

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

[Q270 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

14. 为什么引入 Kafka，同步的是什么数据?

参考答案：Kafka 适合解耦高吞吐事件生产与消费，支持保留日志、按分区顺序读取和重放。可用于业务变更事件、状态通知或异步数据同步；不是所有低频任务都需要引入。生产端以事务 outbox 避免数据库提交与发消息之间的双写丢失，消费者用业务键幂等，明确分区键、重试和死信策略。消费位点提交不代表外部业务副作用已原子提交；跨系统的恰好一次需要限定边界并另行设计。

[Q271 · 八股文 · 通用知识] 依据：参考资料：S8

15. 运行时如何跨客户复用?

参考答案：当前通过普通函数和 Repository 分离业务计算与图路由：营养和目标计算不需要直接操作图 API。新增饮食规则应放入业务模块，新增通用请求或恢复语义才改 Engine。这个边界尚未成为正式接口协议；若继续通用化，应抽出服务端口、节点结果契约和状态版本迁移，而不是把所有规则堆进图节点。

[Q272 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

16. 业务扩展模块里包含哪些内容?

参考答案：当前业务模块包括 drafts 的缺字段检查和增量补丁、nutrition 的营养准备与来源、targets 的默认目标公式、advice/replies 的上下文与表达校验、services 的计划统计归档。运行控制由 Engine 管理，数据由 Repository 保存。它们是普通 Python 模块，并没有通过动态插件清单加载。

[Q273 · 工程现状] 依据：backend/app/chat/drafts.py

17. 上下文如何分层，各层分别放什么?

参考答案：上下文按任务目的构造。首轮规划只收到本次文字、时间和是否选中记录，不再默认携带最近 12 条历史。本地审核计划后，从固定视图取必要数据；纠错绑定单条记录，补充只发送当前草稿和待补字段，营养估算只发送本次食物与分量。建议使用白名单结构化投影，不发送历史 raw_text。目的、资源和实际 payload 都经授权核对，最终返回用途与数据类别说明。

[Q274 · 工程现状与边界] 依据：backend/app/modeling/payloads.py; backend/app/modeling/projections.py; backend/app/modeling/boundary.py; backend/tests/test_model.py

18. 什么时候触发压缩，怎样保留关键内容?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q275 · 八股文 · 通用知识] 依据：参考资料：S1,S6

19. 当前能力处于 PoC 还是生产使用阶段?

参考答案：当前应定位为可运行的单用户本地原型，具备前后端、持久化和自动化测试，已经超出只有提示词的演示，但还不能称为生产平台。缺少用户隔离、生产监控、标准容量测试和核验营养库，也没有可证明的商业客户使用数据。演示时应展示完整流程和异常恢复，不把原型测试当线上运营。

[Q276 · 工程现状] 依据：backend/tests/; README.md; start.ps1

20. PoC 验证和最终交付分别由谁负责?

参考答案：概念验证由产品、业务和研发共同确认场景、数据和成功标准，研发完成技术验证。最终交付需要产品确认范围 ，研发保证功能与性能，测试负责验收，交付和运维负责部署与运行。责任应在项目开始时明确。

[Q277 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

21. 长期记忆如何分类、提取和更新?

参考答案：记忆分四层：工作记忆保存当前 State、草稿、等待点和恢复凭据；事件账本保存健康事件及修订；结构化长期事实保存偏好、过敏、专业限制等及其来源和有效期；派生层保存规律、复盘、计划与归档摘要。派生结果记录窗口、方法版本和来源版本，不能当作原始事实。工作记忆可到期清理，已保存事件按独立保留策略管理；当前不依赖向量记忆或自动对话摘要。

[Q278 · 工程现状与边界] 依据：backend/app/workflows/state.py; backend/app/health/facts.py; backend/app/health/derived.py

22. 多 Agent 如何进行委派、隔离和任务交接?

参考答案：Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果，Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema；结果返回状态、证据及产物引用。Worker 可预声明，也可受控动态创建，动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并，副作用用幂等键；检测循环委派并传播取消，避免所有 Worker 同时修改同一业务对象。

[Q279 · 八股文 · 通用知识] 依据：参考资料：S1,S6

23. 当前职级与入职时相比有什么变化?

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

[Q280 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

24. 为什么考虑换工作，又为什么选择这个方向?

参考答案：用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

[Q281 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

方向2一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q282 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 平台是基于 LangGraph 建设的吗?

参考答案：执行引擎基于 LangGraph，并非从零手写调度器。Engine 启动时构建 chat、plan、analysis 三张 StateGraph，再用同一个 SqliteSaver 按 thread_id 保存运行状态。FastAPI 接受请求，Engine 选择图，普通 Python 模块实现健康业务。自建部分是可靠性和领域规则，不是重新实现 LangGraph。

[Q283 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/main.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

3. 在框架之上，自己补充了哪些能力?

参考答案：LangGraph 之外，工程已经实现请求幂等、恢复回执、模型外发授权与预算、结构化事实冲突、数据血缘删除、受控输出、安全路由、可靠后台任务和分层观测。下一步优先补真实营养来源、真实模型评测及目标部署验收，再按需要做鉴权、多用户和进程拆分。没有文档 RAG、MCP、动态 Skill、任意代码执行或多 Agent 集群，不把这些当作既有成果。

[Q284 · 工程现状与边界] 依据：backend/app/workflows/runtime.py; backend/app/modeling/boundary.py; backend/app/privacy/deletion.py; backend/app/jobs/runner.py; backend/app/observability/service.py

4. 节点和 DSL 怎样转换为可执行图?

参考答案：DSL 应声明版本、节点 ID/类型、参数、类型化输入输出、边、条件、变量引用、超时与错误处理。编译流程是解析 Schema、检查节点和端口、校验引用及类型、分析可达性与依赖、验证循环边界，再生成执行图。自然语言先生成受限中间计划，再编译为 DSL，避免模型直接拼接任意代码。执行前做静态检查与沙箱测试，运行时仍检查权限和写入条件；静态合法不等于业务正确。

[Q285 · 八股文 · 通用知识] 依据：参考资料：S6,S9

5. 文件处理、RAG 和评测能力分别做了哪些扩展?

参考答案：RAG 分离线入库和在线问答。入库先解析与清洗文档，保留结构、页码、来源和权限，再切分、生成向量并写入检索索引；文档更新与删除需同步处理版本。在线先理解问题并按需改写，执行权限过滤、关键词/向量召回、融合、重排与去重，再按 Token 预算组装证据，生成带引用的答案。证据不足时澄清或拒答。评测要把解析、召回、排序、引用和答案分别统计，便于定位故障。

[Q286 · 八股文 · 通用知识] 依据：参考资料：S2,S11

6. 为什么不直接使用 Dify?

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

[Q287 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

7. 平台的定位和能力边界与已有产品有什么不同?

参考答案：轻记的差异化集中在健康业务约束：宽松草稿追问、逐字段营养来源、未知值传播、累计目标变化确认、数据版本防旧计划和删除上下文副本。它不追求覆盖通用平台的画布、插件和多租户功能。这些设计与场景匹配，但是否比其他方案更好要看同任务对照，不能说优势绝对。

[Q288 · 工程现状] 依据：backend/app/chat/drafts.py

8. 目前有哪些真实业务在使用?

参考答案：真实的产品使用路径是：建档后输入“昨天中午吃了一碗牛肉面”，补充分量，保存后在历史页纠错；计划页填写或采用默认目标并确认；饮食记录完整后标记完整，再生成复盘。代码证明这些流程存在，但仓库没有可核验的活跃用户量、使用频率或真实减重效果，不能补造运营数据。

[Q289 · 工程现状] 依据：miniprogram/src/app.tsx; README.md

9. 平台面向 ToB 还是 ToC?

参考答案：ToB 更重租户隔离、审批审计、私有化、领域流程和 SLA。ToC 更重低延迟、个性化 Memory、海量弹性和单位成本。内核可复用，但身份、权限、记忆授权、资源调度和产品交互策略应分别设计，不能简单复制。

[Q290 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

10. Agent 生态变化很快，这些变化对平台有什么影响?

参考答案：演进重点应放在稳定接口和可替换能力：模型、框架、工具协议和记忆实现都通过适配层接入。内核保持任务状态、权限、恢复、观测和评测的稳定，新的模型或框架先经过兼容性和回归评测，再逐步替换。

[Q291 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

11. 平台更偏用于配置和管理的管理台，还是直接承接在线业务流量的运行台?

参考答案：轻记网页直接承接个人记录、纠错和咨询，是业务应用入口，也包含档案与计划设置。它不是只管配置的通用管理台。运行执行在同一个 FastAPI 进程里的 Engine，设置页更新档案与偏好后影响数据版本；当前未拆控制面和运行面，也没有共享 Worker 服务。

[Q292 · 工程现状] 依据：miniprogram/src/app.tsx; backend/app/main.py

12. 配置控制面与运行执行面如何划分?

参考答案：当前控制配置包括 Settings、档案、偏好、目标来源和规则版本，运行执行在 Engine 三张图里，均由同一个 FastAPI 服务承载。单用户和少量固定图还不需要独立控制面。以后有多模型、多用户、动态发布版本时，再把配置管理、发布、权限和评测服务拆出，运行任务只引用不可变版本。

[Q293 · 工程现状] 依据：backend/app/common/config.py

13. 开放自治 Agent 与固定 Workflow 应怎样取舍?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q294 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

14. 哪些具体业务适合采用 Agent?

参考答案：适合目标明确但执行路径需要根据上下文动态决定、需要调用多种工具、允许一定探索且结果可以校验的任务。 规则固定、低延迟、强事务或错误代价极高的流程更适合普通服务或 Workflow。

[Q295 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

15. 公司内部是否已有相关 SDK 或托管产品，为什么仍需要建设这套能力?

参考答案：轻记是独立的垂直业务应用，代码中没有依赖某个原有 AI 平台或企业 SDK。它复用了 LangGraph、FastAPI 和兼容模型 API，自己补健康数据模型与可靠性规则。如果以后挂到已有平台，应让平台复用身份、模型接入和运维能力，轻记保留事件完整性、营养来源、计划版本这些领域约束，不能虚构已有平台共建经历。

[Q296 · 工程现状] 依据：backend/requirements.txt; backend/app/main.py

16. 如果模型跳过权限判断或审批节点，仅靠 Skill 描述能否保证强约束流程?

参考答案：轻记把强约束放进固定图和写入接口：所有新餐食要先经过草稿校验与营养准备，PreparedEvent 不通过就不能保存；计划激活调用 decide_plan 核对确认、数据版本和基准计划。模型即使说“已保存”或“跳过审批”也不能直接改库。将来开放 Agent 时同样需要执行端门禁，Skill 描述不能替代。

[Q297 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

17. 必需的工具没有调用、必需的产物没有生成时，怎样判断任务是否完成?

参考答案：轻记任务完成要看业务后置条件：记录 ID 能在 events 找到、营养未知被明确标识、卡片数值来自保存值；计划任务还要区分 active、rejected、stale。waiting_input 表示尚需用户，不能计为完整保存。未来可把这些条件声明成机器可校验契约；当前 completed 是图执行完毕，仍需结合结果语义判断业务是否达成。

[Q298 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

18. 追问：运行结束时校验不通过，后续怎么处理，是否计为成功?

参考答案：严格事件校验失败时不允许写库；必需信息缺失则追问，用户取消则保留已完成独立记录。建议表达校验失败会本地兜底，记录可以仍然成功；过时计划返回 stale 且目标不变。评测要分记录成功、表达降级和目标未采用，不能把 HTTP 200 或图 completed 一律算作所有业务成功。

[Q299 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

19. 对工作地点是否有要求?

参考答案：按真实情况给明确边界：首选【城市】，可接受【城市/出差频率/迁移时间】。若有家庭或通勤约束，直接说明可协商范围和最早到岗时间。

[Q300 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

方向2二面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q301 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 挑一个最有代表性的项目介绍一下，为什么选择它?

参考答案：我选轻记，因为它能完整讲清一条 Agent 业务闭环：自然语言输入拆成健康草稿，缺字段时追问，营养准备后写库，支持历史纠错，再用完整记录做计划和复盘。最有代表性的设计是 checkpoint 恢复配合幂等写入、未知营养不算零，以及计划确认时核对数据版本。它仍是单用户本地原型。

[Q302 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py; README.md

3. 运行内核、业务扩展和服务入口如何分层?

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q303 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

4. Durable Checkpoint 具体保存了什么?

参考答案：图快照由 LangGraph SqliteSaver 保存，包括 State 和执行进度：extraction、待补字段、事件下标、prepared_events、record_ids、计划数据等。自己维护的 health.db 保存事实、workflow API 结果、请求指纹、回执和操作去重。checkpoint 用于继续执行，Repository 决定业务事实是否已提交；两者不是同一种存储，也不是同一事务。

[Q304 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

5. 这部分是自行实现的，还是直接使用 LangGraph 的快照?

参考答案：checkpoint 快照直接使用 LangGraph SqliteSaver，未自行实现快照格式。自己补的是 health.db 中的请求指纹、workflow 结果、恢复 receipts 和 events 操作键，解决 API 重试及副作用重放。两者通过 thread_id 关联但并非一个事务；因此框架恢复和业务恰好一次不能混为一谈。

[Q305 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

6. 为什么只用 LangGraph 承载核心循环，而不是把全部运行机制交给它?

参考答案：当前 LangGraph 承担 StateGraph、条件路由、interrupt 和 checkpoint；Engine 另外管理请求指纹、回执、锁和 API 结果。营养准备、历史检索、计划版本、统计、归档都在普通 Python 模块中。边界仍有耦合：Engine 把图和业务函数连在一起，还不是可插拔通用 Runtime，但更换编排层不必重写所有健康算法。

[Q306 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

7. 工具调用怎样保证幂等?

参考答案：轻记把一次事件写入的操作键固定为 thread_id:event:事件下标，Repository 在事务内先查 operations，再写 events、revisions 和操作结果；重放时返回旧结果。HTTP 层另用 request_id 和请求指纹识别重试。它覆盖当前本地数据库副作用；若以后调用外部支付或推送服务，还必须由外部服务接受幂等键，checkpoint 本身不提供跨系统恰好一次。

[Q307 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

8. 追问：调用时连接断了，但不知道外部操作是否成功，这种情况怎么办?

参考答案：当前健康写入在本地事务中，操作键可查已提交结果。若以后外部设备或通知调用中断，连接失败不能证明对方未执行，应先用稳定业务键查询状态；无法确定就保留 unknown 并等待对账或确认。不能一边靠 checkpoint 重放，一边对外部副作用盲目重试。当前没有这样的外部状态对账机制。

[Q308 · 现状 + 扩展设计] 依据：backend/app/infrastructure/repositories/repository.py

9. 幂等和可靠性由 Harness 保证，还是由具体业务工具保证?

参考答案：轻记 Engine 的锁防止当前进程内同时执行关键流程，Repository 的操作键和事务保证同一业务操作重放不重复；二者职责不同，锁释放后重试仍可能再次到达。运行层负责稳定传播键，写入服务负责原子去重。跨系统工具也要在副作用所在系统实现幂等，仅 checkpoint 或分布式锁无法保证。

[Q309 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

10. 运行租约和 Fencing Token 分别解决什么问题?

参考答案：当前后台任务已有原子领取、lease_until、owner_token、递增 fencing_token 和 job generation。心跳续租；提交与失败更新必须匹配有效执行权，旧执行者不能覆盖新结果。业务效果用稳定幂等键，报告/通知等通过事务和 outbox 协调。该协议提高单 Engine 内的恢复可靠性，并不表示已支持分布式 Worker；独立运行进程和共享集群属于后续阶段，外部服务副作用还需自己的幂等/对账协议。

[Q310 · 工程现状与边界] 依据：backend/app/infrastructure/repositories/jobs.py; backend/app/infrastructure/repositories/outbox.py; backend/app/jobs/context.py; backend/tests/test_scheduler_reliability.py

11. Token 为什么需要单调递增，由谁发放?

参考答案：当前后台任务已有原子领取、lease_until、owner_token、递增 fencing_token 和 job generation。心跳续租；提交与失败更新必须匹配有效执行权，旧执行者不能覆盖新结果。业务效果用稳定幂等键，报告/通知等通过事务和 outbox 协调。该协议提高单 Engine 内的恢复可靠性，并不表示已支持分布式 Worker；独立运行进程和共享集群属于后续阶段，外部服务副作用还需自己的幂等/对账协议。

[Q311 · 工程现状与边界] 依据：backend/app/infrastructure/repositories/jobs.py; backend/app/infrastructure/repositories/outbox.py; backend/app/jobs/context.py; backend/tests/test_scheduler_reliability.py

12. Worker 写状态时怎样校验 Token，是否使用 CAS?

参考答案：当前后台任务已有原子领取、lease_until、owner_token、递增 fencing_token 和 job generation。心跳续租；提交与失败更新必须匹配有效执行权，旧执行者不能覆盖新结果。业务效果用稳定幂等键，报告/通知等通过事务和 outbox 协调。该协议提高单 Engine 内的恢复可靠性，并不表示已支持分布式 Worker；独立运行进程和共享集群属于后续阶段，外部服务副作用还需自己的幂等/对账协议。

[Q312 · 工程现状与边界] 依据：backend/app/infrastructure/repositories/jobs.py; backend/app/infrastructure/repositories/outbox.py; backend/app/jobs/context.py; backend/tests/test_scheduler_reliability.py

13. 平台的节点抽象是什么样的?

参考答案：节点是接收 State、返回字段更新的 Python 函数。Chat 图包含 parse、validate_drafts、prepare_nutrition、save_ready、prepare_clarification、clarify、persist、fact、respond；计划图是 prepare、approve、activate；分析图是 analyze、review。节点可以调模型、查库、计算或 interrupt，但没有通用用户代码节点和动态节点市场。

[Q313 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

14. 举一个比较复杂的节点，说明它如何执行。

参考答案：较复杂的节点是 prepare_nutrition：先校验完整事件，根据稳定事件下标读取缓存或旧记录，比较食物、分量、做法指纹，保留有效用户营养，缺字段才调用粗估器，再验证数值范围、热量一致性和来源。结果先放 State.prepared_events，再由保存节点读取 PreparedEvent；失败时营养未知而非虚构零值。

[Q314 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

15. 业务数据、逻辑和动作如何接入 Workflow 或 Agent?

参考答案：业务数据通过带权限的查询接口或 RAG 接入，业务逻辑封装为服务或 Workflow，写操作封装为经过审计的工具。Workflow 可确定性调用这些能力，Agent 只能从授权候选中选择。模型不能直接连接底层数据库。

[Q315 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

16. 多个任务共享同一张执行图时，运行状态怎样隔离?

参考答案：Engine 启动后复用已编译图，但每次新请求生成 UUID thread_id，checkpoint 按这个 ID 存独立 State；conversation_id 用于消息归组，不等同于图状态或身份。图中的模型和 Repository 由 Engine 持有，不能放进 State。当前仍共享单用户数据集，若做多用户还必须额外增加身份隔离。

[Q316 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

17. 动态 DSL 每次编译为图，会不会影响性能?

参考答案：将 DSL 规范化后，按内容哈希、编译器版本、节点插件版本与策略版本缓存编译结果；配置变化使缓存失效。共享的是不可变图结构，每次执行创建独立 State、任务 ID 和访问上下文，不能复用有用户数据的运行对象。编译与执行延迟应分开测量；稳定高频流程可预编译，动态低频流程按需编译。缓存命中也不能跳过运行时鉴权和业务版本校验。

[Q317 · 八股文 · 通用知识] 依据：参考资料：S6

18. Agent 如何持续做评测?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q318 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

19. 测试用例、参考轨迹、任务完成度和 Judge 怎么配合?

参考答案: 把评测拆成任务完成、答案质量、轨迹合规、延迟/成本和安全。参考标准（Ground Truth）由真实任务抽样、专家标注和生产失败样本构成，保存期望结果及允许的多种轨迹。LLM Judge 用明确的评分标准、盲测和多 Judge/人工抽检校准，不能只让同类模型自评。发布前回归，线上灰度 A/B 并设回滚阈值。

[Q319 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

20. 如果主 Agent 跳过了必须执行的规则检查，却直接完成发布，怎么发现和阻止?

参考答案: 轻记把强约束放进固定图和写入接口：所有新餐食要先经过草稿校验与营养准备，PreparedEvent 不通过就不能保存；计划激活调用 decide_plan 核对确认、数据版本和基准计划。模型即使说“已保存”或“跳过审批”也不能直接改库。将来开放 Agent 时同样需要执行端门禁，Skill 描述不能替代。

[Q320 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

21. 如何声明必须调用的工具、必须生成的产物?

参考答案: 轻记任务完成要看业务后置条件：记录 ID 能在 events 找到、营养未知被明确标识、卡片数值来自保存值；计划任务还要区分 active、rejected、stale。waiting_input 表示尚需用户，不能计为完整保存。未来可把这些条件声明成机器可校验契约；当前 completed 是图执行完毕，仍需结合结果语义判断业务是否达成。

[Q321 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

22. 最后的校验没通过时怎么处理?

参考答案: 严格事件校验失败时不允许写库；必需信息缺失则追问，用户取消则保留已完成独立记录。建议表达校验失败会本地兜底，记录可以仍然成功；过时计划返回 stale 且目标不变。评测要分记录成功、表达降级和目标未采用，不能把 HTTP 200 或图 completed 一律算作所有业务成功。

[Q322 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

23. 将 Workflow 封装为 Skill，会不会限制 Agent 自由度，又带来什么成本?

参考答案: Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q323 · 八股文 · 通用知识] 依据：参考资料：S3,S6

24. 大量运行中任务需要迁移时，怎样处理?

参考答案：迁移先固定旧任务的图、Schema、插件及模型策略版本，暂停新领取或让旧 Worker 排空，并确保任务已到可恢复边界。复制或转换检查点后，通过租约和递增 fencing token 转移执行权，旧 Worker 写入必须被拒绝。按稳定 task_id/幂等键恢复，核对外部副作用，使用小批量、可回滚切流和卡住任务扫描。新旧状态不兼容时保留旧运行版本执行完，不应强行让新代码解释旧快照。

[Q324 · 八股文 · 通用知识] 依据：参考资料：S6,S8

25. 追问：如何避免任务丢失或一直卡住?

参考答案：迁移先固定旧任务的图、Schema、插件及模型策略版本，暂停新领取或让旧 Worker 排空，并确保任务已到可恢复边界。复制或转换检查点后，通过租约和递增 fencing token 转移执行权，旧 Worker 写入必须被拒绝。按稳定 task_id/幂等键恢复，核对外部副作用，使用小批量、可回滚切流和卡住任务扫描。新旧状态不兼容时保留旧运行版本执行完，不应强行让新代码解释旧快照。

[Q325 · 八股文 · 通用知识] 依据：参考资料：S6,S8

26. 如果同时有独立复现和团队项目，如何区分各自覆盖的能力与个人贡献?

参考答案：把内容分成三部分：独立复现验证了哪些机制，团队项目交付了哪些生产能力，个人在团队项目中负责哪些决策和实现。分别说明代码来源、运行环境、数据规模和验证结果，不能把复现结果当成团队生产成果。

[Q326 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

27. 如何比较 LangGraph、AgentScope、Google ADK 等框架?

参考答案：不要只列功能。用同一组任务比较状态模型、持久化恢复、多 Agent 协议、工具生态、可观测性、部署复杂度、扩展边界和社区成熟度，再做最小概念验证（PoC）。最终选择取决于团队语言栈、故障模型和交付形态，并保留适配层降低替换成本。

[Q327 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

五、美团

一面

1. 选一个最有代表性的项目，说明解决的问题、业务价值和个人职责。

参考答案：轻记解决的是个人健康记录从“随口说”到“可纠错、可统计”的转换问题。交付包括微信小程序、五类事件数据、可恢复工作流、计划与复盘。工程价值可用字段正确率、重试零重复、追问完成率、过时计划拦截率和用户记录完整度衡量；当前没有生产付费客户和医学效果数据，职责也应以本人实际贡献为准。

[Q328 · 工程现状] 依据：backend/tests/test_agent.py; README.md

2. 为什么划分为通用运行内核、业务扩展和服务入口?

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q329 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

3. 业务扩展模块与 Skill 是什么关系?

参考答案：Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q330 · 八股文 · 通用知识] 依据：参考资料：S3,S6

4. 除了声明文件或 YAML 配置，还包含哪些真正的运行逻辑?

参考答案：轻记的真正业务逻辑在 Python 代码中：检查缺失字段、合并限定补丁、营养单位换算、保存修订、计划变化比较和归档。图用 Python 装配，页面只是收集输入与展示结果。即使以后增加 YAML 或 Skill 描述，也只能声明接口与策略，状态恢复、事务和健康约束仍需要研发实现并测试。

[Q331 · 工程现状] 依据：backend/app/chat/drafts.py

5. 用代码实现的 Workflow，怎样装配进运行时?

参考答案：轻记的 Workflow 是可执行 Python 图：Engine 启动时把节点函数绑定到 StateGraph，添加固定和条件边，compile 时注入 SqliteSaver。当前没有读取 Markdown 自动执行脚本的 Skill 运行器。若对外包装 Skill，描述文件说明用途和契约，执行入口则调用这张图，返回 completed、waiting_input 或 failed 结果。

[Q332 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

6. Tool 和 Skill 数量增加后，上下文怎么控制?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q333 · 八股文 · 通用知识] 依据：参考资料：S3,S9

7. 不同业务域的 Sub-agent，能力和权限怎样划分?

参考答案：身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q334 · 八股文 · 通用知识] 依据：参考资料：S9,S10

8. 渐进式披露具体怎样实现?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q335 · 八股文 · 通用知识] 依据：参考资料：S3,S9

9. 这些设计与标准 ReAct Loop 或现有开源 Agent 有什么不同?

参考答案：核心循环是 Thought/Decision→Action→工具结果（Observation）→更新状态，直到 Final 或触发预算/取消/错误边界。生产实现还必须加入工具鉴权、参数校验、超时、幂等、Checkpoint、上下文预算和可观测事件。循环迟迟不结束时用最大轮次、重复调用检测和无进展判定收敛。

[Q336 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

10. 你主要负责运行稳定性，还是也负责最终输出质量?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q337 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

11. Workflow 什么时候接管执行，执行后如何归还控制权?

参考答案：当命中固定业务规则、模型明确选择对应 Skill，或进入必须执行的阶段时，主循环保存状态并把控制权交给 Workflow。Workflow 完成、失败或等待后返回结构化结果与状态，主循环再决定继续推理、补救还是结束。

[Q338 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

12. 触发条件是固定规则，还是由模型决定?

参考答案：轻记结合两类触发：模型理解文本产生 intent，程序根据必需信息、纠错目标和事件是否完整选节点；计划与复盘也可以通过明确 API 或后台任务触发。咨询场景由本地正则分类。模型不能决定跳过 PreparedEvent、累计变化确认或版本检查，所以语义路由和业务门禁分开。

[Q339 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

13. 自然语言输入、页面操作和显式选择 Skill，是否走同一条链路?

参考答案：自然语言走 POST /api/chat 与 Chat 图，计划页走 /api/plans/propose 与 Plan 图，复盘走 /api/reviews 与 Analysis 图，恢复统一走 workflows/{thread_id}/resume。它们共享 Engine、Repository 和领域服务，但并非所有页面动作都先进入同一 ReAct Loop，也没有显式 Skill 选择入口。

[Q340 · 工程现状] 依据：backend/app/main.py

14. 必须执行某个 Skill 时，怎样保证模型不会选错或跳过?

参考答案：轻记把强约束放进固定图和写入接口：所有新餐食要先经过草稿校验与营养准备，PreparedEvent 不通过就不能保存；计划激活调用 decide_plan 核对确认、数据版本和基准计划。模型即使说“已保存”或“跳过审批”也不能直接改库。将来开放 Agent 时同样需要执行端门禁，Skill 描述不能替代。

[Q341 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

15. 没有调用必需工具时，系统能否发现和补救?

参考答案：当前固定图保证营养准备排在两条保存路径之前，prepared_record 还要求状态里存在且通过 PreparedEvent 的结果，缺少就拒绝保存。未来动态 Agent 若遗漏步骤，应由后置条件检测并进入受限补救流程；没有可补证据就返回未完成，不能仅靠模型在回复中承诺“已检查”。

[Q342 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

16. 举一个真实 Bad Case，说明原因、修复过程和结果。

参考答案：可讲一个有测试证据的困难场景：用户说“昨天吃得有点咸”，其中没有食物名称。如果直接要求严格餐食 Schema，会校验失败；若猜一个食物又会污染数据。现在先保留宽松 ExtractionEvent，由本地 check_drafts 决定是否追问；补充只更新待补字段，保留首次日期。测试覆盖重启和跨午夜补充，不能据此虚构某次线上事故。

[Q343 · 工程现状] 依据：backend/app/chat/drafts.py; backend/tests/test_draft_clarification.py

17. 追问：除了修改 Prompt，还采取了什么工程措施?

参考答案：把用户文本和模型输出视为待验证数据。系统不授予任意 SQL 或命令执行权，模型计划只能触发本地批准的固定视图与业务动作，外发按用途、资源、Schema 和一次性授权检查。输出策略只接受已批准的 point_id/variant_id，数字、日期、保存状态和追问由本地生成；未知标识或自由正文失败回退本地表达。输入安全路由与写入校验也在恢复路径执行。提示词只是辅助，执行端约束才是边界。

[Q344 · 工程现状与边界] 依据：backend/app/modeling/boundary.py; backend/app/modeling/payloads.py; backend/app/policy/output.py; backend/app/workflows/safety.py; backend/tests/test_output_policy.py

18. 跨多轮对话使用同一个 Skill，会不会反复注入相同内容?

参考答案：不应在每轮无条件重复注入整个 Skill。可在会话记录已激活的 Skill ID 与版本，保留最小规则摘要，只有切换能力、规则更新或关键边界需要重申时加载详细内容。每次实际调用仍独立做权限、参数和状态检查，不能把“曾注入过规则”当作授权。上下文压缩后要保留当前能力、关键约束、未完成步骤和版本；发现规则冲突时由明确优先级处理。

[Q345 · 八股文 · 通用知识] 依据：参考资料：S3,S9

19. 哪些消息会持久化到历史记录里?

参考答案：业务聊天与工作流状态仍可能包含健康信息，不能说全部匿名化。当前新记录按已验证片段保留最小原文，记录来源、版本、查询范围和原文副本血缘。删除先预览实际影响；共享原文可重绑独立片段，无法证明独立的旧数据扩大预览范围，变化后重新确认。持久删除作业清理业务副本和 checkpoint，恢复前先完成清理；观测日志另用白名单，不能混用业务正文。

[Q346 · 工程现状与边界] 依据：backend/app/privacy/fragments.py; backend/app/privacy/provenance.py; backend/app/privacy/deletion.py; backend/tests/test_privacy_deletion.py

20. 追问：不希望保存的内容在哪一步过滤，数据模型怎样区分?

参考答案：业务聊天与工作流状态仍可能包含健康信息，不能说全部匿名化。当前新记录按已验证片段保留最小原文，记录来源、版本、查询范围和原文副本血缘。删除先预览实际影响；共享原文可重绑独立片段，无法证明独立的旧数据扩大预览范围，变化后重新确认。持久删除作业清理业务副本和 checkpoint，恢复前先完成清理；观测日志另用白名单，不能混用业务正文。

[Q347 · 工程现状与边界] 依据：backend/app/privacy/fragments.py; backend/app/privacy/provenance.py; backend/app/privacy/deletion.py; backend/tests/test_privacy_deletion.py

21. 创建一个结果与修改已有结果，是否分别处理?

参考答案：新增与纠错走不同路径。新事件生成 ID；纠错候选由本地查询并绑定，歧义时先让用户选择，模型只返回选中记录的受限修改，不能自由指定数据库 ID。提交复核来源和版本，保留修订历史。食物、分量或做法变更会使营养指纹失效并重新准备。检索仍是结构化 SQL，不存在向量重排器；如果以后加重排，也应只允许返回已批准候选并校验完整性。

[Q348 · 工程现状与边界] 依据：backend/app/modeling/payloads.py; backend/app/evidence/corrections.py; backend/app/workflows/chat_parsing.py; backend/tests/test_grounding.py

22. 用户提出超出能力范围的请求时，系统怎么回应?

参考答案：轻记能处理记录、查询和普通饮食建议；医疗诊断、药物调整和未支持的外部动作应明确说明范围。当前代码对部分医疗词有边界提示，不能当成完整医学识别器。表达模型生成不受支持数字或诊疗结论时会被拒绝并本地兜底，不能为了显得有帮助而编造事实或保存空记录。

[Q349 · 工程现状] 依据：backend/app/chat/replies.py

23. 普通文本回答与结构化 Artifact 如何区分?

参考答案：轻记的结构化输出主要是餐食卡片、计划与复盘 JSON，分别由保存值和业务函数生成，不是任意文件/布局 Artifact。普通 reply 负责解释，卡片负责显示数值。用户要求修改不支持字段时，应说明当前可修改范围并走明确接口；不能只改变文字却让数据库保持旧值，也不应把未知字段直接写进 JSON。

[Q350 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

24. 如果用户要求修改 Artifact 中不支持的字段或布局，怎么办?

参考答案：轻记当前卡片和计划字段由 Schema 与前端组件固定，不能任意改布局。对不支持的字段应说明可改范围，必要时转成用户备注或功能需求；已经支持的食物、分量修改则走 event_id 纠错，重新校验营养。不能仅在回复中说“已修改”却没有数据提交，也不应把任意字段写入结构化结果。

[Q351 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

25. 多租户隔离在哪些层实现?

参考答案：身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q352 · 八股文 · 通用知识] 依据：参考资料：S9,S10

26. 沙箱解决什么问题?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q353 · 八股文 · 通用知识] 依据：参考资料：S9,S10

27. 工具幂等与分布式锁是什么关系?

参考答案：轻记 Engine 的锁防止当前进程内同时执行关键流程，Repository 的操作键和事务保证同一业务操作重放不重复；二者职责不同，锁释放后重试仍可能再次到达。运行层负责稳定传播键，写入服务负责原子去重。跨系统工具也要在副作用所在系统实现幂等，仅 checkpoint 或分布式锁无法保证。

[Q354 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

28. 超时、重试、降级和错误返回怎样处理?

参考答案：当前模型接口非流式，普通调用默认 45 秒超时，安全分类默认 8 秒，并受运行段截止时间、响应大小和持久化调用预算约束。解析失败返回安全错误，营养失败保留未知，建议选择失败回退本地已批准表达；它们不是同一降级语义。后台可重试任务有独立退避与失败分类，不等于所有模型请求自动重试。尚无供应商级 Token 费用统计或自动备用模型切换。

[Q355 · 工程现状与边界] 依据：backend/app/modeling/transport.py; backend/app/modeling/errors.py; backend/app/common/config.py; backend/app/jobs/runner.py

29. 每次任务是否都有独立工作空间?

参考答案：每个任务至少应有独立的逻辑工作空间和访问令牌。高风险代码或文件任务使用独立容器或临时目录。低风险只读任务可共享底层存储，但路径、权限、缓存和清理必须按任务与租户隔离。

[Q356 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

30. Agent 从一个业务域访问另一个业务域时，权限怎样控制?

参考答案：身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q357 · 八股文 · 通用知识] 依据：参考资料：S9,S10

31. 当前实际使用规模如何?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q358 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

32. 为什么不直接用通用 Agent，加行业专家和同一套 MCP 工具?

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

[Q359 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

33. 追问：在相同模型、工具和任务条件下，自研方案到底好在哪里?

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

[Q360 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

34. 如果通用 Agent 效果更好，客户为什么还要为自研方案付费?

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

[Q361 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

35. 怎样避免最终变成高成本的定制开发?

参考答案：先限定产品边界，再把需求分为通用能力、可配置差异和客户专有逻辑。通用能力进入平台，可配置差异进入模板或插件，专有逻辑由项目层维护。交付前评估复用率、维护责任和升级成本，不能让客户代码进入核心内核。

[Q362 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

36. 探索型任务和稳定的生产任务，是否应该采用不同方案?

参考答案：应采用不同方案。探索型任务可以保留较高自主性和较宽工具范围，但需要预算和安全边界。稳定生产任务应使用 Workflow、强校验、固定权限和明确回滚。两者可以共享工具和运行基础设施，但控制方式不同。

[Q363 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

37. 产品或解决方案角色如何参与定义价值，而不是只由研发判断?

参考答案：产品或解决方案人员负责确认目标用户、业务流程、成功标准和付费价值，研发负责可行性、成本、风险和指标实现。立项、评测和验收都应由双方共同完成，不能只用技术指标判断价值。

[Q364 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

六、拼多多

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q365 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 项目是否已经落地，实际使用规模如何?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q366 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

3. 怎样区分 Agent Harness 与 Agent Framework?

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、 工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

[Q367 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

4. 运行内核包含哪些主要模块?

参考答案：当前业务模块包括 drafts 的缺字段检查和增量补丁、nutrition 的营养准备与来源、targets 的默认目标公式、advice/replies 的上下文与表达校验、services 的计划统计归档。运行控制由 Engine 管理，数据由 Repository 保存。它们是普通 Python 模块，并没有通过动态插件清单加载。

[Q368 · 工程现状] 依据：backend/app/chat/drafts.py

5. 项目主要面向 ToB 还是 ToC?

参考答案：ToB 更重租户隔离、审批审计、私有化、领域流程和 SLA。ToC 更重低延迟、个性化 Memory、海量弹性和单位成本。内核可复用，但身份、权限、记忆授权、资源调度和产品交互策略应分别设计，不能简单复制。

[Q369 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

6. 通用运行层与业务能力如何分层?

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q370 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

7. 多个任务共享 Worker 时，状态怎么管理?

参考答案：Engine 启动后复用已编译图，但每次新请求生成 UUID thread_id，checkpoint 按这个 ID 存独立 State；conversation_id 用于消息归组，不等同于图状态或身份。图中的模型和 Repository 由 Engine 持有，不能放进 State。当前仍共享单用户数据集，若做多用户还必须额外增加身份隔离。

[Q371 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

8. 为什么同一时刻只激活一个 Skill?

参考答案：不应在每轮无条件重复注入整个 Skill。可在会话记录已激活的 Skill ID 与版本，保留最小规则摘要，只有切换能力、规则更新或关键边界需要重申时加载详细内容。每次实际调用仍独立做权限、参数和状态检查，不能把“曾注入过规则”当作授权。上下文压缩后要保留当前能力、关键约束、未完成步骤和版本；发现规则冲突时由明确优先级处理。

[Q372 · 八股文 · 通用知识] 依据：参考资料：S3,S9

9. Workflow 与 ReAct 如何衔接?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q373 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

10. 不同行业的业务差异怎样处理，能否不修改内核?

参考答案：当前通过普通函数和 Repository 分离业务计算与图路由：营养和目标计算不需要直接操作图 API。新增饮食规则应放入业务模块，新增通用请求或恢复语义才改 Engine。这个边界尚未成为正式接口协议；若继续通用化，应抽出服务端口、节点结果契约和状态版本迁移，而不是把所有规则堆进图节点。

[Q374 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

11. 共用工作空间会不会导致文件或数据串到其他任务?

参考答案：每个任务至少应有独立的逻辑工作空间和访问令牌。高风险代码或文件任务使用独立容器或临时目录。低风险只读任务可共享底层存储，但路径、权限、缓存和清理必须按任务与租户隔离。

[Q375 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

12. 沙箱如何设计?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q376 · 八股文 · 通用知识] 依据：参考资料：S9,S10

13. Docker、Kubernetes 或 Serverless 分别适合什么情况?

参考答案: 按信任和风险分级隔离：低风险只读调用可共享服务，高风险代码使用独立受限环境。Serverless 通常便于弹性与托管，代价是冷启动、时限及运行环境限制；按任务创建 Pod 更便于镜像、资源和网络定制，但启动与运维成本更高。可用预热池和镜像缓存减延迟，同时严格清除上一任务状态。评估需比较冷/热启动 P95、资源单价、并发、隔离边界和清理可靠性，不能仅看平均启动时间。

[Q377 · 八股文 · 通用知识] 依据：参考资料：S10

14. 文件、网络和系统权限如何限制?

参考答案: 把用户文本和模型输出视为待验证数据。系统不授予任意 SQL 或命令执行权，模型计划只能触发本地批准的固定视图与业务动作，外发按用途、资源、Schema 和一次性授权检查。输出策略只接受已批准的 point_id/variant_id，数字、日期、保存状态和追问由本地生成；未知标识或自由正文失败回退本地表达。输入安全路由与写入校验也在恢复路径执行。提示词只是辅助，执行端约束才是边界。

[Q378 · 工程现状与边界] 依据：backend/app/modeling/boundary.py; backend/app/modeling/payloads.py; backend/app/policy/output.py; backend/app/workflows/safety.py; backend/tests/test_output_policy.py

15. Skill 选择正确与否，如何评估?

参考答案: 验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q379 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

16. 主 Agent、专用 Agent 和 Sub-agent 如何配置?

参考答案: Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果，Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema；结果返回状态、证据及产物引用。Worker 可预声明，也可受控动态创建，动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并，副作用用幂等键；检测循环委派并传播取消，避免所有 Worker 同时修改同一业务对象。

[Q380 · 八股文 · 通用知识] 依据：参考资料：S1,S6

17. Memory 由谁决定写入，什么时候写?

参考答案: 记忆写入由业务边界控制：完整事件经校验、营养准备和事务入账；明确长期信息走结构化事实服务，带证据、有效期和修订。助手建议、模型推理或一次偏咸反馈不会自动升级为长期事实。读取使用与任务目的匹配的投影，纠错后让相关派生结果过期。管理接口支持新增、修改、撤回、冲突选择、历史与隐私删除；它仍是领域服务，不是动态 Memory 插件协议。

[Q381 · 工程现状与边界] 依据：backend/app/health/facts.py; backend/app/api/profile.py; backend/app/infrastructure/repositories/facts.py; backend/app/modeling/projections.py

18. 记忆存在哪里，怎样向量化和检索?

参考答案：向量存储按数据量、过滤能力、更新频率、一致性、延迟和运维条件选择。已有 PostgreSQL 且规模合适时，可评估向量扩展；搜索与关键词过滤需求强时，可评估搜索引擎的向量能力；需要专用向量服务时，再比较其索引、分片和混合检索能力。选型要用同一数据与查询集测试 Recall@K、P95、写入/删除传播和资源成本。保存 doc_id、chunk_id、权限、版本和来源；向量索引通常可重建，不能替代原文真值库。

[Q382 · 八股文 · 通用知识] 依据：参考资料：S2

19. 记忆只是不断追加吗，怎样更新和删除?

参考答案：事件纠错保留事件 ID、递增版本并保存修订；长期事实另有结构化主题、值、极性、来源、有效期和变更记录。明确改口的普通偏好可替代旧值，临时偏好不静默覆盖长期信息；冲突可让用户选择。过敏和专业限制到期进入待核对，继续保留保护，不因低使用频率遗忘。用户确认只是明确陈述，转述专业建议不等于独立核验；旧自由文本不自动猜成可靠结构。

[Q383 · 工程现状与边界] 依据：backend/app/health/fact_resolution.py; backend/app/health/facts.py; backend/app/contracts/facts.py; backend/tests/test_structured_facts.py

20. 新旧记忆发生冲突怎么办?

参考答案：事件纠错保留事件 ID、递增版本并保存修订；长期事实另有结构化主题、值、极性、来源、有效期和变更记录。明确改口的普通偏好可替代旧值，临时偏好不静默覆盖长期信息；冲突可让用户选择。过敏和专业限制到期进入待核对，继续保留保护，不因低使用频率遗忘。用户确认只是明确陈述，转述专业建议不等于独立核验；旧自由文本不自动猜成可靠结构。

[Q384 · 工程现状与边界] 依据：backend/app/health/fact_resolution.py; backend/app/health/facts.py; backend/app/contracts/facts.py; backend/tests/test_structured_facts.py

21. 低价值记忆怎样遗忘?

参考答案：保留期与隐私删除是两套机制。待补草稿和待确认计划默认闲置 7 天过期，完成流程上下文默认 30 天清理，同时留下最小请求凭据避免旧请求重写；一年后完整过期月份归档为摘要。用户删除则按实际数据血缘预览并清理相关副本、派生结果与 checkpoint，失败可恢复。长期事实有有效期和冲突规则，过敏等到期仍保护并提示核对，不能因少用自动移除。

[Q385 · 工程现状与边界] 依据：backend/app/common/config.py; backend/app/health/retention.py; backend/app/privacy/deletion.py; backend/tests/test_privacy_deletion.py

22. 追问：如果存进去的是错误记忆，怎样纠正?

参考答案：事件纠错保留事件 ID、递增版本并保存修订；长期事实另有结构化主题、值、极性、来源、有效期和变更记录。明确改口的普通偏好可替代旧值，临时偏好不静默覆盖长期信息；冲突可让用户选择。过敏和专业限制到期进入待核对，继续保留保护，不因低使用频率遗忘。用户确认只是明确陈述，转述专业建议不等于独立核验；旧自由文本不自动猜成可靠结构。

[Q386 · 工程现状与边界] 依据：backend/app/health/fact_resolution.py; backend/app/health/facts.py; backend/app/contracts/facts.py; backend/tests/test_structured_facts.py

23. 工作记忆、短期记忆和长期记忆如何区分?

参考答案: 记忆分四层:工作记忆保存当前 State、草稿、等待点和恢复凭据;事件账本保存健康事件及修订;结构化长期事实保存偏好、过敏、专业限制等及其来源和有效期;派生层保存规律、复盘、计划与归档摘要。派生结果记录窗口、方法版本和来源版本,不能当作原始事实。工作记忆可到期清理,已保存事件按独立保留策略管理;当前不依赖向量记忆或自动对话摘要。

[Q387 · 工程现状与边界] 依据:backend/app/workflows/state.py; backend/app/health/facts.py; backend/app/health/derived.py

24. 多 Agent 相互交接时,会不会出现循环调用?

参考答案: Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果,Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema;结果返回状态、证据及产物引用。Worker 可预声明,也可受控动态创建,动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并,副作用用幂等键;检测循环委派并传播取消,避免所有 Worker 同时修改同一业务对象。

[Q388 · 八股文 · 通用知识] 依据:参考资料:S1,S6

25. Handoff 应传哪些结构化信息?

参考答案: Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果,Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema;结果返回状态、证据及产物引用。Worker 可预声明,也可受控动态创建,动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并,副作用用幂等键;检测循环委派并传播取消,避免所有 Worker 同时修改同一业务对象。

[Q389 · 八股文 · 通用知识] 依据:参考资料:S1,S6

26. 进程重启后,Checkpoint 如何恢复?

参考答案: Engine 启动时打开 checkpoints.db 并初始化 SqliteSaver。追问草稿、待补字段和节点进度以 thread_id 保存;恢复接口核对工作流和 interrupt_id,再用同一 thread_id 执行 Command(resume)。失败的初始请求可以用相同 request_id 重试,已有 checkpoint 时从图状态继续。健康事实以 health.db 为准,两库提交不是同一原子事务,所以仍需要业务幂等。

[Q390 · 工程现状] 依据:backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

27. 追问:父任务和子任务存在依赖时,怎样恢复它们的关系和执行顺序?

参考答案: Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果,Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema;结果返回状态、证据及产物引用。Worker 可预声明,也可受控动态创建,动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并,副作用用幂等键;检测循环委派并传播取消,避免所有 Worker 同时修改同一业务对象。

[Q391 · 八股文 · 通用知识] 依据:参考资料:S1,S6

28. 工具已经产生副作用，恢复后如何避免重复操作?

参考答案：轻记把一次事件写入的操作键固定为 thread_id:event:事件下标，Repository 在事务内先查 operations，再写 events、revisions 和操作结果；重放时返回旧结果。HTTP 层另用 request_id 和请求指纹识别重试。它覆盖当前本地数据库副作用；若以后调用外部支付或推送服务，还必须由外部服务接受幂等键，checkpoint 本身不提供跨系统恰好一次。

[Q392 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

29. 完整 RAG 链路如何设计?

参考答案：RAG 分离线入库和在线问答。入库先解析与清洗文档，保留结构、页码、来源和权限，再切分、生成向量并写入检索索引；文档更新与删除需同步处理版本。在线先理解问题并按需改写，执行权限过滤、关键词/向量召回、融合、重排与去重，再按 Token 预算组装证据，生成带引用的答案。证据不足时澄清或拒答。评测要把解析、召回、排序、引用和答案分别统计，便于定位故障。

[Q393 · 八股文 · 通用知识] 依据：参考资料：S2,S11

30. 如果记忆规模很小，还有必要使用向量库和 Reranker 吗?

参考答案：向量存储按数据量、过滤能力、更新频率、一致性、延迟和运维条件选择。已有 PostgreSQL 且规模合适时，可评估向量扩展；搜索与关键词过滤需求强时，可评估搜索引擎的向量能力；需要专用向量服务时，再比较其索引、分片和混合检索能力。选型要用同一数据与查询集测试 Recall@K、P95、写入/删除传播和资源成本。保存 doc_id、chunk_id、权限、版本和来源；向量索引通常可重建，不能替代原文真值库。

[Q394 · 八股文 · 通用知识] 依据：参考资料：S2

31. 怎样看待 grep、glob、PageIndex 等不同检索方式?

参考答案：向量存储按数据量、过滤能力、更新频率、一致性、延迟和运维条件选择。已有 PostgreSQL 且规模合适时，可评估向量扩展；搜索与关键词过滤需求强时，可评估搜索引擎的向量能力；需要专用向量服务时，再比较其索引、分片和混合检索能力。选型要用同一数据与查询集测试 Recall@K、P95、写入/删除传播和资源成本。保存 doc_id、chunk_id、权限、版本和来源；向量索引通常可重建，不能替代原文真值库。

[Q395 · 八股文 · 通用知识] 依据：参考资料：S2

32. 网页内容中带有 Prompt Injection，怎么办?

参考答案：把用户文本和模型输出视为待验证数据。系统不授予任意 SQL 或命令执行权，模型计划只能触发本地批准的固定视图与业务动作，外发按用途、资源、Schema 和一次性授权检查。输出策略只接受已批准的 point_id/variant_id，数字、日期、保存状态和追问由本地生成；未知标识或自由正文失败回退本地表达。输入安全路由与写入校验也在恢复路径执行。提示词只是辅助，执行端约束才是边界。

[Q396 · 工程现状与边界] 依据：backend/app/modeling/boundary.py; backend/app/modeling/payloads.py; backend/app/policy/output.py; backend/app/workflows/safety.py; backend/tests/test_output_policy.py

33. 信任边界、最小权限和人工确认应如何配合?

参考答案：把用户文本和模型输出视为待验证数据。系统不授予任意 SQL 或命令执行权，模型计划只能触发本地批准的固定视图与业务动作，外发按用途、资源、Schema 和一次性授权检查。输出策略只接受已批准的 point_id/variant_id，数字、日期、保存状态和追问由本地生成；未知标识或自由正文失败回退本地表达。输入安全路由与写入校验也在恢复路径执行。提示词只是辅助，执行端约束才是边界。

[Q397 · 工程现状与边界] 依据：backend/app/modeling/boundary.py; backend/app/modeling/payloads.py; backend/app/policy/output.py; backend/app/workflows/safety.py; backend/tests/test_output_policy.py

34. Thread.sleep(0) 有什么作用?

参考答案：Thread.sleep(0) 指定零毫秒睡眠，不保证实际暂停，也不保证另一个线程获得执行机会，具体效果受 JVM 和系统调度影响。它不会释放当前持有的 monitor 锁，也不建立线程间的同步或可见性保证，不能用来修复竞态。Thread.yield() 同样只是可被调度器忽略的让步提示；Object.wait() 则在满足监视器要求时释放对应 monitor 并等待。等待条件应优先用锁、Condition、Latch 等同步机制，而不是 sleep(0) 自旋。

[Q398 · 八股文 · 通用知识] 依据：参考资料：S12

35. 【算法题】 给一个含重复元素的数组，重新排列，使相邻元素不相同。

参考答案：要重排数组使相邻元素不同，先统计频次；若最大频次大于 ceil(n/2)，则无解。可用最大堆，每次取频次最高且不同于上一项的值，输出后暂存上一项，下一轮再放回剩余次数，避免立刻重复。时间 O(n log k)，空间 O(k)，k 为不同值数量。还可按频次先填偶数位置再填奇数位置。必须保留原多重集合并验证相邻约束，空数组与单元素单独处理。代码见附录 E。

[Q399 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

36. 【算法题•追问】 什么情况下无解，如何判断?

参考答案：设数组长度 n、最高出现次数 f，存在相邻不同排列的充要条件是 f <= ceil(n/2)。必要性来自其他 n-f 个元素最多形成 n-f+1 个间隔，必须能分隔 f 个相同值；充分性可由按频率填偶数再填奇数位置的构造说明。空数组和单元素均可行。判断只需统计频次，时间 O(n)、空间 O(k)；若还要求输出一种排列，可用最大堆构造。

[Q400 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

37. 平时有什么兴趣，最近读了什么书?

参考答案：选择真实、熟悉、能继续讨论的内容。说明最近阅读的【书/源码/论文】、关注的问题、得到的结论和适用限制 。兴趣部分只需说明一项长期坚持的活动。

[Q401 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

七、腾讯

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q402 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 项目有哪些模块，一次请求的执行流程是什么?

参考答案：请求进入 POST /api/chat 后先校验参数和 request_id 指纹，创建或恢复 thread_id。图依次解析、检查草稿、准备营养，完整独立事件可先保存；缺字段则 interrupt。恢复后仅合并待补字段，最终保存、组装卡片与证据，将状态、耗时和消息落库。模型失败、取消和等待用户分别有不同结果，HTTP 返回并不必然代表新事件已保存。

[Q403 · 工程现状] 依据：backend/app/main.py

3. 系统怎样部署，最终由谁使用?

参考答案：当前入口是 Taro/React/TypeScript 微信小程序，连接独立 FastAPI JSON API；旧 Web 前端已废弃，根路径不再托管页面。Engine 复用聊天、计划、分析图，health.db 保存业务事实，checkpoints.db 保存图进度。开发启动用 start.ps1；仓库另有 Docker 加固部署配置。后台任务与 Web 仍共享唯一 Engine，数据目录所有者锁阻止第二实例，不是独立 Worker 集群。正式对外服务仍需 HTTPS、鉴权及相应部署验收。

[Q404 · 工程现状与边界] 依据：miniprogram/package.json; backend/app/main.py; backend/app/infrastructure/data_owner.py; README.md; compose.hardened.yaml; start.ps1

4. 上下文预算如何分配?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q405 · 八股文 · 通用知识] 依据：参考资料：S1,S6

5. 历史消息和工具结果分别如何压缩?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q406 · 八股文 · 通用知识] 依据：参考资料：S1,S6

6. 压缩丢掉的信息后续又需要了，怎么找回来?

参考答案：分三种：本轮未选入提示词的健康事件仍在 SQLite，可按日期或 ID 再查，但不会自动全文回填；被 30 天清理掉的消息/checkpoint 无法恢复，已保存事件通常还在；一年月度归档删除逐餐原文后，只能读摘要，细节回不来。工程没有摘要自动反查原文的通用工具，不能说所有压缩信息都可恢复。

[Q407 · 工程现状] 依据：backend/app/health/advice.py

7. 在什么时候触发压缩？同步做还是异步做?

参考答案：先按模型上下文窗口扣除输出预留及安全余量，再给系统规则、当前任务、近期消息、证据和工具结果分配预算。接近阈值时先去重和删除低价值日志，再将旧对话或长结果做结构化摘要；阈值依据模型和任务调优。必须保留目标、硬约束、已确认事实、未完成项、引用、工具副作用和授权状态。完整结果放外部存储，摘要带可回读引用；压缩后用回归任务验证约束与证据是否丢失。

[Q408 · 八股文 · 通用知识] 依据：参考资料：S1,S6

8. 是否支持多模态输入，整体链路如何处理?

参考答案：解析先保留文档结构与来源，再做统一索引。PDF/扫描件需要版面分析、OCR 和阅读顺序；Word 按标题、段落及表格；PPT 保留页、标题和图文关系；Excel 保留工作表、表头、单位及行列坐标；图片可用 OCR 和视觉描述，必要时保留可回看的图像引用。表格与图片描述应绑定原页，避免单独文本失去条件。解析质量会限制召回上限，应检查缺页、乱码、表格错列及来源定位。

[Q409 · 八股文 · 通用知识] 依据：参考资料：S2

9. 视觉模型与文本模型什么时候切换，由谁决定?

参考答案：定义统一 Chat/Embedding/Rerank 接口，把消息、工具参数格式、流式事件、错误码和用量归一化。适配器处理各 Provider 差异。网关按能力、数据边界、延迟、成本和健康度路由，配合限流、熔断、降级与成本标签。模型选择应版本化并可灰度回滚。

[Q410 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

10. PDF、Word 等文档怎么处理?

参考答案：解析先保留文档结构与来源，再做统一索引。PDF/扫描件需要版面分析、OCR 和阅读顺序；Word 按标题、段落及表格；PPT 保留页、标题和图文关系；Excel 保留工作表、表头、单位及行列坐标；图片可用 OCR 和视觉描述，必要时保留可回看的图像引用。表格与图片描述应绑定原页，避免单独文本失去条件。解析质量会限制召回上限，应检查缺页、乱码、表格错列及来源定位。

[Q411 · 八股文 · 通用知识] 依据：参考资料：S2

11. 历史数据与实时信息如何结合，更新怎样生效?

参考答案：轻记的实时数据来自用户新增或纠错后的 SQLite 记录，data_version 递增让旧规律失效；咨询每次从库重建上下文。计划草案确认时核对数据版本与基准计划，过时就拒绝激活。当前没有实时设备流或外部知识增量索引，历史月度摘要也不能用于精确的逐日复算。

[Q412 · 工程现状] 依据：backend/app/infrastructure/repositories/repository.py

12. 用户手工编排 Workflow，与自然语言生成计划有什么区别?

参考答案：轻记网页让用户编辑目标和确认计划，不提供流程画布。自然语言主要生成记录草稿，plan_draft 用本地规则生成目标差异与七天安排；它不是模型生成任意执行 DSL。将来人工编排强调明确节点和边，自然语言计划需要多一层受限结构校验与用户确认，两者最终仍应执行同样的门禁。

[Q413 · 工程现状] 依据：miniprogram/src/app.tsx; backend/app/health/analysis.py

13. DSL 如何转换为执行图?

参考答案：DSL 应声明版本、节点 ID/类型、参数、类型化输入输出、边、条件、变量引用、超时与错误处理。编译流程是解析 Schema、检查节点和端口、校验引用及类型、分析可达性与依赖、验证循环边界，再生成执行图。自然语言先生成受限中间计划，再编译为 DSL，避免模型直接拼接任意代码。执行前做静态检查与沙箱测试，运行时仍检查权限和写入条件；静态合法不等于业务正确。

[Q414 · 八股文 · 通用知识] 依据：参考资料：S6,S9

14. 视频生成等耗时较长的工具，怎样放进 Agent 执行流程?

参考答案：长工具调用可提交外部任务并返回 job_id，运行时持久保存 job_id 到 task_id/step_id、等待点和步骤版本的映射，标记 WAITING_EXTERNAL 后释放执行资源。外部结果通过认证回调或轮询到达，先校验签名、幂等事件 ID、任务归属和版本，再抢占恢复权继续执行。重复或过期结果不能推进旧流程，超时进入重试、取消或人工处理。通知丢失时可扫描等待任务并对账，不能靠最近一条聊天猜恢复位置。

[Q415 · 八股文 · 通用知识] 依据：参考资料：S6,S9

15. 计划保存在哪里？步骤失败、中断后怎样继续?

参考答案：执行计划应持久保存为带版本的步骤 DAG，每步有 ID、依赖、能力版本、输入引用、状态、attempt、结果引用和错误。运行时从已提交边界找出依赖已满足的未完成步骤，领取执行权后恢复；等待人工或外部任务使用明确等待状态。成功副作用先查幂等结果，不重复执行；部分失败按策略重试、补偿或重规划。计划更新创建新版本，保留完成步骤的兼容结果，对旧回调和旧 Worker 做版本及 fencing 校验。

[Q416 · 八股文 · 通用知识] 依据：参考资料：S6,S8

16. 异步结果通过轮询还是回调拿到?

参考答案：长工具调用可提交外部任务并返回 job_id，运行时持久保存 job_id 到 task_id/step_id、等待点和步骤版本的映射，标记 WAITING_EXTERNAL 后释放执行资源。外部结果通过认证回调或轮询到达，先校验签名、幂等事件 ID、任务归属和版本，再抢占恢复权继续执行。重复或过期结果不能推进旧流程，超时进入重试、取消或人工处理。通知丢失时可扫描等待任务并对账，不能靠最近一条聊天猜恢复位置。

[Q417 · 八股文 · 通用知识] 依据：参考资料：S6,S9

17. 追问：拿到结果后，怎样恢复原来的任务?

参考答案：长工具调用可提交外部任务并返回 job_id，运行时持久保存 job_id 到 task_id/step_id、等待点和步骤版本的映射，标记 WAITING_EXTERNAL 后释放执行资源。外部结果通过认证回调或轮询到达，先校验签名、幂等事件 ID、任务归属和版本，再抢占恢复权继续执行。重复或过期结果不能推进旧流程，超时进入重试、取消或人工处理。通知丢失时可扫描等待任务并对账，不能靠最近一条聊天猜恢复位置。

[Q418 · 八股文 · 通用知识] 依据：参考资料：S6,S9

18. Human-in-the-loop 在什么时候介入?

参考答案：人工介入点是 clarify 与 approve_plan：前者补必需字段或消除纠错歧义，后者确认首份计划或超过 5% 的单次/累计目标变化。interrupt 返回问题并保存等待状态；恢复接口校验 thread_id、interrupt_id 和输入类型，确认时再次检查版本。尚未实现所有高风险外部动作的通用审批系统。

[Q419 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

19. 用户补充输入、修改方向和等待确认，分别怎么处理?

参考答案：补充回答通过 merge_patch 只修改待补字段，原日期和已知食物保留；用户可取消未完成草稿，已保存独立记录保留；新独立记录走新 thread。计划等待时只能明确接受或拒绝，修改方向需要重新提出草案；当前没有任意重规划编辑器。这样能避免一句补充意外改掉另一餐或正在生效目标。

[Q420 · 工程现状] 依据：backend/app/chat/drafts.py

20. 恢复执行时，如何确定输入对应哪个任务、哪个等待点?

参考答案：长工具调用可提交外部任务并返回 job_id，运行时持久保存 job_id 到 task_id/step_id、等待点和步骤版本的映射，标记 WAITING_EXTERNAL 后释放执行资源。外部结果通过认证回调或轮询到达，先校验签名、幂等事件 ID、任务归属和版本，再抢占恢复权继续执行。重复或过期结果不能推进旧流程，超时进入重试、取消或人工处理。通知丢失时可扫描等待任务并对账，不能靠最近一条聊天猜恢复位置。

[Q421 · 八股文 · 通用知识] 依据：参考资料：S6,S9

21. RAG 链路如何设计?

参考答案：RAG 分离线入库和在线问答。入库先解析与清洗文档，保留结构、页码、来源和权限，再切分、生成向量并写入检索索引；文档更新与删除需同步处理版本。在线先理解问题并按需改写，执行权限过滤、关键词/向量召回、融合、重排与去重，再按 Token 预算组装证据，生成带引用的答案。证据不足时澄清或拒答。评测要把解析、召回、排序、引用和答案分别统计，便于定位故障。

[Q422 · 八股文 · 通用知识] 依据：参考资料：S2,S11

22. Excel 或大型表格怎样解析和检索?

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走 OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用 SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

[Q423 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

23. 【算法题】 二叉树的层序遍历。题型来自面试后的补充回忆，完整题面及作答过程未能确认。

参考答案：二叉树层序遍历用队列。根非空时入队，每轮先记录当前队列长度，只处理这一轮的节点，依次将左右孩子入队；这一轮的值构成一层。每个节点入队出队一次，时间 O(n)，辅助空间为最大层宽 O(w)，最坏 O(n)。空树返回空列表；不要在遍历中用不断增长的队列长度当本层边界。若需锯齿层序，可按层号调整输出方向。代码见附录 F。

[Q424 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

八、蚂蚁

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q425 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 如果参与过 Java 微服务项目，主要负责哪些模块?

参考答案：按真实经历说明服务拆分、接口设计、数据库、缓存、队列、并发、监控和部署中实际负责的部分。只描述亲自做过的模块，并说明规模、故障处理和结果。

[Q426 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

3. 以实时通信系统为例，底层采用 SIP、VoIP 还是其他协议，应如何选择?

参考答案：协议选择看终端兼容、媒体能力、NAT 穿透、合规和运维成本。SIP 负责会话信令，RTP/相关协议承载媒体。底层维护连接与协议状态，上层维护业务会话和坐席分配。事件带会话 ID、序号与时间戳，经幂等消费和对账修正状态。

[Q427 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

4. 通信控制层与上层业务服务分别负责什么?

参考答案：协议选择看终端兼容、媒体能力、NAT 穿透、合规和运维成本。SIP 负责会话信令，RTP/相关协议承载媒体。底层维护连接与协议状态，上层维护业务会话和坐席分配。事件带会话 ID、序号与时间戳，经幂等消费和对账修正状态。

[Q428 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

5. 系统的坐席和并发规模如何?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q429 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

6. 连接状态由底层通信组件还是上层调度服务维护?

参考答案：底层组件维护网络连接和协议状态，上层服务维护业务会话、坐席分配和任务状态。两层通过带序号的状态事件同步。上层不能凭接口返回猜测连接状态，底层也不应直接决定业务流程。

[Q430 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

7. 怎样保证通话状态同步和准确?

参考答案：协议选择看终端兼容、媒体能力、NAT 穿透、合规和运维成本。SIP 负责会话信令，RTP/相关协议承载媒体。底层维护连接与协议状态，上层维护业务会话和坐席分配。事件带会话 ID、序号与时间戳，经幂等消费和对账修正状态。

[Q431 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

8. 状态事件通过消息队列还是回调传递?

参考答案：跨服务、需要重放和削峰的状态事件更适合消息队列。低延迟且调用关系简单的通知可用回调。关键状态仍应落库，队列或回调只负责传递事件，并通过事件编号、幂等消费和对账保证一致。

[Q432 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

9. 如果参与过数据处理平台，平台解决什么问题，个人负责哪部分?

参考答案：建议答法：平台面向【内部开发者/企业客户】，把模型、知识、工具和流程统一成可发布、可治理的 AI 应用。 核心解决重复接入、权限审计、长任务可靠性和效果回归。交付物应明确为产品实例、配置或扩展包、接口与运维指标，不能只说“交付一个 Agent”。

[Q433 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

10. 以设备数据为例，怎样设计上报和存储链路?

参考答案：采集链路应有设备身份、序列号/事件时间、消息队列、幂等落库和数据质量告警。原始数据不可变，指标定义带版本。窗口按业务语义选择滚动/滑动并处理乱序水位。重算从原始数据按版本回放。Redis 可做低延迟临时状态，但不宜作为唯一的可审计历史存储。

[Q434 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

11. 使用什么时序数据库?

参考答案：时序存储按写入速率、指标基数、保留周期、查询模式和运维条件选择。监控指标常用专门指标存储；设备事件可选时序数据库或按时间分区的关系库；分析聚合可评估列式存储。模式通常含时间戳、实体/标签、指标值与质量标记，避免不受控高基数标签。结合分区、压缩、降采样和保留策略降低成本，热点查询可缓存；原始事件身份与版本需保留，便于迟到、纠错和重算。

[Q435 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

12. 如果数据上报丢失，怎样发现和定位?

参考答案：先定义上报链路和可追踪身份：设备/事件 ID、递增序号、发生时间、发送批次和接收回执。监控预期采样与实收数量、序号缺口、重复率、延迟和积压，分别核查设备缓存、网络重试、入口、队列、处理与数据库。通过端到端事件 ID 对账，丢失可请求补传，消费端幂等去重；未知缺失不能当作真实零值。补数保留原发生时间，并触发对应窗口重算。

[Q436 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

13. 用户怎样配置指标计算逻辑?

参考答案：指标定义应采用受限 DSL 或配置，包含名称、输入事件、过滤条件、维度、窗口、聚合、单位与版本，先做类型和资源校验，避免直接运行任意表达式。结果绑定定义版本、输入快照和数据截止时间；新增、纠错或规则升级产生可幂等的重算任务。按依赖确定受影响范围，新旧版本并跑校验后切换，失败保留旧结果并标记过期。原始数据不足时明确不可重算，不能承诺从摘要还原全部细节。

[Q437 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

14. 流式数据的时间窗口如何实现?

参考答案：流处理按事件时间或处理时间分配窗口。滚动窗口互不重叠，滑动窗口按步长重复覆盖，会话窗口按无事件间隔分组。事件时间适合乱序数据，需水位线估计进度，配合允许迟到、触发器和状态清理；迟到数据可更新结果或进入侧输出。滑动窗口能连续观察变化，但状态与计算成本更高，可用增量聚合或小窗口复用。需明确时区、边界、去重、补数及更新/撤回语义，不能只用定时 SQL 等同完整流引擎。

[Q438 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

15. 追问：为什么没有采用滑动窗口?

参考答案：流处理按事件时间或处理时间分配窗口。滚动窗口互不重叠，滑动窗口按步长重复覆盖，会话窗口按无事件间隔分组。事件时间适合乱序数据，需水位线估计进度，配合允许迟到、触发器和状态清理；迟到数据可更新结果或进入侧输出。滑动窗口能连续观察变化，但状态与计算成本更高，可用增量聚合或小窗口复用。需明确时区、边界、去重、补数及更新/撤回语义，不能只用定时 SQL 等同完整流引擎。

[Q439 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

16. 为什么需要中间汇总计算层?

参考答案：中间汇总把事件转成每日 totals、体重、睡眠、运动和完整性标记，供页面、建议和复盘复用，避免模型分别算出不一致数字。当前 dashboard 按需计算，没有独立物化汇总集群。每项营养未知时保留 None，只有满足完整性条件才算均值或剩余额度，这比单纯优化查询更重要。

[Q440 · 工程现状] 依据：backend/app/health/analysis.py

17. 指标定义发生变化后，历史数据怎么重算?

参考答案：当前新增或纠错会递增 data_version 并把 active insights 标 stale，后续分析重新计算；结果保存 rule_version、data_version，业务计划另核对基准版本。若改指标定义，应保存不可变规则版本和受影响日期范围，重算时验证统计口径，不能直接覆盖历史结论。归档后无原始细节，不能保证所有新指标都能追溯重算。

[Q441 · 工程现状] 依据：backend/app/infrastructure/repositories/repository.py

18. 追问：如果重新设计指标版本与重算机制，会怎样改进?

参考答案：指标定义应采用受限 DSL 或配置，包含名称、输入事件、过滤条件、维度、窗口、聚合、单位与版本，先做类型和资源校验，避免直接运行任意表达式。结果绑定定义版本、输入快照和数据截止时间；新增、纠错或规则升级产生可幂等的重算任务。按依赖确定受影响范围，新旧版本并跑校验后切换，失败保留旧结果并标记过期。原始数据不足时明确不可重算，不能承诺从摘要还原全部细节。

[Q442 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

19. 是否考虑过使用 Redis 承载中间计算?

参考答案：时序存储按写入速率、指标基数、保留周期、查询模式和运维条件选择。监控指标常用专门指标存储；设备事件可选时序数据库或按时间分区的关系库；分析聚合可评估列式存储。模式通常含时间戳、实体/标签、指标值与质量标记，避免不受控高基数标签。结合分区、压缩、降采样和保留策略降低成本，热点查询可缓存；原始事件身份与版本需保留，便于迟到、纠错和重算。

[Q443 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

20. 为什么选择 LangGraph?

参考答案：轻记需要两类等待：记录缺字段时等用户补充，计划变化较大时等用户确认。因此选择 LangGraph 的显式状态图、interrupt 和 SQLite checkpoint。图负责编排，草稿完整性、营养来源、请求去重和计划版本检查由自己的业务代码实现。当前没有统一任务基准证明它优于其他框架；选择主要依据是与这两类暂停恢复需求匹配。

[Q444 · 工程现状] 依据：backend/app/workflows/engine.py; backend/tests/test_agent.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

21. LangGraph 在平台中具体承担什么职责?

参考答案：当前 LangGraph 承担 StateGraph、条件路由、interrupt 和 checkpoint；Engine 另外管理请求指纹、回执、锁和 API 结果。营养准备、历史检索、计划版本、统计、归档都在普通 Python 模块中。边界仍有耦合：Engine 把图和业务函数连在一起，还不是可插拔通用 Runtime，但更换编排层不必重写所有健康算法。

[Q445 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

22. 为什么不用封装程度更高的 Agent 框架?

参考答案：封装越高，上手越快，但状态、恢复、工具执行和观测更难按业务要求调整。需要强权限、私有化、长任务和领域流程时，选择较低层框架更容易控制边界。前提是团队愿意承担更多工程建设和维护成本。

[Q446 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

23. 从文件上传到问答，知识库的完整处理链路是什么?

参考答案：RAG 分离线入库和在线问答。入库先解析与清洗文档，保留结构、页码、来源和权限，再切分、生成向量并写入检索索引；文档更新与删除需同步处理版本。在线先理解问题并按需改写，执行权限过滤、关键词/向量召回、融合、重排与去重，再按 Token 预算组装证据，生成带引用的答案。证据不足时澄清或拒答。评测要把解析、召回、排序、引用和答案分别统计，便于定位故障。

[Q447 · 八股文 · 通用知识] 依据：参考资料：S2,S11

24. 超大型文件或十几万行的 Excel 如何处理?

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走 OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用 SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

[Q448 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

25. 关联信息在文档中相距很远，怎样保证一起召回?

参考答案：常见切分有固定长度加重叠、递归分隔符、标题层级、语义切分和父子块。固定切分简单稳定；结构切分适合章节明确文档；语义切分尝试保留主题完整性但成本较高。表格保留表头与单位，代码按函数/类及依赖处理。检索可用小块定位，再扩展相邻块或父章节，处理跨块证据。每块保存来源、位置、层级、版本和权限，重叠过多会增加重复与索引成本。

[Q449 · 八股文 · 通用知识] 依据：参考资料：S2

26. Metadata Filter 在检索中怎么使用?

参考答案：每个片段至少保留 document_id、chunk_id、来源 URI、标题路径、页码或行号、版本、更新时间和访问控制元数据；还可加入语言、类型及父子关系。元数据支持检索过滤、引用定位、更新删除和故障回放。权限过滤必须来自服务端身份，不能让模型选择任意租户。文档更新时先构建新版本并验证，再切换可见版本，清理旧索引；保留版本能解释当时答案依据。

[Q450 · 八股文 · 通用知识] 依据：参考资料：S2

27. PPL 语义边界检测如何实现?

参考答案：PPL 是语言模型对文本预测难度的度量，通常为平均负对数似然的指数。基于 PPL 的切分利用相邻句或局部窗口的预测难度变化，寻找可能的主题边界；不同实现的上下文、打分与边界规则并不统一。可对候选句边界计算条件负对数似然，平滑后取局部峰值，同时施加最小/最大块长及结构规则。分数高也可能来自术语、代码或 OCR 错误，不能直接等同语义断点。需与结构切分对照评测。

[Q451 · 八股文 · 通用知识] 依据：参考资料：S2

28. 追问：这种切分方案有哪些问题?

参考答案：PPL 是语言模型对文本预测难度的度量，通常为平均负对数似然的指数。基于 PPL 的切分利用相邻句或局部窗口的预测难度变化，寻找可能的主题边界；不同实现的上下文、打分与边界规则并不统一。可对候选句边界计算条件负对数似然，平滑后取局部峰值，同时施加最小/最大块长及结构规则。分数高也可能来自术语、代码或 OCR 错误，不能直接等同语义断点。需与结构切分对照评测。

[Q452 · 八股文 · 通用知识] 依据：参考资料：S2

29. 如果 Recall@5 突然大幅下降，怎么排查?

参考答案：召回下降先固定失败查询与历史基线，检查数据入库、解析/OCR、切分、索引数量和版本是否变化，再检查过滤条件、权限、查询改写、Embedding 版本、向量归一化与距离度量、ANN 搜索参数。区分“正确证据不在索引”“在索引但未召回”“召回后被过滤或重排丢失”。用少量查询做精确搜索或人工定位对照，逐层回放定位原因，再调整参数；不要一开始就扩大 TopK 掩盖问题。

[Q453 · 八股文 · 通用知识] 依据：参考资料：S2

30. 追问：用户、场景和配置都没有变化，应优先怀疑什么?

参考答案：召回下降先固定失败查询与历史基线，检查数据入库、解析/OCR、切分、索引数量和版本是否变化，再检查过滤条件、权限、查询改写、Embedding 版本、向量归一化与距离度量、ANN 搜索参数。区分“正确证据不在索引”“在索引但未召回”“召回后被过滤或重排丢失”。用少量查询做精确搜索或人工定位对照，逐层回放定位原因，再调整参数；不要一开始就扩大 TopK 掩盖问题。

[Q454 · 八股文 · 通用知识] 依据：参考资料：S2

31. Embedding 是否“幂等”，同一输入能否得到稳定输出?

参考答案：Embedding 是否稳定要固定输入字节、分词/归一化、模型版本、推理配置和数值精度。即使输出有很小数值误差，也应比较余弦相似度与最近邻排名，不能只做逐浮点完全相等判断。模型版本或维度变化时，应重建匹配索引，避免新旧向量混用。向量稳定不等于检索稳定，ANN 参数、过滤与索引更新也会影响结果。测试同时覆盖重复调用、语义近邻、远离样本和业务 Recall。

[Q455 · 八股文 · 通用知识] 依据：参考资料：S2

32. 追问：怎样验证输出的稳定性?

参考答案：Embedding 是否稳定要固定输入字节、分词/归一化、模型版本、推理配置和数值精度。即使输出有很小数值误差，也应比较余弦相似度与最近邻排名，不能只做逐浮点完全相等判断。模型版本或维度变化时，应重建匹配索引，避免新旧向量混用。向量稳定不等于检索稳定，ANN 参数、过滤与索引更新也会影响结果。测试同时覆盖重复调用、语义近邻、远离样本和业务 Recall。

[Q456 · 八股文 · 通用知识] 依据：参考资料：S2

33. 使用什么 Reranker?

参考答案：Reranker 对召回后的少量 query-document 对做更精细的相关性评分。常见 Cross-Encoder 联合编码问题和文档，效果通常比独立向量相似度更细，但计算量更高，因此放在召回之后。限制候选数和文本长度，批处理并设超时；失效时可回退融合排序。评测看 NDCG、MRR、必要证据覆盖与端到端延迟。重排不能找到召回阶段完全遗漏的文档，也不能承担权限授权。

[Q457 · 八股文 · 通用知识] 依据：参考资料：S2,S11

34. Reranker 的输出是否稳定?

参考答案：Reranker 对召回后的少量 query-document 对做更精细的相关性评分。常见 Cross-Encoder 联合编码问题和文档，效果通常比独立向量相似度更细，但计算量更高，因此放在召回之后。限制候选数和文本长度，批处理并设超时；失效时可回退融合排序。评测看 NDCG、MRR、必要证据覆盖与端到端延迟。重排不能找到召回阶段完全遗漏的文档，也不能承担权限授权。

[Q458 · 八股文 · 通用知识] 依据：参考资料：S2,S11

35. 追问：如何保证模型排序结果符合格式和行为约束?

参考答案：同一模型和参数通常近似稳定，但服务版本、硬件、浮点计算、批处理和预处理变化都可能造成漂移。固定模型版本与归一化流程，重复请求统计向量余弦差/排序一致性，并对 Reranker 做 JSON 参数格式、分数范围、候选 ID 白名单和回退排序校验。

[Q459 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

36. Agent 执行链路如何做到可观测?

参考答案：已有运行元数据白名单、HTTP 关联 ID、节点/模型/数据库/锁/任务分层耗时，以及 Prometheus Counter、Histogram、Gauge。模型传输成功、Schema 成功和业务完成分别计数。业务质量贡献在同一业务事务写入，按保留窗口聚合并参与删除。日志和 span 不采集健康原文、完整 prompt 或模型响应；业务 process_trace 不能直接复制到遥测。指标导出默认关闭，尚无完整集中观测平台或分角色运维权限。

[Q460 · 工程现状与边界] 依据：backend/app/observability/metrics.py; backend/app/observability/spans.py; backend/app/observability/contracts.py; backend/app/api/observability.py; backend/tests/test_observability.py

37. 追问：有了可观测性，怎样进一步做到可评测?

参考答案：每次请求绑定 trace_id/run_id/租户编号，Trace 记录模型、工具、节点和等待时段，Metric 统计成功率、完成率、P50/P95 延迟、Token/成本、重试和队列积压，Log 记录结构化错误但脱敏。可观测解决“发生了什么”，评测再把轨迹与任务质量标签连接起来。

[Q461 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

38. 最终输出与工具调用轨迹分别怎样评价?

参考答案：最终输出看正确性、完整性、忠实性、格式和业务完成度。工具轨迹看必需步骤是否执行、工具和参数是否正确 、是否出现越权或重复副作用。两者分别计分，因为结果正确不代表过程合规，过程合规也不代表结果正确。

[Q462 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

39. 为什么独立建设 Agent Harness?

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、 工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

[Q463 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

40. 通用运行内核、服务入口与业务扩展怎样分工?

参考答案：接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q464 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

41. 如何约束业务实现，避免侵入运行内核?

参考答案：当前通过普通函数和 Repository 分离业务计算与图路由：营养和目标计算不需要直接操作图 API。新增饮食规则应放入业务模块，新增通用请求或恢复语义才改 Engine。这个边界尚未成为正式接口协议；若继续通用化，应抽出服务端口、节点结果契约和状态版本迁移，而不是把所有规则堆进图节点。

[Q465 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

42. Codex 与 Claude Code 有什么区别?

参考答案：比较时应分开看模型能力和 Harness 能力。重点比较工具协议、上下文管理、计划与执行方式、代码与终端权限 、检查点、人工交互、扩展机制和安全边界。不能只根据一次回答效果判断产品差异。

[Q466 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

43. 追问：行为差异主要来自模型，还是 Harness?

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、 工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

[Q467 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

二面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q468 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 业务 Agent Harness 是当前的重点工作吗?

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、 工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

[Q469 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

3. 项目团队的开发人员和其他角色怎样配置?

参考答案：按真实团队说明产品、业务、算法、平台、后端、前端、测试、交付和运维人员数量及职责。重点解释需求、架构、代码、评测、发布和线上支持分别由谁负责。

[Q470 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

4. 主要采用 AI 辅助编码，还是传统人工编码?

参考答案：按真实比例说明。AI 可以用于代码草稿、测试和重构，但接口设计、安全、核心逻辑和最终合并由工程师负责。所有 AI 生成代码都应经过评审、静态检查和测试，不能按生成量衡量贡献。

[Q471 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

5. 你在项目中具体负责什么?

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

[Q472 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

6. 多人同时使用 AI Coding 时，怎样拆分任务、控制协作和集成风险?

参考答案：按稳定接口和业务域拆分模块，使用独立版本、测试数据和运行命名空间。合并前执行接口兼容检查、自动测试 、代码评审和端到端回归。共享内核只接受通过扩展接口的能力，禁止业务模块直接修改全局状态。

[Q473 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

7. 业务扩展模块是否按业务域划分?

参考答案：轻记模块按业务职责划分而不是按 Agent 人设：drafts 管完整性，nutrition 管营养，targets 管目标，advice/replies 管咨询表达，services 管统计计划，repository 管数据。它们都服务同一个健康域；跨行业时才考虑独立领域包。这样的划分有利于单独验证数值、来源和恢复，不必每个模块都变成独立服务。

[Q474 · 工程现状] 依据：backend/app/; README.md

8. 运行内核由多少人维护?

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、 工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

[Q475 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

9. 你在团队中的职级和角色是什么?

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

[Q476 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

10. 不同用工类型的人员怎样分工?

参考答案：按责任和风险分工，不按身份简单分配。核心架构、权限、安全和生产发布由长期负责团队掌握。边界清晰的模块、测试和交付工作可由其他成员承担。所有代码使用同一评审、测试和权限标准。

[Q477 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

11. 运行内核、业务扩展和服务入口是独立 REST 服务，还是装配在同一个服务内?

参考答案：当前入口是 Taro/React/TypeScript 微信小程序，连接独立 FastAPI JSON API；旧 Web 前端已废弃，根路径不再托管页面。Engine 复用聊天、计划、分析图，health.db 保存业务事实，checkpoints.db 保存图进度。开发启动用 start.ps1；仓库另有 Docker 加固部署配置。后台任务与 Web 仍共享唯一 Engine，数据目录所有者锁阻止第二实例，不是独立 Worker 集群。正式对外服务仍需 HTTPS、鉴权及相应部署验收。

[Q478 · 工程现状与边界] 依据：miniprogram/package.json; backend/app/main.py; backend/app/infrastructure/data_owner.py; README.md; compose.hardened.yaml; start.ps1

12. 运行内核从零实现，还是基于开源项目?

参考答案：执行引擎基于 LangGraph，并非从零手写调度器。Engine 启动时构建 chat、plan、analysis 三张 StateGraph，再用同一个 SqliteSaver 按 thread_id 保存运行状态。FastAPI 接受请求，Engine 选择图，普通 Python 模块实现健康业务。自建部分是可靠性和领域规则，不是重新实现 LangGraph。

[Q479 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/main.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

13. LangGraph 在其中承担什么职责?

参考答案：当前 LangGraph 承担 StateGraph、条件路由、interrupt 和 checkpoint；Engine 另外管理请求指纹、回执、锁和 API 结果。营养准备、历史检索、计划版本、统计、归档都在普通 Python 模块中。边界仍有耦合：Engine 把图和业务函数连在一起，还不是可插拔通用 Runtime，但更换编排层不必重写所有健康算法。

[Q480 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

14. 一轮 ReAct 包含哪些节点?

参考答案：最小 ReAct Loop 的步骤是：构建受预算约束的上下文，调用模型，解析 final 或 tool_calls；若为工具调用，先做 Schema、权限及幂等检查，执行后把 Observation 反馈给下一轮，并保存检查点。循环检查取消、截止时间、最大轮数、重复调用和完成条件。Hook 可插在构建上下文前、模型前后、工具前后和任务结束时，用来接入 Memory、Skill、日志及策略。示例伪代码见附录 C。

[Q481 · 八股文 · 通用知识] 依据：参考资料：S1

15. Observation 里记录哪些信息?

参考答案：记录工具名称与版本、调用编号、参数摘要、开始结束时间、状态、结构化结果、错误类型、重试次数、外部副作用、结果引用和可见范围。敏感字段必须脱敏，过长结果只保存引用。

[Q482 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

16. 为什么可以把这套运行内核看作最小 Harness?

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、 工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

[Q483 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

17. 业务扩展模块具体包含什么?

参考答案：当前业务模块包括 drafts 的缺字段检查和增量补丁、nutrition 的营养准备与来源、targets 的默认目标公式、advice/replies 的上下文与表达校验、services 的计划统计归档。运行控制由 Engine 管理，数据由 Repository 保存。它们是普通 Python 模块，并没有通过动态插件清单加载。

[Q484 · 工程现状] 依据：backend/app/chat/drafts.py

18. 追问：它与 Skill 是什么关系?

参考答案：Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q485 · 八股文 · 通用知识] 依据：参考资料：S3,S6

19. MCP、外部业务系统适配和权限约束放在哪一层?

参考答案：MCP 协议与外部系统适配放在能力接入层，负责把外部接口转换为统一工具。身份传递、权限策略和审计由平台治理层定义，并在工具执行层强制校验。业务扩展只声明需要的权限，不能自行绕过。

[Q486 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

20. 描述式 Skill 和 Workflow Skill 有什么区别?

参考答案：轻记的 Workflow 是可执行 Python 图：Engine 启动时把节点函数绑定到 StateGraph，添加固定和条件边，compile 时注入 SqliteSaver。当前没有读取 Markdown 自动执行脚本的 Skill 运行器。若对外包装 Skill，描述文件说明用途和契约，执行入口则调用这张图，返回 completed、waiting_input 或 failed 结果。

[Q487 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

21. 确定性 Workflow 如何挂起并恢复 ReAct?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q488 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

22. 目前经过多少真实客户的验证?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q489 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

23. 为什么考虑换工作?

参考答案：用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

[Q490 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

24. 目前的职级、绩效和薪酬结构如何?

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

[Q491 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

25. 应用平台与 Dify、Coze、Palantir 分别是什么关系?

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

[Q492 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

26. 下层业务数据、逻辑和动作怎样被应用平台使用?

参考答案：数据通过受权限控制的查询接口或 RAG 接入，业务逻辑通过服务或 Workflow 接入，写操作通过有明确参数和审计的工具接入。所有调用携带用户和租户身份，读取与写入分别授权，模型不能直接连接底层数据库。

[Q493 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

27. 平台当前有多少客户和线上用户?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q494 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

28. 如果没有深度业务共建，平台怎样形成超越通用低代码产品的竞争力?

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

[Q495 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

29. 低代码产品本来是为了降低门槛，为什么仍需要研发人员陪同客户落地?

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

[Q496 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

30. 什么情况下需要从画布平台转向 Agent Harness?

参考答案：当任务路径难以预先画出、需要动态选工具、长期运行、持续交互或个人工作空间时，才需要 Agent Harness。规则固定、可视化配置足够的业务继续使用画布 Workflow。两者可以共享工具、权限和发布能力。

[Q497 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

31. 一次客户交付之后，哪些东西能够成为长期资产?

参考答案：这个工程可以沉淀的资产是五类事件 Schema、草稿到严格记录的校验流程、营养来源追踪、request_id/操作键幂等模板、计划差异确认和回归用例。它们比某个用户的饮食原文更适合复用。若交付给其他健康场景，应把目标计算、提醒和统计模板作为领域策略替换，同时保留恢复与验证接口。

[Q498 · 工程现状] 依据：backend/app/api/schemas.py; backend/tests/

32. 原平台的 Workflow 和 Agent 能否迁移到新运行时?

参考答案：迁移先固定旧任务的图、Schema、插件及模型策略版本，暂停新领取或让旧 Worker 排空，并确保任务已到可恢复边界。复制或转换检查点后，通过租约和递增 fencing token 转移执行权，旧 Worker 写入必须被拒绝。按稳定 task_id/幂等键恢复，核对外部副作用，使用小批量、可回滚切流和卡住任务扫描。新旧状态不兼容时保留旧运行版本执行完，不应强行让新代码解释旧快照。

[Q499 · 八股文 · 通用知识] 依据：参考资料：S6,S8

33. 最复杂的 Workflow 大概有多大?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q500 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

34. 平台支持哪些节点类型?

参考答案：节点是接收 State、返回字段更新的 Python 函数。Chat 图包含 parse、validate_drafts、prepare_nutrition、save_ready、prepare_clarification、clarify、persist、fact、respond；计划图是 prepare、approve、activate；分析图是 analyze、review。节点可以调模型、查库、计算或 interrupt，但没有通用用户代码节点和动态节点市场。

[Q501 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

35. 迁移后，原有 Workflow 能力怎样继续承载?

参考答案：迁移时可把原 Workflow 包装为有版本的能力或子图，保持原输入输出、分支、中断和错误处理语义。新运行时通过适配器调用，原流程内的工具也接统一权限、预算、观测和幂等契约。先用历史任务比较结果与合法轨迹，再灰度切换；运行中旧任务绑定旧版本执行完，或在兼容检查点处迁移。将其称为 Skill 只是封装，是否保留可靠性要靠语义映射与故障回归验证。

[Q502 · 八股文 · 通用知识] 依据：参考资料：S6

36. 你说的 Workflow Skill，究竟是描述文件，还是一段可执行代码？

参考答案：Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q503 · 八股文 · 通用知识] 依据：参考资料：S3,S6

37. 追问：业务逻辑是不是仍然需要研发人员编写代码？

参考答案：轻记的真正业务逻辑在 Python 代码中：检查缺失字段、合并限定补丁、营养单位换算、保存修订、计划变化比较和归档。图用 Python 装配，页面只是收集输入与展示结果。即使以后增加 YAML 或 Skill 描述，也只能声明接口与策略，状态恢复、事务和健康约束仍需要研发实现并测试。

[Q504 · 工程现状] 依据：backend/app/chat/drafts.py

38. 运行内核以外，还有哪些能够稳定复用的公共资产？

参考答案：运行控制之外可复用的公共资产包括五类事件 Schema、上海时区处理、营养标签基准、证据引用格式、计划差异展示、失败样本和前端待追问组件。复用时要区分通用工程契约与健康领域规则：默认目标公式和医疗边界需单独审核，不能直接当任意行业通用逻辑。

[Q505 · 工程现状] 依据：backend/app/api/schemas.py; backend/tests/

九、百度

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q506 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 是否了解百度千帆 AppBuilder?

参考答案：应先说明了解程度，再从应用编排、模型与知识接入、工具、发布和企业治理几个方面概括。若没有深入使用， 应明确说明只做过资料阅读或基础验证，不要把了解产品说成生产经验。

[Q507 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

3. 是否了解 Coze、Dify 等 AI 应用开发平台?

参考答案：按真实使用深度说明。可以从画布编排、模型与知识库接入、工具、发布、运行时和企业治理比较，但要区分阅读资料、实际试用和生产使用。不了解的实现细节不要推测。

[Q508 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

4. 当前平台的产品定位和技术选型是什么?

参考答案：先说明服务对象、核心问题和交付形态，再说明运行框架、服务语言、存储、队列、模型网关和部署方式。每项技术选择都要对应业务约束，不能只列技术名称。

[Q509 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

5. 为什么使用 LangGraph，平台自行实现了哪些部分?

参考答案：当前 LangGraph 承担 StateGraph、条件路由、interrupt 和 checkpoint；Engine 另外管理请求指纹、回执、锁和 API 结果。营养准备、历史检索、计划版本、统计、归档都在普通 Python 模块中。边界仍有耦合：Engine 把图和业务函数连在一起，还不是可插拔通用 Runtime，但更换编排层不必重写所有健康算法。

[Q510 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

6. 画布式 Workflow 编排中最大的难点是什么?

参考答案：DSL 先经过语法、参数格式、类型、引用和环路校验，再解析为带版本的 IR，完成默认值、依赖解析和节点绑定，最后编译为 DAG/状态图。自然语言只生成受限 IR 草案，不直接发布最终 DSL。生成后做静态校验、沙箱试跑、测试用例验证和基于错误的有限修复。

[Q511 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

7. 大型复杂流程如何通过子 Workflow 拆分?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q512 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

8. 主流程与子流程怎样传递上下文和结果?

参考答案：Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果，Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema；结果返回状态、证据及产物引用。Worker 可预声明，也可受控动态创建，动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并，副作用用幂等键；检测循环委派并传播取消，避免所有 Worker 同时修改同一业务对象。

[Q513 · 八股文 · 通用知识] 依据：参考资料：S1,S6

9. 追问：子流程出现中断或异常，主流程如何处理?

参考答案：Manager 负责拆解目标、分配预算、跟踪依赖并汇总结果，Worker 处理独立子任务。任务契约包含 task_id、parent_id、目标、输入引用、权限、截止时间、版本和输出 Schema；结果返回状态、证据及产物引用。Worker 可预声明，也可受控动态创建，动态方式需限制数量、深度和权限。共享状态通过 reducer、事务或版本检查合并，副作用用幂等键；检测循环委派并传播取消，避免所有 Worker 同时修改同一业务对象。

[Q514 · 八股文 · 通用知识] 依据：参考资料：S1,S6

10. 代码节点怎样保证执行安全?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q515 · 八股文 · 通用知识] 依据：参考资料：S9,S10

11. Docker 或 Kubernetes 提供了哪些隔离机制?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q516 · 八股文 · 通用知识] 依据：参考资料：S9,S10

12. 从文件上传到生成回答，完整 RAG 链路是什么？

参考答案：RAG 分离线入库和在线问答。入库先解析与清洗文档，保留结构、页码、来源和权限，再切分、生成向量并写入检索索引；文档更新与删除需同步处理版本。在线先理解问题并按需改写，执行权限过滤、关键词/向量召回、融合、重排与去重，再按 Token 预算组装证据，生成带引用的答案。证据不足时澄清或拒答。评测要把解析、召回、排序、引用和答案分别统计，便于定位故障。

[Q517 · 八股文 · 通用知识] 依据：参考资料：S2,S11

13. Excel 等结构化文档如何解析和切分？

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走 OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用 SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

[Q518 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

14. 固定切分、层级切分和语义切分如何选择？

参考答案：常见切分有固定长度加重叠、递归分隔符、标题层级、语义切分和父子块。固定切分简单稳定；结构切分适合章节明确文档；语义切分尝试保留主题完整性但成本较高。表格保留表头与单位，代码按函数/类及依赖处理。检索可用小块定位，再扩展相邻块或父章节，处理跨块证据。每块保存来源、位置、层级、版本和权限，重叠过多会增加重复与索引成本。

[Q519 · 八股文 · 通用知识] 依据：参考资料：S2

15. PPL 语义切分的原理和依据是什么？

参考答案：PPL 是语言模型对文本预测难度的度量，通常为平均负对数似然的指数。基于 PPL 的切分利用相邻句或局部窗口的预测难度变化，寻找可能的主题边界；不同实现的上下文、打分与边界规则并不统一。可对候选句边界计算条件负对数似然，平滑后取局部峰值，同时施加最小/最大块长及结构规则。分数高也可能来自术语、代码或 OCR 错误，不能直接等同语义断点。需与结构切分对照评测。

[Q520 · 八股文 · 通用知识] 依据：参考资料：S2

16. 追问：怎么通过评测证明这种切分有效？

参考答案：PPL 是语言模型对文本预测难度的度量，通常为平均负对数似然的指数。基于 PPL 的切分利用相邻句或局部窗口的预测难度变化，寻找可能的主题边界；不同实现的上下文、打分与边界规则并不统一。可对候选句边界计算条件负对数似然，平滑后取局部峰值，同时施加最小/最大块长及结构规则。分数高也可能来自术语、代码或 OCR 错误，不能直接等同语义断点。需与结构切分对照评测。

[Q521 · 八股文 · 通用知识] 依据：参考资料：S2

17. BM25、向量检索、RRF 和 Rerank 如何组合？

参考答案：BM25 擅长实体、术语和精确词匹配，稠密向量检索擅长语义近邻，二者互补。分别召回后按文档/片段 ID 去重，用 RRF 等方法融合；RRF 的分数是各路 1/(k+rank) 之和，避免直接混加不同量纲的原始分数。随后用 Cross-Encoder 等重排器精排少量候选。结果高度重复时可用 MMR 在相关性与多样性间取舍。每增加一层都要验证 Recall、答案质量、延迟和成本，不能默认链路越长越好。

[Q522 · 八股文 · 通用知识] 依据：参考资料：S2,S11

18. 答案需要的信息跨多个 Chunk 时，怎样召回?

参考答案：常见切分有固定长度加重叠、递归分隔符、标题层级、语义切分和父子块。固定切分简单稳定；结构切分适合章节明确文档；语义切分尝试保留主题完整性但成本较高。表格保留表头与单位，代码按函数/类及依赖处理。检索可用小块定位，再扩展相邻块或父章节，处理跨块证据。每块保存来源、位置、层级、版本和权限，重叠过多会增加重复与索引成本。

[Q523 · 八股文 · 通用知识] 依据：参考资料：S2

19. 怎样看待 Coding Agent 使用 grep 或文件遍历，而不是向量 RAG?

参考答案：向量存储按数据量、过滤能力、更新频率、一致性、延迟和运维条件选择。已有 PostgreSQL 且规模合适时，可评估向量扩展；搜索与关键词过滤需求强时，可评估搜索引擎的向量能力；需要专用向量服务时，再比较其索引、分片和混合检索能力。选型要用同一数据与查询集测试 Recall@K、P95、写入/删除传播和资源成本。保存 doc_id、chunk_id、权限、版本和来源；向量索引通常可重建，不能替代原文真值库。

[Q524 · 八股文 · 通用知识] 依据：参考资料：S2

20. PageIndex 一类层级索引适合什么场景?

参考答案：向量存储按数据量、过滤能力、更新频率、一致性、延迟和运维条件选择。已有 PostgreSQL 且规模合适时，可评估向量扩展；搜索与关键词过滤需求强时，可评估搜索引擎的向量能力；需要专用向量服务时，再比较其索引、分片和混合检索能力。选型要用同一数据与查询集测试 Recall@K、P95、写入/删除传播和资源成本。保存 doc_id、chunk_id、权限、版本和来源；向量索引通常可重建，不能替代原文真值库。

[Q525 · 八股文 · 通用知识] 依据：参考资料：S2

21. 流式节点与非流式节点如何衔接，并统一展示?

参考答案：SSE 使用 HTTP 的 text/event-stream 单向推送，事件可包含 id、event 和 data。业务先持久化 task_id、递增 seq 和事件，再发送；客户端保存游标，重连携带 Last-Event-ID，服务端重放其后事件，客户端按 ID 去重。游标过期时返回快照并重新同步；心跳、代理缓冲、断开和背压需单独处理。节点可统一输出进度、内容增量、卡片、等待和完成事件。SSE 重连本身不持久化任务，也不保证副作用只执行一次。

[Q526 · 八股文 · 通用知识] 依据：参考资料：S7

22. 追问：是否研究过 Dify 对这两类节点的抽象?

参考答案：SSE 使用 HTTP 的 text/event-stream 单向推送，事件可包含 id、event 和 data。业务先持久化 task_id、递增 seq 和事件，再发送；客户端保存游标，重连携带 Last-Event-ID，服务端重放其后事件，客户端按 ID 去重。游标过期时返回快照并重新同步；心跳、代理缓冲、断开和背压需单独处理。节点可统一输出进度、内容增量、卡片、等待和完成事件。SSE 重连本身不持久化任务，也不保证副作用只执行一次。

[Q527 · 八股文 · 通用知识] 依据：参考资料：S7

23. 平台的客户量、调用量和并发规模怎样?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q528 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

24. 平台如何部署，Pod 和 Worker 如何组织?

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

[Q529 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

25. Python GIL 对多核并发有什么影响?

参考答案：同步模型 I/O 仍在线程内执行，但已不再在整个网络等待期间持有 Engine 全局锁：model_io 释放本地锁，返回后重取并检查执行代次及读依赖。相同工作流/会话通过键控锁协调，SQLite 写事务仍需互斥；RuntimeLimiter 默认允许两个运行段。数据目录仅允许一个 Engine，不能直接增加 Uvicorn workers 扩容。现有局部基准不代表生产 QPS，应分别测锁等待、模型、数据库和端到端负载。

[Q530 · 工程现状与边界] 依据：backend/app/workflows/concurrency.py; backend/app/common/execution_budget.py; backend/app/infrastructure/data_owner.py; backend/tests/test_workflow_concurrency.py

26. FastAPI 的并发依靠协程、线程还是进程?

参考答案：同步模型 I/O 仍在线程内执行，但已不再在整个网络等待期间持有 Engine 全局锁：model_io 释放本地锁，返回后重取并检查执行代次及读依赖。相同工作流/会话通过键控锁协调，SQLite 写事务仍需互斥；RuntimeLimiter 默认允许两个运行段。数据目录仅允许一个 Engine，不能直接增加 Uvicorn workers 扩容。现有局部基准不代表生产 QPS，应分别测锁等待、模型、数据库和端到端负载。

[Q531 · 工程现状与边界] 依据：backend/app/workflows/concurrency.py; backend/app/common/execution_budget.py; backend/app/infrastructure/data_owner.py; backend/tests/test_workflow_concurrency.py

27. 线上接口延迟突然升高，怎么排查?

参考答案：当前非流式 JSON，不能把请求 duration_ms 称为 TTFT。已有 HTTP、节点、模型、数据库、锁等待和后台任务的分层计时，可区分上游慢、排队慢和本地事务慢；用户端网络与小程序渲染还需另测。模型超时和本地兜底要单独计数，恢复段耗时与跨用户等待的完整任务耗时也不同。仓库的局部观测基准不代表生产容量或真实模型成本。

[Q532 · 工程现状与边界] 依据：backend/app/observability/metrics.py; backend/app/api/request_context.py; backend/app/workflows/concurrency.py; scripts/benchmark-observability.py

28. PostgreSQL 慢查询怎么定位，EXPLAIN 和索引如何使用?

参考答案：先从慢查询日志和统计视图找到高耗时 SQL，再用 EXPLAIN ANALYZE 查看实际执行计划、扫描行数、连接方式 、排序和临时文件。索引应服务真实过滤、连接和排序条件，同时检查基数估计、统计信息、锁等待和连接池。

[Q533 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

29. 【口述算法题】 怎样实现固定容量的 LRU Cache?

参考答案：LRU 淘汰最久未使用的数据，可用哈希表定位节点、双向链表维护访问顺序。get 命中后移到最近端，put 更新或插入后也移到最近端，超容量删除最久端并同步哈希表，平均时间 O(1)，空间 O(capacity)。需处理容量为零、更新已有键和并发访问。LRU 不保证数据新鲜，业务缓存还需版本/TTL/主动失效；常见大量一次性扫描可能污染缓存。参考实现见附录 G。

[Q534 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

30. 【口述系统设计题】 如果自己实现线程池，需要哪些核心组件，执行流程是什么?

参考答案：协程在事件循环中协作调度，适合大量可等待 I/O；线程适合阻塞 I/O 及可释放 GIL 的调用；多进程适合需要多核的 Python CPU 计算并提供内存隔离。常规有 GIL 的 CPython 线程不能同时执行 Python 字节码，某些无 GIL 构建及扩展另论。线程池由队列、Worker、Future、拒绝和关闭策略组成，Worker 阻塞取任务。限制队列与并发，避免同步阻塞事件循环或持锁跨网络等待。

[Q535 · 八股文 · 通用知识] 依据：参考资料：S4,S5

31. 追问：任务进入队列之后，由谁驱动 Worker 取出并执行?

参考答案：协程在事件循环中协作调度，适合大量可等待 I/O；线程适合阻塞 I/O 及可释放 GIL 的调用；多进程适合需要多核的 Python CPU 计算并提供内存隔离。常规有 GIL 的 CPython 线程不能同时执行 Python 字节码，某些无 GIL 构建及扩展另论。线程池由队列、Worker、Future、拒绝和关闭策略组成，Worker 阻塞取任务。限制队列与并发，避免同步阻塞事件循环或持锁跨网络等待。

[Q536 · 八股文 · 通用知识] 依据：参考资料：S4,S5

32. 如果项目涉及 Ray，会用它承担哪些工作?

参考答案：Ray 用于分布式任务、Actor 与资源调度，适合独立计算和需要状态服务的并行工作。业务输入、任务身份、重要检查点和结果引用应持久化，不能只存在某个 Worker 的内存中。任务重试、Actor 重启与控制面恢复有不同配置和前提，不能笼统保证 Head 重启后运行状态无损。设计时明确哪些计算可重放、哪些状态需恢复、哪些副作用需幂等，并对选定版本与部署逐项做故障测试。

[Q537 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

33. Ray Head 与 Worker 如何解耦?

参考答案：Ray 用于分布式任务、Actor 与资源调度，适合独立计算和需要状态服务的并行工作。业务输入、任务身份、重要检查点和结果引用应持久化，不能只存在某个 Worker 的内存中。任务重试、Actor 重启与控制面恢复有不同配置和前提，不能笼统保证 Head 重启后运行状态无损。设计时明确哪些计算可重放、哪些状态需恢复、哪些副作用需幂等，并对选定版本与部署逐项做故障测试。

[Q538 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

34. Head 故障后，任务调度、结果汇聚和后续任务怎样继续?

参考答案：Ray 用于分布式任务、Actor 与资源调度，适合独立计算和需要状态服务的并行工作。业务输入、任务身份、重要检查点和结果引用应持久化，不能只存在某个 Worker 的内存中。任务重试、Actor 重启与控制面恢复有不同配置和前提，不能笼统保证 Head 重启后运行状态无损。设计时明确哪些计算可重放、哪些状态需恢复、哪些副作用需幂等，并对选定版本与部署逐项做故障测试。

[Q539 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

二面

1. 研发经历是否都与 Agent 有关？如果涉及方向转型，是在什么阶段？

参考答案：按时间顺序说明当前岗位、主要工作和转向 Agent 的阶段，再说明最熟悉的技术方向。重点写清实际负责的模块 、投入时间和已经交付的结果，不要把学习、验证和生产经验混在一起。

[Q540 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. AI 应用平台具体是什么，客户是谁？

参考答案：轻记面向希望用自然语言记录饮食和身体状态的个人用户。核心问题是表单记录成本高、补记和纠错容易出错、后续建议缺少可追溯依据。当前交付是单用户微信小程序原型，具备五类记录、关键追问、历史查询、目标确认、复盘与站内提醒；没有企业租户、客户应用发布市场或通用低代码平台。

[Q541 · 工程现状] 依据：backend/app/main.py; miniprogram/src/app.tsx; README.md

3. 部署在哪里，目前规模多大？

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

[Q542 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

4. 你在平台中具体承担哪些工作？

参考答案：明确列出自己负责的模块、设计决策、代码范围、上线责任和结果指标。说明与产品、算法、后端、交付和运维的分工，并区分团队已有能力、开源能力和个人新增内容。

[Q543 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

5. 为什么选择 LangGraph，它在系统中负责什么？

参考答案：轻记需要两类等待：记录缺字段时等用户补充，计划变化较大时等用户确认。因此选择 LangGraph 的显式状态图、interrupt 和 SQLite checkpoint。图负责编排，草稿完整性、营养来源、请求去重和计划版本检查由自己的业务代码实现。当前没有统一任务基准证明它优于其他框架；选择主要依据是与这两类暂停恢复需求匹配。

[Q544 · 工程现状] 依据：backend/app/workflows/engine.py; backend/tests/test_agent.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

6. 工具分层最终解决谁的问题？模型看到的不都是工具吗？

参考答案：服务入口处理认证、协议、会话、限流和 SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

[Q545 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

7. 用户创建的 AI 应用如何评价，有哪些具体指标和数据?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q546 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

8. P95 TTFT 统计什么阶段？首响应较长时应如何分析原因?

参考答案：当前非流式 JSON，不能把请求 duration_ms 称为 TTFT。已有 HTTP、节点、模型、数据库、锁等待和后台任务的分层计时，可区分上游慢、排队慢和本地事务慢；用户端网络与小程序渲染还需另测。模型超时和本地兜底要单独计数，恢复段耗时与跨用户等待的完整任务耗时也不同。仓库的局部观测基准不代表生产容量或真实模型成本。

[Q547 · 工程现状与边界] 依据：backend/app/observability/metrics.py; backend/app/api/request_context.py; backend/app/workflows/concurrency.py; scripts/benchmark-observability.py

9. 画布平台、Agent Harness 和客户交付是什么关系?

参考答案：轻记是独立的垂直业务应用，代码中没有依赖某个原有 AI 平台或企业 SDK。它复用了 LangGraph、FastAPI 和兼容模型 API，自己补健康数据模型与可靠性规则。如果以后挂到已有平台，应让平台复用身份、模型接入和运维能力，轻记保留事件完整性、营养来源、计划版本这些领域约束，不能虚构已有平台共建经历。

[Q548 · 工程现状] 依据：backend/requirements.txt; backend/app/main.py

10. 客户最终购买什么，拿到什么交付物?

参考答案：建议答法：平台面向【内部开发者/企业客户】，把模型、知识、工具和流程统一成可发布、可治理的 AI 应用。 核心解决重复接入、权限审计、长任务可靠性和效果回归。交付物应明确为产品实例、配置或扩展包、接口与运维指标，不能只说“交付一个 Agent”。

[Q549 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

11. 客户数据在这条链路中如何处理?

参考答案：本地存储不等于模型模式下不外发。当前可由服务端或单次请求启用 local_only，统一禁止规划、纠错、估算与表达等外部调用，该约束随暂停流程保存，恢复不能放宽。模型模式按用途授权与最小字段投影发送，并披露实际调用类别。加固部署提供受限模型出口和密钥文件配置；若要求完全内网运行，需使用纯本地模式或经过验证的内网模型，并检查部署出口，不能只依赖提示词。

[Q550 · 工程现状与边界] 依据：backend/app/modeling/boundary.py; backend/app/modeling/payloads.py; backend/app/common/config.py; deploy/network-policy.md

12. 平台的执行引擎与独立 Agent 运行内核，本质区别是什么?

参考答案：执行引擎只负责节点调度、状态传递和条件跳转。Agent 运行内核还负责模型与工具循环、上下文、权限、预算 、恢复、事件、评测和多租户。前者是调度组件，后者是完整运行环境。

[Q551 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

13. Skill 有什么作用?

参考答案：Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q552 · 八股文 · 通用知识] 依据：参考资料：S3,S6

14. 追问：为什么还要把 Workflow 包装成 Skill?

参考答案：Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q553 · 八股文 · 通用知识] 依据：参考资料：S3,S6

15. 企业 Agent 为什么不像 Codex、Claude Code 那样保持更开放的自主性?

参考答案：企业任务通常有更严格的权限、审批、数据边界、审计和稳定性要求，因此不能把所有步骤交给模型自由决定。 可以在低风险、可校验的局部保留自主性，关键写入和发布仍由 Workflow、规则和人工确认控制。

[Q554 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

16. App 关闭再打开后，怎样继续看到原任务的流式输出?

参考答案：后台规划器按时间生成持久任务，执行器负责领取、执行、心跳与重试，当前仍同进程共享唯一 Engine。关闭小程序后，只要服务存活就能生成站内消息；服务停机期间不会实时通知，重启按限定窗口补跑。失败区分可重试与终止，达到上限进入可处理的失败状态。没有设备推送、SSE 字流续传或独立分布式队列；交互请求的重试仍使用原 request_id。

[Q555 · 工程现状与边界] 依据：backend/app/jobs/scheduler.py; backend/app/jobs/planner.py; backend/app/jobs/runner.py; backend/app/jobs/manage.py; backend/tests/test_scheduler_operations.py

17. 追问：SSE 重连怎样做到不漏字、不重复，协议具体怎么设计?

参考答案：SSE 使用 HTTP 的 text/event-stream 单向推送，事件可包含 id、event 和 data。业务先持久化 task_id、递增 seq 和事件，再发送；客户端保存游标，重连携带 Last-Event-ID，服务端重放其后事件，客户端按 ID 去重。游标过期时返回快照并重新同步；心跳、代理缓冲、断开和背压需单独处理。节点可统一输出进度、内容增量、卡片、等待和完成事件。SSE 重连本身不持久化任务，也不保证副作用只执行一次。

[Q556 · 八股文 · 通用知识] 依据：参考资料：S7

18. Workflow-as-Skill 中是否包含外部 I/O?

参考答案：轻记图包含实际 I/O：模型 HTTP 请求、SQLite 写入、checkpoint 和通知记录。若包装成 Workflow Skill，与普通一次性 API 的区别是有多个持久步骤、等待用户和恢复状态；外部仍可以用 API 暴露。命名为 Skill 不会自动获得可靠性，I/O 副作用必须在对应服务用稳定操作键去重。

[Q557 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

19. 追问：这样一个 Workflow 与普通 API 服务有什么区别?

参考答案：轻记的计划流程可以通过 REST 暴露，但内部包含持久步骤与等待确认；普通单次 API 也能实现这些语义，区别在契约和生命周期而非“Skill”名称。若包装 Workflow，应把 pending、resume、版本和幂等显式化。仅把一个 HTTP 调用写入描述文件，不会自动获得可恢复执行。

[Q558 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

20. Human-in-the-loop 在哪里中断，如何恢复?

参考答案：人工介入点是 clarify 与 approve_plan：前者补必需字段或消除纠错歧义，后者确认首份计划或超过 5% 的单次/累计目标变化。interrupt 返回问题并保存等待状态；恢复接口校验 thread_id、interrupt_id 和输入类型，确认时再次检查版本。尚未实现所有高风险外部动作的通用审批系统。

[Q559 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

21. Redis 中的状态过期以后，任务还能继续吗?

参考答案：如果 Redis 是唯一状态源，过期后任务无法可靠恢复，因此不能这样设计。任务关键状态应保存在持久化数据库或对象存储，Redis 只做缓存、锁或短期事件窗口。缓存丢失后可从持久化状态重建。

[Q560 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

22. 上下文工程具体做了哪些事情?

参考答案：上下文按任务目的构造。首轮规划只收到本次文字、时间和是否选中记录，不再默认携带最近 12 条历史。本地审核计划后，从固定视图取必要数据；纠错绑定单条记录，补充只发送当前草稿和待补字段，营养估算只发送本次食物与分量。建议使用白名单结构化投影，不发送历史 raw_text。目的、资源和实际 payload 都经授权核对，最终返回用途与数据类别说明。

[Q561 · 工程现状与边界] 依据：backend/app/modeling/payloads.py; backend/app/modeling/projections.py; backend/app/modeling/boundary.py; backend/tests/test_model.py

23. 项目是否包含 Memory，什么应该记、什么不应该记?

参考答案：记忆写入由业务边界控制：完整事件经校验、营养准备和事务入账；明确长期信息走结构化事实服务，带证据、有效期和修订。助手建议、模型推理或一次偏咸反馈不会自动升级为长期事实。读取使用与任务目的匹配的投影，纠错后让相关派生结果过期。管理接口支持新增、修改、撤回、冲突选择、历史与隐私删除；它仍是领域服务，不是动态 Memory 插件协议。

[Q562 · 工程现状与边界] 依据：backend/app/health/facts.py; backend/app/api/profile.py; backend/app/infrastructure/repositories/facts.py; backend/app/modeling/projections.py

24. “支持同步、异步”具体指什么?

参考答案: 交互工作流仍用同步图调用，在完成或等待用户时返回 JSON。后台由 asyncio 调度，规划与任务执行分离，阻塞业务在线程中执行，并使用持久任务、租约和恢复。不存在同步/异步两套 ReAct 引擎；只是不同入口复用同一业务边界。未来若将交互也改为异步提交，需要明确任务状态和结果获取协议，不能把函数改成 async 就认为完成进程解耦。

[Q563 · 工程现状与边界] 依据：backend/app/workflows/runtime.py; backend/app/jobs/scheduler.py; backend/app/jobs/runner.py

25. 追问：是否存在两套不同的 ReAct Loop?

参考答案: 交互工作流仍用同步图调用，在完成或等待用户时返回 JSON。后台由 asyncio 调度，规划与任务执行分离，阻塞业务在线程中执行，并使用持久任务、租约和恢复。不存在同步/异步两套 ReAct 引擎；只是不同入口复用同一业务边界。未来若将交互也改为异步提交，需要明确任务状态和结果获取协议，不能把函数改成 async 就认为完成进程解耦。

[Q564 · 工程现状与边界] 依据：backend/app/workflows/runtime.py; backend/app/jobs/scheduler.py; backend/app/jobs/runner.py

26. 现在是否还独立写代码，能否现场完成一道简单 Coding?

参考答案: 如实说明最近独立编码的频率、负责的模块和使用的语言，并明确可以完成现场编码。题面信息不完整时只说明解题流程、边界和复杂度，不补写无法确认的代码。

[Q565 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

三面

1. 介绍一下当前工作，重点讲技术设计、遇到的问题和解决方式。

参考答案: 我选轻记，因为它能完整讲清一条 Agent 业务闭环：自然语言输入拆成健康草稿，缺字段时追问，营养准备后写库，支持历史纠错，再用完整记录做计划和复盘。最有代表性的设计是 checkpoint 恢复配合幂等写入、未知营养不算零，以及计划确认时核对数据版本。它仍是单用户本地原型。

[Q566 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py; README.md

2. Harness 怎么设计，运行过程是什么，如何提升效果?

参考答案: Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、 工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

[Q567 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

3. 通用运行内核、业务扩展和服务入口怎样分工?

参考答案: 接口在 api，工作流编排在 workflows，模型适配与外发边界在 modeling；health 承担营养、目标、分析和长期事实，privacy 管血缘删除，jobs 管任务，observability 管受控观测，infrastructure 管事务和仓储。LangGraph 提供状态图与暂停恢复，业务约束由这些模块共同执行。当前是一个应用内的模块化分层，不是已经独立部署的微服务或通用插件平台。

[Q568 · 工程现状与边界] 依据：backend/app/main.py; backend/app/workflows/engine.py; backend/app/modeling/parser.py; backend/app/infrastructure/repositories/repository.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

4. 实际交付给客户的是中间件，还是云上的产品?

参考答案：必须按真实交付形态回答，并说明客户得到的是云服务、私有部署软件、运行时组件还是配置与扩展包。还要说明部署位置、运维责任、升级方式、数据边界和服务指标，不能只用“平台”概括。

[Q569 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

5. ReAct、Skill 和 Browser Use 等能力运行在哪里?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q570 · 八股文 · 通用知识] 依据：参考资料：S9,S10

6. 为什么多个 Agent 共享 Worker?

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

[Q571 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

7. 不同用户和业务数据怎样隔离?

参考答案：身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q572 · 八股文 · 通用知识] 依据：参考资料：S9,S10

8. 是否提供代码和 Bash 能力，代码在哪里执行?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q573 · 八股文 · 通用知识] 依据：参考资料：S9,S10

9. 为什么使用 LangGraph，而不是直接实现一个 Loop?

参考答案：轻记需要两类等待：记录缺字段时等用户补充，计划变化较大时等用户确认。因此选择 LangGraph 的显式状态图、interrupt 和 SQLite checkpoint。图负责编排，草稿完整性、营养来源、请求去重和计划版本检查由自己的业务代码实现。当前没有统一任务基准证明它优于其他框架；选择主要依据是与这两类暂停恢复需求匹配。

[Q574 · 工程现状] 依据：backend/app/workflows/engine.py; backend/tests/test_agent.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

10. 运行图包含哪些节点?

参考答案：节点是接收 State、返回字段更新的 Python 函数。Chat 图包含 parse、validate_drafts、prepare_nutrition、save_ready、prepare_clarification、clarify、persist、fact、respond；计划图是 prepare、approve、activate；分析图是 analyze、review。节点可以调模型、查库、计算或 interrupt，但没有通用用户代码节点和动态节点市场。

[Q575 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

11. 追问：LangGraph 能否替换，替换会影响哪些部分?

参考答案：应把 LangGraph 限定在状态图和节点调度层。任务存储、租约、权限、工具执行、事件协议、评测和业务扩展通过自有接口实现。这样替换框架时主要重写图适配层和检查点转换，不会影响对外接口与业务能力。

[Q576 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

12. Skill 中的代码怎样运行?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q577 · 八股文 · 通用知识] 依据：参考资料：S9,S10

13. Workflow 形式的 Skill 如何定义和执行?

参考答案：Skill 可以是 Markdown 说明、配置、脚本或固定 Workflow 的组合，宿主负责加载和选择受控入口。Workflow-as-Skill 将验证过的流程封装为有版本、输入输出、权限和预算的能力，Agent 只需决定何时调用；内部仍按确定性节点执行。优点是复用、可审计、上下文短，代价是灵活性下降以及契约/版本维护。暂停、人工确认和异步等待必须返回明确状态，完成后再交回主决策循环。

[Q578 · 八股文 · 通用知识] 依据：参考资料：S3,S6

14. LangGraph Checkpoint 与自行建设的持久化恢复机制，有什么区别?

参考答案：图快照由 LangGraph SqliteSaver 保存，包括 State 和执行进度：extraction、待补字段、事件下标、prepared_events、record_ids、计划数据等。自己维护的 health.db 保存事实、workflow API 结果、请求指纹、回执和操作去重。checkpoint 用于继续执行，Repository 决定业务事实是否已提交；两者不是同一种存储，也不是同一事务。

[Q579 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

15. 如何评估性能和稳定性?

参考答案：轻记稳定性用幂等、重启恢复、模型失败、歧义纠错、营养未知和计划过期等故障场景验证；性能应另测并发、锁等待、模型耗时、数据库耗时与端到端延迟。当前有自动化逻辑测试，没有可核验生产负载基准或容量结论。前端 DOM 测试也不能替代真实浏览器视觉与网络体验。

[Q580 · 工程现状] 依据：backend/tests/; README.md

16. 如果一次工具调用失败，但最后任务成功了，成功率应该怎么算?

参考答案：若轻记某次表达模型失败但本地兜底返回且记录正确保存，可计记录任务成功、表达调用失败、降级一次；若营养估算失败但保存未知，则按产品定义计记录完成且营养未齐全。需要同时报告节点失败率、任务完成率和降级率，不能只统计最终 200 掩盖上游不稳定，也不能把一切工具失败都算任务失败。

[Q581 · 工程现状] 依据：backend/app/health/nutrition.py

17. Agent 的 Ground Truth 和测试用例如何构建?

参考答案：验证分输入理解、业务不变量、运行故障和安全四层：用合成案例与 mock 响应检查日期/分量、未知营养、证据、计划确认；用重放和故障注入检查幂等、租约、删除恢复、超时及旧结果拒写；用固定安全样例检查风险路由和受限输出。仓库有测试与历史验证报告，本次资料更新未重跑完整应用测试。测试通过不等于真实模型准确率、用户增长或医学效果，生产 A/B 和统一 Judge 仍需另建。

[Q582 · 工程现状与边界] 依据：backend/tests/test_harness.py; backend/tests/test_runtime_recovery.py; backend/tests/test_scheduler_reliability.py; backend/tests/test_safety_routing.py; docs/evidence/safety-evaluation-2026-10-26.json

18. 追问：怎样判断任务真正完成?

参考答案：轻记任务完成要看业务后置条件：记录 ID 能在 events 找到、营养未知被明确标识、卡片数值来自保存值；计划任务还要区分 active、rejected、stale。waiting_input 表示尚需用户，不能计为完整保存。未来可把这些条件声明成机器可校验契约；当前 completed 是图执行完毕，仍需结合结果语义判断业务是否达成。

[Q583 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py

19. 耗时、性能和 Token 消耗目前是什么水平?

参考答案：当前非流式 JSON，不能把请求 duration_ms 称为 TTFT。已有 HTTP、节点、模型、数据库、锁等待和后台任务的分层计时，可区分上游慢、排队慢和本地事务慢；用户端网络与小程序渲染还需另测。模型超时和本地兜底要单独计数，恢复段耗时与跨用户等待的完整任务耗时也不同。仓库的局部观测基准不代表生产容量或真实模型成本。

[Q584 · 工程现状与边界] 依据：backend/app/observability/metrics.py; backend/app/api/request_context.py; backend/app/workflows/concurrency.py; scripts/benchmark-observability.py

20. 追问：如何判断首个可见响应时间是否过长，统计的究竟是哪段时间?

参考答案：当前非流式 JSON，不能把请求 duration_ms 称为 TTFT。已有 HTTP、节点、模型、数据库、锁等待和后台任务的分层计时，可区分上游慢、排队慢和本地事务慢；用户端网络与小程序渲染还需另测。模型超时和本地兜底要单独计数，恢复段耗时与跨用户等待的完整任务耗时也不同。仓库的局部观测基准不代表生产容量或真实模型成本。

[Q585 · 工程现状与边界] 依据：backend/app/observability/metrics.py; backend/app/api/request_context.py; backend/app/workflows/concurrency.py; scripts/benchmark-observability.py

21. 实际客户、终端用户和并发规模如何?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q586 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

22. BM25、Embedding、RRF、Reranker 与 MMR 怎么组合?

参考答案：BM25 擅长实体、术语和精确词匹配，稠密向量检索擅长语义近邻，二者互补。分别召回后按文档/片段 ID 去重，用 RRF 等方法融合；RRF 的分数是各路 1/(k+rank) 之和，避免直接混加不同量纲的原始分数。随后用 Cross-Encoder 等重排器精排少量候选。结果高度重复时可用 MMR 在相关性与多样性间取舍。每增加一层都要验证 Recall、答案质量、延迟和成本，不能默认链路越长越好。

[Q587 · 八股文 · 通用知识] 依据：参考资料：S2,S11

23. 如果要继续提高召回率，最难的部分是什么?

参考答案：最难的是建立可靠的参考数据并处理需要跨段、跨文档或隐含关系的问题。继续调参数的收益通常有限，更关键的是文档结构、查询理解、权限过滤和多跳召回。每项改动都要用固定数据集确认是否真正提高有效证据命中。

[Q588 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

24. RAG 评测是否亲自做过，用过哪些指标?

参考答案：先建立带相关证据标注的固定问题集，并按实体、同义表达、多跳、无答案和权限场景分层。Recall@K 衡量相关证据覆盖；MRR 是首个相关结果倒数排名的均值；NDCG 结合相关性等级与位置衡量排序。回答另测忠实性、完整性、引用正确率和拒答。对照实验固定语料版本、模型和预算，一次改变主要因素，报告样本、差值和不确定性；最终质量提升不应只由单个检索分数推断。

[Q589 · 八股文 · 通用知识] 依据：参考资料：S2

25. Recall@K、MRR、NDCG 分别适合衡量什么?

参考答案：先建立带相关证据标注的固定问题集，并按实体、同义表达、多跳、无答案和权限场景分层。Recall@K 衡量相关证据覆盖；MRR 是首个相关结果倒数排名的均值；NDCG 结合相关性等级与位置衡量排序。回答另测忠实性、完整性、引用正确率和拒答。对照实验固定语料版本、模型和预算，一次改变主要因素，报告样本、差值和不确定性；最终质量提升不应只由单个检索分数推断。

[Q590 · 八股文 · 通用知识] 依据：参考资料：S2

26. 是否关注 Codex、Claude Code、WorkBuddy 等新方案?

参考答案：可从长任务持久化、统一工具协议、渐进式能力加载、工作空间与 Artifact、上下文压缩、计算机操作和多模型路由说明。回答重点是这些变化会影响现有架构的哪一层，以及是否经过实际验证。

[Q591 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

27. 是否阅读源码，最近看到了哪些设计变化?

参考答案：如实说明阅读过的项目、跟踪的调用链和完成的验证。重点应覆盖会话创建、上下文组装、工具执行、退出条件 、状态保存、恢复和流式事件，并说明自己确认了哪些设计限制。

[Q592 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

28. 你提到的新 Harness 有什么优势？这种优势是绝对的吗?

参考答案：轻记的差异化集中在健康业务约束：宽松草稿追问、逐字段营养来源、未知值传播、累计目标变化确认、数据版本防旧计划和删除上下文副本。它不追求覆盖通用平台的画布、插件和多租户功能。这些设计与场景匹配，但是否比其他方案更好要看同任务对照，不能说优势绝对。

[Q593 · 工程现状] 依据：backend/app/chat/drafts.py

29. 为什么现在考虑换工作?

参考答案：用正向原因回答：当前工作已经形成了哪些积累，下一阶段希望扩大哪类责任或技术深度，以及目标岗位为什么更匹配。不要评价原公司的人和管理，也不要只把薪资作为原因。

[Q594 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

十、深信服

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

[Q595 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

2. 最近的工作主要都与 Agent 有关吗?

参考答案：按时间顺序说明当前岗位、主要工作和转向 Agent 的阶段，再说明最熟悉的技术方向。重点写清实际负责的模块 、投入时间和已经交付的结果，不要把学习、验证和生产经验混在一起。

[Q596 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

3. 具体涉及平台研发、框架开发还是业务应用落地?

参考答案：按实际投入比例说明三类工作的范围。平台研发关注发布与治理，框架开发关注运行机制和扩展接口，业务落地关注领域数据、流程与效果。说明主要职责和次要参与内容，避免把所有方向都说成核心经验。

[Q597 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

4. 选一个项目，讲讲怎样快速落地 Agent 应用。

参考答案：我选轻记，因为它能完整讲清一条 Agent 业务闭环：自然语言输入拆成健康草稿，缺字段时追问，营养准备后写库，支持历史纠错，再用完整记录做计划和复盘。最有代表性的设计是 checkpoint 恢复配合幂等写入、未知营养不算零，以及计划确认时核对数据版本。它仍是单用户本地原型。

[Q598 · 工程现状] 依据：backend/app/workflows/engine.py; backend/app/workflows/chat_flow.py; backend/app/workflows/runtime.py; README.md

5. 刚才业务场景讲得比较多，能否进一步介绍 Agent 架构设计?

参考答案：当前入口是 Taro/React/TypeScript 微信小程序，连接独立 FastAPI JSON API；旧 Web 前端已废弃，根路径不再托管页面。Engine 复用聊天、计划、分析图，health.db 保存业务事实，checkpoints.db 保存图进度。开发启动用 start.ps1；仓库另有 Docker 加固部署配置。后台任务与 Web 仍共享唯一 Engine，数据目录所有者锁阻止第二实例，不是独立 Worker 集群。正式对外服务仍需 HTTPS、鉴权及相应部署验收。

[Q599 · 工程现状与边界] 依据：miniprogram/package.json; backend/app/main.py; backend/app/infrastructure/data_owner.py; README.md; compose.hardened.yaml; start.ps1

6. 框架更偏固定 Workflow，还是自主规划 Agent?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q600 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

7. 在 Agent 的具体设计中，你承担了哪些工作?

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

[Q601 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

8. 参考通用 Coding Agent 的实现思路，当前业务 Agent 还有哪些可以改进的地方?

参考答案：可借鉴通用 Coding Agent 的是明确任务状态、受限能力、按需检索与验证，不必照搬开放 Bash。轻记已有节点级计时、调用次数与字节预算、任务版本和实际读取血缘，后续可补精确 Token 预算及更完善的证据回读。具体例子是旧营养指纹变化才重估、过时计划拒绝激活；这些已有规则比泛泛“模型反思”更可检查。

[Q602 · 工程现状] 依据：backend/app/health/nutrition.py

9. 如何将具体业务场景和固定 Workflow 加入 ReAct 执行过程?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

[Q603 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

10. 追问：有哪些具体参考例子?

参考答案：具体例子可以讲两项：食物或分量变了，营养指纹失效并重新准备；计划草案生成后用户又补记，data_version 不匹配就拒绝激活。二者都是轻记的本地验证，不依赖模型自觉反思。若参考 Coding Agent 改进，已有节点计时，后续可继续完善证据回读体验，但当前没有任意代码执行闭环。

[Q604 · 工程现状] 依据：backend/app/health/nutrition.py

11. 是否有后端或平台研发经验？如有，涉及哪些方面?

参考答案：按真实经历说明服务拆分、接口设计、数据库、缓存、队列、并发、监控和部署中实际负责的部分。只描述亲自做过的模块，并说明规模、故障处理和结果。

[Q605 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

12. 是否有 NLP 相关研究经历?

参考答案：如实区分论文研究、工程应用和自学验证。说明涉及的任务、数据、模型、评测方法和最终产出。如果没有正式研究经历，应直接说明，并补充已有的文本处理和大模型应用经验。

[Q606 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

13. 工作中是否参与过大模型训练?

参考答案：如实区分应用调用、推理部署和训练经验。多数企业 Agent 先用 Prompt、工具、RAG 与评测解决问题。只有错误模式稳定、数据量足且收益可验证时才考虑 SFT/DPO，并建立数据授权、离线评测和灰度回滚。

[Q607 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

14. 当前 Agent 项目涉及模型训练，还是主要调用模型 API 或本地部署模型?

参考答案：模型通过兼容 Chat Completions 的传输层接入，供应商地址、模型名和凭据由运行配置显式提供，不能把示例模型名当作运行时事实。源码不再硬编码密钥；支持仓库外密钥文件，开发模式也可用环境变量。配置全空进入演示模式，部分缺失会失败；模型失败不会悄悄切换演示营养。当前没有多供应商路由网关，换模型需回归结构化输出、错误映射和数据边界。

[Q608 · 工程现状与边界] 依据：backend/app/common/config.py; backend/app/modeling/parser.py; backend/app/modeling/transport.py; backend/tests/test_config.py

十一、同花顺

一面

1. 近期的主力开发语言是什么？如使用过 Python 或 Java，分别用于哪些场景?

参考答案：轻记后端实际使用 Python，主要是 FastAPI、Pydantic、LangGraph 和 SQLite；前端使用 TypeScript/React。当前工程没有 Java 后端代码，因此只能把 Java 单例、线程池等作为知识题说明，不能声称在本项目做过 Java 微服务。其他语言经验需结合本人的真实履历。

[Q609 · 工程现状] 依据：backend/app/; miniprogram/src/; miniprogram/package.json

2. 公司内部的技术栈是否受到合规等限制?

参考答案：按真实情况说明语言、依赖、模型、网络访问、数据存储和开源许可证方面的限制。重点说明这些限制如何影响技术选型，以及团队怎样通过审批、私有部署、依赖扫描和数据分级满足要求。

[Q610 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

3. 团队使用哪些 AI Coding 工具和模型?

参考答案：AI 负责样板、测试草案和局部重构，人对架构、接口、安全和最终合并负责。多人协作按模块/接口拆分，小批次提交，强制代码审查、静态检查、测试和依赖扫描。统计比例不如说明哪些结果由人验证、出现错误怎样发现更有意义。

[Q611 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

4. AI 应用平台是什么产品，解决什么问题?

参考答案：轻记面向希望用自然语言记录饮食和身体状态的个人用户。核心问题是表单记录成本高、补记和纠错容易出错、后续建议缺少可追溯依据。当前交付是单用户微信小程序原型，具备五类记录、关键追问、历史查询、目标确认、复盘与站内提醒；没有企业租户、客户应用发布市场或通用低代码平台。

[Q612 · 工程现状] 依据：backend/app/main.py; miniprogram/src/app.tsx; README.md

5. 已有很多低代码 AI 平台，为什么仍然选择自研?

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

[Q613 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

6. 平台是否上线，用户量、调用量和并发量如何?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

[Q614 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

7. LangGraph、FastAPI 和 Python Worker 为什么能够支撑商业化平台?

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

[Q615 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

8. 如果 Workflow 没有 LLM 节点，而是大量代码节点，应该怎样执行?

参考答案：应按普通确定性任务执行，不必进入 ReAct。代码节点由 Workflow 引擎按依赖调度，在沙箱或受控 Worker 中运行，设置超时、资源配额、重试和检查点。使用 LLM 与否不影响流程的状态和观测。

[Q616 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

9. 追问：代码中出现死循环怎么办?

参考答案：执行放到短生命周期、无特权的隔离环境：只读基础镜像、独立用户和命名空间、CPU/内存/进程数/时长配额、 临时工作目录和默认断网。文件与网络按白名单开放。调度器在沙箱外做鉴权、审计和 Kill，死循环由超时与资源配额终止。

[Q617 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

10. ReAct 的核心执行逻辑是什么?

参考答案：最小 ReAct Loop 的步骤是：构建受预算约束的上下文，调用模型，解析 final 或 tool_calls；若为工具调用，先做 Schema、权限及幂等检查，执行后把 Observation 反馈给下一轮，并保存检查点。循环检查取消、截止时间、最大轮数、重复调用和完成条件。Hook 可插在构建上下文前、模型前后、工具前后和任务结束时，用来接入 Memory、Skill、日志及策略。示例伪代码见附录 C。

[Q618 · 八股文 · 通用知识] 依据：参考资料：S1

11. 以查询天气、控制电视和空调为例，从用户输入到 Function Call、Observation 和 Final Answer，完整 链路怎样运行?

参考答案：注册表保存稳定 tool_id、版本、JSON 参数格式、权限、超时/重试、幂等性和执行适配器。模型只看到经权限与场景过滤后的短描述。解析后在服务端做参数格式校验、参数规范化、鉴权和风险审批，再执行并返回结构化工具结果（Observation）。用调用正确率、参数合法率和任务完成率评估。

[Q619 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

12. 工具数量增加后，怎样避免选错工具或参数错误?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q620 · 八股文 · 通用知识] 依据：参考资料：S3,S9

13. 渐进式披露具体怎样实现?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q621 · 八股文 · 通用知识] 依据：参考资料：S3,S9

14. 几十个 Skill 如何被发现和激活?

参考答案：先用租户、身份、任务阶段、权限和风险做确定性过滤，再按名称/描述/标签或语义检索召回少量能力，必要时让路由模型选择。渐进式披露先提供简短目录，选中后再加载详细说明、Schema 或资源，避免把所有工具全文塞进上下文。相似能力需明确适用与排除条件，歧义时询问或支持显式选择；执行前再次鉴权。用路由准确率、误选率、参数合法率、完成率和成本评测。

[Q622 · 八股文 · 通用知识] 依据：参考资料：S3,S9

15. Skill 同时包含 Markdown、脚本、代码和工具时，运行时怎样执行?

参考答案：沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q623 · 八股文 · 通用知识] 依据：参考资料：S9,S10

16. Workflow-as-Skill 如何与 ReAct 主循环衔接?

参考答案：Skill 包含声明、说明、资源与可执行入口。运行时只把必要说明渐进式注入上下文。Workflow Skill 以受版本管理的确定性流程执行：ReAct 挂起并传入结构化参数，子流程返回结果/错误/产物后主循环恢复。脚本和代码必须经过签名、权限与沙箱策略。

[Q624 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

17. 为什么有了 MCP 之后，还需要 Skill?

参考答案：Skill 描述某类任务何时执行、怎样组合步骤、规则与产物要求；MCP 提供 Host、Client、Server 之间的能力发现和调用协议，可暴露 tools、resources 和 prompts。两者可独立使用：Skill 可调用 CLI/OpenAPI，MCP 工具也可直接供宿主使用。MCP 的价值是跨宿主的统一契约、连接和能力协商，而非自动提供全部安全。凭据在环境变量也不解决工具发现、参数协议与授权；执行方仍需鉴权、限流、审计和幂等。

[Q625 · 八股文 · 通用知识] 依据：参考资料：S3

18. 反过来，有了 Skill，MCP 能不能完全不用?

参考答案：可以不用 MCP。Skill 可以直接调用受控 CLI、OpenAPI 或本地函数；MCP Server 也可直接向宿主提供工具而不经过 Skill。前者描述任务方法，后者统一能力接入，解决的问题不同。少量稳定内部接口直接适配即可；多个宿主需要复用工具、资源和连接管理时，引入 MCP 更有意义。是否采用取决于集成与运维成本，权限、审计和幂等无论使用什么协议都要落实。

[Q626 · 八股文 · 通用知识] 依据：参考资料：S3

19. Skill 直接调用 OpenAPI、CLI 或临时生成代码，为什么不能替代 MCP?

参考答案：技术上可以用 OpenAPI 或 CLI 完成同样的业务调用，MCP 并非唯一通道。MCP 的收益是把能力发现、Schema、调用/结果和连接生命周期统一，减少每个宿主分别适配命令和错误输出的成本。CLI 更贴近现有运维工具，但要处理参数、环境、返回值和进程权限；OpenAPI 擅长描述 HTTP 服务。临时生成代码还增加执行和审查成本。工具少且关系稳定可直接集成，跨宿主复用多时再比较协议收益。

[Q627 · 八股文 · 通用知识] 依据：参考资料：S3

20. 追问：如果 API Key 已在环境变量中，模型只需要执行简化命令，MCP 还提供什么额外价值?

参考答案：环境变量只解决凭据如何注入，没有定义模型怎样发现能力、理解参数、处理结果和协商协议。MCP 可提供统一工具目录、输入输出契约、资源访问及客户端-服务端交互，使多个宿主复用同一服务。它不自动隐藏所有密钥或授予最小权限：凭据应由可信执行端持有，工具仍按用户和资源授权。若只有几条稳定命令且已有良好适配，MCP 带来的额外收益可能有限。

[Q628 · 八股文 · 通用知识] 依据：参考资料：S3

21. MCP 相比标准 OpenAPI 真正增加了什么?

参考答案：OpenAPI 主要描述 HTTP API 的路径、方法、参数、响应与安全方案；MCP 面向模型宿主与能力服务之间的交互，约定初始化/能力协商，以及 tools、resources、prompts 的发现与使用。可以把已有 OpenAPI 包装为 MCP 服务，二者不冲突。MCP 不是自动完成业务语义理解或权限治理的魔法层；客户端仍要选择能力、校验结果和执行授权。比较时看跨宿主复用与协议适配成本。

[Q629 · 八股文 · 通用知识] 依据：参考资料：S3

22. 你提到的安全、治理和复用价值，是 MCP 独有的吗?

参考答案：安全、审计、限流、幂等、版本管理和租户隔离都不是 MCP 独有能力，CLI 或 OpenAPI 接入也能实现。MCP 的主要价值是统一发现和交互契约，使治理能力更容易在宿主与服务间一致接入；实际策略仍由平台和服务执行。不能把“遵循 MCP”当成已经安全，也不能因治理可自行实现就否定协议标准化的价值。应分别评估协议互操作收益与平台已有治理能力。

[Q630 · 八股文 · 通用知识] 依据：参考资料：S3,S9

23. 面向千万用户的云端 Agent，整体怎样设计?

参考答案：身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q631 · 八股文 · 通用知识] 依据：参考资料：S9,S10

24. 是否需要给每个用户运行一个独立 Agent 实例?

参考答案：通常不需要。Agent 是配置、状态和任务记录，不等同于常驻进程。用户请求可由共享 Worker 池执行，状态按用户和任务隔离。只有持续占用本地资源、需要强隔离或长连接的场景才考虑独占实例。

[Q632 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

25. 用户自定义 Skill、长期记忆、偏好和上传文件如何加载、隔离?

参考答案: 身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q633 · 八股文 · 通用知识] 依据：参考资料：S9,S10

26. 公共资产与租户私有资产如何做到物理复用和逻辑隔离?

参考答案: 身份与 tenant_id 由服务端认证生成，贯穿请求、任务、工具、存储和审计。每次操作按主体-资源-动作执行 RBAC/ABAC，并检查资源归属，模型不能选择任意租户。数据库行、缓存键、队列、工作目录和文件都需命名空间；强隔离场景采用独立库或执行环境。图和公共模板可复用不可变版本，私有记忆与文件按授权加载。海量用户采用无状态入口、持久任务队列及 Worker 池，逻辑 Agent 无需每用户常驻进程。

[Q634 · 八股文 · 通用知识] 依据：参考资料：S9,S10

27. 用户要求执行 ps -ef 或终止某个 Java 进程时，如何保证安全?

参考答案: 沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q635 · 八股文 · 通用知识] 依据：参考资料：S9,S10

28. Agent 在沙箱内运行与在沙箱外调度，架构有什么不同?

参考答案: 沙箱用于限制代码对文件、网络、进程和资源的影响。可组合独立进程/容器、非 root、只读文件系统、最小挂载、能力裁剪、系统调用约束、出口白名单与 CPU/内存/PID/时限配额；更强隔离可考虑虚拟机。宿主管授权、凭据、状态和审计，执行环境只获得短期最小能力。ps/kill 等动作需验证进程归属、稳定标识及允许范围，高影响操作需明确授权。容器共享内核，不能仅凭容器化宣称安全。

[Q636 · 八股文 · 通用知识] 依据：参考资料：S9,S10

29. 云端 Agent 的性能、资源成本、隔离和安全如何权衡?

参考答案: 按信任和风险分级隔离：低风险只读调用可共享服务，高风险代码使用独立受限环境。Serverless 通常便于弹性与托管，代价是冷启动、时限及运行环境限制；按任务创建 Pod 更便于镜像、资源和网络定制，但启动与运维成本更高。可用预热池和镜像缓存减延迟，同时严格清除上一任务状态。评估需比较冷/热启动 P95、资源单价、并发、隔离边界和清理可靠性，不能仅看平均启动时间。

[Q637 · 八股文 · 通用知识] 依据：参考资料：S10

30. 【Python 编码题】 编写一个协程示例。

参考答案：async def 定义协程函数，调用后得到协程对象，需 await、create_task 或由 asyncio.run 驱动。只有调度成任务才会与其他任务并发，连续 await 两个慢操作通常仍串行；可用 gather 或 TaskGroup 组织独立任务。asyncio.sleep(0) 会暂停当前任务，让其他就绪任务有机会运行，但不保证公平顺序，也不会把 CPU 密集代码变成并行。取消和超时需传播，finally 释放资源；示例见附录 I。

[Q638 · 八股文 · 通用知识] 依据：参考资料：S4

31. 【Java 编码题】 给出两种单例实现方案。

参考答案：枚举单例由类初始化机制保证单实例及线程安全，通常也能避免反序列化创建新实例。双重检查锁先在锁外判空，进入类级锁后再次判空，首次创建后复用；instance 必须声明 volatile，保证引用可见性和安全发布。只加锁而不处理不安全发布不能保证锁外读正确。两者通常限定在同一类加载器范围，不能当作跨进程全局单例；有依赖注入框架时可优先用容器管理生命周期。参考代码见附录 J。

[Q639 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

32. 【Java 编码题•追问】 双重检查锁为什么需要 volatile，锁应该加在哪里?

参考答案：枚举单例由类初始化机制保证单实例及线程安全，通常也能避免反序列化创建新实例。双重检查锁先在锁外判空，进入类级锁后再次判空，首次创建后复用；instance 必须声明 volatile，保证引用可见性和安全发布。只加锁而不处理不安全发布不能保证锁外读正确。两者通常限定在同一类加载器范围，不能当作跨进程全局单例；有依赖注入框架时可优先用容器管理生命周期。参考代码见附录 J。

[Q640 · 八股文 · 通用知识] 依据：通用原理与答题方法；不表示项目已实现

33. 平时是否在本机使用 Codex 等 AI Coding 工具?

参考答案：AI 负责样板、测试草案和局部重构，人对架构、接口、安全和最终合并负责。多人协作按模块/接口拆分，小批次提交，强制代码审查、静态检查、测试和依赖扫描。统计比例不如说明哪些结果由人验证、出现错误怎样发现更有意义。

[Q641 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

34. 项目代码由谁编写，AI Coding 占多大比例?

参考答案：AI 负责样板、测试草案和局部重构，人对架构、接口、安全和最终合并负责。多人协作按模块/接口拆分，小批次提交，强制代码审查、静态检查、测试和依赖扫描。统计比例不如说明哪些结果由人验证、出现错误怎样发现更有意义。

[Q642 · 个人经历答题模板] 依据：按本人真实经历补充；不代填职级、规模、贡献或效果数字

编码题参考实现

附录 A：Two Sum（独立算法，工程未使用）

先查询再记录当前元素，支持重复数值且不会使用同一个下标两次。

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return None
```

附录 B：原地合并有序数组（独立算法）

a 必须预留 m+n 长度。当前。

```
def merge_sorted(a, m, b, n):
    i, j, k = m - 1, n - 1, m + n - 1
    while j >= 0:
        if i >= 0 and a[i] > b[j]:
            a[k] = a[i]
            i -= 1
        else:
            a[k] = b[j]
            j -= 1
        k -= 1
```

附录 C：受限 ReAct Loop（扩展示意）

这是伪代码，不是现有 Engine。保存健康事实必须调用原有校验服务；未知副作用不能盲目重试。

```
def bounded_loop(state, model, tools, memory, limit=8):
    for turn in range(limit):
        context = memory.read_for(state)
        decision = model.decide(state, context, tools.schemas())
        if decision.kind == "final":
            validate_completion(state, decision)
            return decision.answer
        tool = tools.authorize(decision.name, state.identity)
        args = tool.validate(decision.arguments)
        key = stable_operation_key(state.run_id, turn)
        result = tool.execute(args, idempotency_key=key)
        state = checkpoint_result(state, key, result)
    raise RuntimeError("task budget exhausted")
```

附录 D：Plan-and-Execute：两城市温差（注入工具的示例）

weather 是注入的只读工具，没有实际天气服务。实际工程计划是健康目标，不是自主任务 Planner。

```
def weather_plan():
    return [
        {"id": "a", "tool": "weather", "city": "Shanghai"},
        {"id": "b", "tool": "weather", "city": "Beijing"},
        {"id": "d", "compute": "difference", "deps": ["a", "b"]},
    ]

def execute_weather(plan, weather):
    results = {}
    for step in plan:
        if step.get("tool") == "weather":
            value = weather(step["city"])
            if type(value) not in (int, float):
                raise ValueError("invalid temperature")
            results[step["id"]] = value
        else:
            a, b = step["deps"]
            results[step["id"]] = results[a] - results[b]
    return results
```

附录 E：重复元素重排且相邻不同

示例假设元素可比较。将刚取出的元素延迟放回，防止连续使用。

```
from collections import Counter
import heapq

def rearrange(nums):
    counts = Counter(nums)
    if max(counts.values(), default=0) > (len(nums) + 1) // 2:
        return None
    heap = [(-c, value) for value, c in counts.items()]
    heapq.heapify(heap)
    answer = []
    previous = (0, None)
    while heap:
        count, value = heapq.heappop(heap)
        answer.append(value)
        if previous[0] < 0:
            heapq.heappush(heap, previous)
        previous = (count + 1, value)
    return answer
```

附录 F：二叉树层序遍历

标准树遍历，未补造原面试中的输入条件。LangGraph 调度不是二叉树 BFS。

```
from collections import deque

def level_order(root):
    if root is None:
        return []
    queue, result = deque([root]), []
    while queue:
        layer = []
        for _ in range(len(queue)):
            node = queue.popleft()
            layer.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(layer)
    return result
```

附录 G：LRU Cache

这是单线程示例。若缓存。

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        if capacity < 0:
            raise ValueError("capacity must be nonnegative")
        self.capacity = capacity
        self.data = OrderedDict()

    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)
        return self.data[key]

    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        while len(self.data) > self.capacity:
            self.data.popitem(last=False)
```

附录 H：线程池执行流程（伪代码）

完整实现还需关闭、取消、队列边界和异常隔离。

```
submit(task):
    if state != RUNNING: reject(task)
    else if worker_count < core: start_worker(task)
    else if queue.offer(task): ensure_worker_exists()
    else if worker_count < maximum: start_worker(task)
    else: reject(task)

worker_loop():
    while can_run_or_drain():
        task = blocking_take()
        try: complete_future(task.run())
        catch error: fail_future(error)
        finally: update_metrics()
```

附录 I：Python 协程

gather 返回结果顺序与参数顺序一致。

```
import asyncio

async def fetch(name, delay):
    await asyncio.sleep(delay)
    return name + " done"

async def main():
    values = await asyncio.gather(
        fetch("a", 0.02), fetch("b", 0.01)
    )
    return values

if __name__ == "__main__":
    print(asyncio.run(main()))
```

附录 J：Java 两种单例

volatile 保障安全发布；锁住类级初始化并在锁内再次判空。

```
public enum Singleton {
    INSTANCE;
}

final class DclSingleton {
    private static volatile DclSingleton instance;
    private DclSingleton() {}

    public static DclSingleton getInstance() {
        if (instance == null) {
            synchronized (DclSingleton.class) {
                if (instance == null) {
                    instance = new DclSingleton();
                }
            }
        }
        return instance;
    }
}
```

通用知识参考资料

S1 • ReAct 原始论文
https://arxiv.org/abs/2210.03629

S2 • RAG 原始论文
https://arxiv.org/abs/2005.11401

S3 • MCP 架构
https://modelcontextprotocol.io/docs/learn/architecture

S4 • Python 协程与任务
https://docs.python.org/3/library/asyncio-task.html

S5 • Python 线程与 GIL
https://docs.python.org/3/library/threading.html

S6 • LangGraph 持久化
https://docs.langchain.com/oss/python/langgraph/persistence

S7 • SSE 与 Last-Event-ID 标准
https://html.spec.whatwg.org/multipage/server-sent-events.html

S8 • SQLite 事务
https://www.sqlite.org/lang_transaction.html

S9 • OWASP Prompt Injection 防护
https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html

S10 • Kubernetes Pod 安全标准
https://kubernetes.io/docs/concepts/security/pod-security-standards/

S11 • RRF 排名融合公式
https://www.elastic.co/docs/reference/elasticsearch/rest-apis/reciprocal-rank-fusion

S12 • Java Thread.sleep 与 yield
https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html